

---

# Flex2 SDK 用户手册 v2.0

杭州曦诺未来科技有限公司

2026 年 08 月 13 日

|           |                      |          |
|-----------|----------------------|----------|
| <b>1</b>  | <b>概述</b>            | <b>1</b> |
| <b>2</b>  | <b>免责声明</b>          | <b>2</b> |
| <b>3</b>  | <b>快速开始</b>          | <b>4</b> |
| <b>4</b>  | <b>文档结构</b>          | <b>5</b> |
| 4.1       | 示例                   | 5        |
| 4.1.1     | 简介                   | 5        |
| 4.1.2     | 可以学到什么               | 5        |
| 4.1.3     | 按模块划分的示例             | 6        |
| 4.1.3.1   | 通信 (Communication)   | 6        |
| 4.1.3.1.1 | 初始化通信管理器             | 6        |
| 4.1.3.1.2 | 注册 CAN 驱动            | 9        |
| 4.1.3.1.3 | 升级固件                 | 13       |
| 4.1.3.2   | 手部设备 (Hand Device)   | 14       |
| 4.1.3.2.1 | 运行灵巧手设备              | 14       |
| 4.1.3.2.2 | 产品信息                 | 16       |
| 4.1.3.2.3 | 关节控制示例               | 18       |
| 4.1.3.2.4 | 异步关节控制               | 19       |
| 4.1.3.2.5 | 触觉传感                 | 21       |
| 4.1.3.2.6 | 触觉敏感元感知              | 23       |
| 4.1.3.2.7 | 接近觉测量                | 28       |
| 4.1.3.2.8 | 错误状态                 | 30       |
| 4.1.3.3   | 日志 (Logging)         | 33       |
| 4.1.3.3.1 | 创建日志实例               | 33       |
| 4.1.3.4   | C API                | 36       |
| 4.1.3.4.1 | 灵巧手控制 (C API)        | 36       |
| 4.1.4     | 完整示例 (Full Examples) | 38       |
| 4.1.4.1   | 灵巧手—伺服控制             | 38       |
| 4.1.4.1.1 | 概述                   | 38       |
| 4.1.4.1.2 | 功能                   | 38       |
| 4.1.4.1.3 | 前置条件                 | 39       |
| 4.1.4.1.4 | 执行流程                 | 39       |
| 4.1.4.1.5 | 源代码                  | 39       |
| 4.1.5     | 使用说明                 | 43       |
| 4.2       | API 参考               | 43       |

|           |                  |     |
|-----------|------------------|-----|
| 4.2.1     | 概述               | 43  |
| 4.2.2     | API 分类           | 43  |
| 4.2.2.1   | C API            | 43  |
| 4.2.2.1.1 | C API 参考         | 44  |
| 4.2.2.1.2 | 通信模块 (C API)     | 44  |
| 4.2.2.1.3 | CAN 传输模块 (C API) | 51  |
| 4.2.2.1.4 | 手部控制模块 (C API)   | 56  |
| 4.2.2.1.5 | 灵巧手 (C API)      | 57  |
| 4.2.2.1.6 | 基础设施 (C API)     | 77  |
| 4.2.2.1.7 | 日志 (C API)       | 88  |
| 4.2.2.1.8 | 错误码              | 97  |
| 4.2.2.2   | C++ API          | 100 |
| 4.2.2.2.1 | C++ API 参考       | 100 |
| 4.2.2.2.2 | 通信 (C++ API)     | 101 |
| 4.2.2.2.3 | CAN 通信 (C++ API) | 107 |
| 4.2.2.2.4 | 手部控制 (C++ API)   | 109 |
| 4.2.2.2.5 | 灵巧手 (C++ API)    | 110 |
| 4.2.2.2.6 | 基础设施 (C++ API)   | 120 |
| 4.2.2.2.7 | 日志 (C++ API)     | 121 |
| 4.2.2.2.8 | 值类型 (C++ API)    | 128 |
| 4.2.3     | 说明               | 132 |
| 4.2.4     | 使用               | 132 |
| 4.3       | 术语               | 132 |
| 5         | 附加资源             | 134 |
|           | 索引               | 135 |

**Xynova SDK** 提供用于控制和监控 Xynova 灵巧手设备的公共 C 与 C++ 接口。它结合了稳定的 C ABI 与现代 C++ 封装，支持通信、设备控制、日志记录以及 CAN 后端集成。

SDK 的功能主要分为以下几类：

- **通信**：管理器、主/设备生命周期以及 CAN 传输设置。
- **灵巧手控制**：支持共享的灵巧手 API 以及 Dexterous Hand 设备封装。
- **基础设施**：日志、错误处理和通用值辅助函数。
- **驱动**：SocketCAN 与厂商特定 CAN 后端。

本 SDK 适用于 Linux (x86\_64 / aarch64) 平台，通过 CAN 通信 (SocketCAN 或厂商驱动，如 ChuangXinCAN) 控制 Xynova 灵巧手设备；如需实时调度 (SCHED\_FIFO)，需要 root 权限或 CAP\_SYS\_NICE。

---

### 免责声明

---

感谢您购买本公司的灵巧手产品。为保障您的权益，请您在使用产品前，仔细阅读并充分理解本声明中的全部内容，特别是以粗体、下划线等显著方式提示的条款。**您开始使用本产品，即视为您已阅读、理解并同意接受本声明的约束。若您不同意本声明中的任何内容，请勿使用本产品。**

如本声明的任何条款被认定无效、违法或不可执行，不影响其他条款的有效性与可执行性。

#### 一、使用规范与责任承担

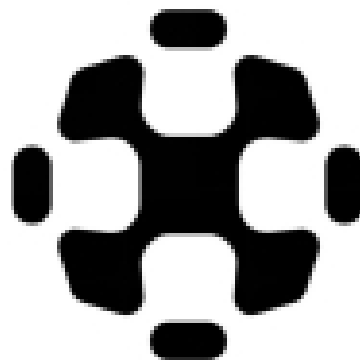
您应严格遵守随附的《用户手册》及相关法律法规，在安全的预期环境中安装和使用本产品。

您应对因您的下述行为（包括但不限于）所直接造成的任何损失、损害或责任自行负责：

1. 未按照《用户手册》的要求进行安装、使用、维护或存放；
2. 擅自对产品进行任何形式的改装、拆卸或修理；
3. 将产品用于非设计预期的用途或环境；
4. 将本产品与非我司生产、指定或认可的配件、零部件或耗材结合使用；
5. 其他明显疏忽或故意的不当行为。

在法律允许的最大范围内，对于因上述用户行为导致的任何损失或损害，杭州曦诺未来科技有限公司不承担责任。但本声明的任何内容均不排除或限制我司在如下方面的责任：(a) 因我司产品存在《中华人民共和国产品质量法》等法律法规定义的缺陷而导致的人身伤亡或财产损失；(b) 适用法律明确规定不得排除或限制的责任。

## 二、知识产权声明

The logo for Xynova, featuring the word "Xynova" in a bold, sans-serif font.The logo for Xynova, featuring a stylized gear icon to the left of the word "Xynova" and the Chinese text "曦诺未来" above it.

Xynova 是杭州曦诺未来科技有限公司的注册商标。本声明及产品包装上出现的其他产品名称、品牌标志等, 均为其各自权利人所有的知识产权。

本产品及其相关技术方案受中华人民共和国专利、外观设计等知识产权法律的保护。本声明及随附《用户手册》的全部内容（包括但不限于文字、图表、图像）的版权归杭州曦诺未来科技有限公司。未经我司事先书面许可，任何单位或个人不得以任何形式（包括但不限于复制、翻印、翻译、改编等）对上述内容进行商业性使用或传播。

杭州曦诺未来科技有限公司

1. 从公共 SDK 接口开始，了解通信、设备控制和日志相关内容。
2. 在应用中包含相关头文件：

```
#include "xynova/cpp/communication/Manager.hpp"  
#include "xynova/cpp/hand/dexterous_hand/Device.hpp"  
#include "xynova/cpp/infra/log/LogManager.hpp"
```

#### 备注

C++ API 需要支持 C++17 的编译器。

3. 初始化通信管理器，并配置主设备/从设备对。
4. 启动设备，并使用支持的同步或异步控制 API。
5. 预先配置日志，以便在运行时获得诊断信息。

完整可运行的工作流请参考[示例](#) 章节；可先从[运行灵巧手设备](#) 了解管理器初始化和设备启动，也可以查看[端到端示例灵巧手—伺服控制](#)。

文档按 SDK 使用示例与 API 参考组织:

## 4.1 示例

### 4.1.1 简介

本部分汇集了与仓库中通信（communication）、手部控制（hand-control）与基础设施（infrastructure）模块实现相对应的实用示例。每个示例均聚焦于单一主题，便于扩展为更完整的应用程序。

### 4.1.2 可以学到什么

示例涵盖以下主题:

- 初始化通信管理器并配置 CAN 主站（master）
- 注册自定义 CAN 驱动
- 升级设备固件
- 连接并运行灵巧手设备
- 读取产品信息（型号、序列号、版本、手型）
- 使用同步与异步控制接口
- 读取区域级与敏感元（taxel）级触觉反馈
- 读取接近觉（DToF）距离测量
- 监控设备错误状态（全局告警/故障与关节错误）
- 配置并使用日志系统
- 使用 C API 完成完整控制流程
- 走查完整的伺服控制工作流

### 4.1.3 按模块划分的示例

#### 4.1.3.1 通信 (Communication)

##### 4.1.3.1.1 初始化通信管理器

本示例演示如何使用 C++ API 初始化 **Xynova 通信管理器**，并创建通过 CAN 连接的手部设备。

#### 工作流程

初始化包括以下步骤：

1. 配置 CAN 主站
2. 配置从站与设备
3. 将其注册到管理器
4. 初始化并启动主站
5. 获取设备句柄

#### 配置说明

##### 主站 (Master)

- name —主站 (*master*) 的逻辑标识
- cpu\_index —CPU 绑定。使用 -1 禁用绑定 (仅限 root)
- period\_us —控制循环周期 (μs)
- comm.type —当前仅支持 CAN
- **comm.sub\_type** —驱动后端
  - SockCAN : Linux SocketCAN
  - ChuangXinCAN : 厂商驱动
  - 也可注册用户自定义驱动
- **iface** —物理 CAN 接口标识
  - 对于 SockCAN，指定 Linux SocketCAN 接口名，如 can0 或 can1。
  - 对于 ChuangXinCAN，指定厂商 CAN 设备/通道标识，如 0.0 或 0.1。
- bitrate / dbitrage —仲裁位速率 / 数据位速率 (CAN-FD)
- sample\_point / dsample\_point —位时序采样点
- manager\_can —SocketCAN 自动上电/断电所需 (仅限 root)

##### 从站 (Slave)

- 表示物理硬件 (当前为灵巧手从站 (*slave*))
- type —从站实现名称，如 DexterousHandSlave
- id —从站标识，0xFFFFFFFF (全 F)
- 提供两种时序配置：

### 通用配置 (General profile)

用于常规配置 API (getter/setter)

- user\_max\_timeout\_us 一用户侧最大超时
- user\_poll\_interval\_us 一等待时的轮询间隔
- execute\_timeout\_us 一后端执行超时
- execute\_poll\_interval\_us 一后端重试间隔
- execute\_max\_retry 一最大重试次数

### 快速配置 (Fast profile)

用于高频控制 API

- 字段同上，但针对低延迟优化

### 设备 (Device)

- 绑定以下对象的逻辑高级`device` 封装: - 一个主站 - 一个或多个从站

### 示例代码

```
using namespace xynova;
using namespace xynova::communication;

#define CHECK_EC(expr) \
do { \
    ec = (expr); \
    if (ec != XYNOVA_ERROR_CODE_OK) { \
        std::cerr << "[ERROR] " << #expr \
        << " failed, ec=" << ec << std::endl; \
        return -1; \
    } \
} while (0)

// -----
// Configure master (CAN)
// -----

MasterConfig master{};
master.name = "test_master";
master.cpu_index = -1;
master.period_us = 1000;

master.comm.type = CommType::Can;
master.comm.sub_type = "ChuangXinCAN";

master.comm.can.iface = "0.0";
master.comm.can.bitrate = 1'000'000;
master.comm.can.dbitrate = 5'000'000;
master.comm.can.sample_point = 0.75;
master.comm.can.dsampling_point = 0.75;

// -----
// Configure slave
// -----
```

(续下页)

(接上页)

```

SlaveConfig slave{};
slave.type = "DexterousHandSlave";
slave.id   = static_cast<uint32_t>(-1); // ID = 0xFFFFFFFF (all Fs)

// General timing
slave.general_time_setting.user_max_timeout_us   = 50 * 1000;
slave.general_time_setting.user_poll_interval_us = 1 * 1000;
slave.general_time_setting.execute_timeout_us    = 5 * 1000;
slave.general_time_setting.execute_poll_interval_us = 10 * 1000;
slave.general_time_setting.execute_max_retry     = 3;

// Fast timing (control path)
slave.fast_time_setting.user_max_timeout_us      = 5 * 1000;
slave.fast_time_setting.user_poll_interval_us    = 100;
slave.fast_time_setting.execute_timeout_us       = 3 * 1000;
slave.fast_time_setting.execute_poll_interval_us = 3 * 1000;
slave.fast_time_setting.execute_max_retry        = 0;

// -----
// Configure device
// -----

DeviceConfig device_cfg{};
device_cfg.type      = "DexterousHandDevice";
device_cfg.name      = "test_hand";
device_cfg.master_name = master.name;
device_cfg.slaves.push_back(slave);

// -----
// Configure manager
// -----

ManagerConfig cfg;
cfg.masters.push_back(master);
cfg.devices.push_back(device_cfg);

// -----
// Initialize and run
// -----

XynovaErrorCode ec;

auto& mgr = Manager::instance();

CHECK_EC(mgr.init(cfg));
CHECK_EC(mgr.run(master.name));

// -----
// Acquire device handle
// -----

auto hand = mgr.getDevice<xynova::hand::dexterous_hand::Device>(
    master.name,
    device_cfg.type,
    device_cfg.name);

```

(续下页)

(接上页)

```

if (!hand || !hand->valid())
{
    std::cerr << "[ERROR] device invalid" << std::endl;
    return -1;
}

```

## 说明

- 管理器拥有进程内的所有主站与设备。
- 使用设备前务必验证 `device->valid()`。
- Manager 采用单例模式。

### 4.1.3.1.2 注册 CAN 驱动

#### 概述

Xynova SDK 通过驱动抽象层支持 **可插拔的 CAN 后端**。

每个 CAN 后端实现底层通信逻辑（如 SocketCAN、USB-CAN、PCIe-CAN、厂商 SDK 或仿真）。

驱动在运行时通过以下接口注册：

```
XynovaCommunicationCan_registerDriver()
```

这使通信管理器能够按字符串类型动态选择 CAN 驱动。

典型应用场景：

- 自定义硬件适配器
- 厂商专用 CAN 库
- 仿真后端
- 单元测试 / mock 驱动

#### 架构

驱动集成流程如下：

1. 实现 `XynovaCommunicationCanVTable`
2. 提供全部必需回调
3. 以唯一类型名注册驱动
4. 在 `MasterConfig.comm.sub_type` 中使用该类型名

```

SDK Manager

    lookup("socketcan")

    VTable → create/open/send/recv

```

## 何时实现自定义驱动

在以下需求下创建自定义驱动:

- 非 SocketCAN 硬件
- 专有厂商 SDK
- 虚拟 CAN / 仿真器
- 无需真实总线的确定性测试
- 嵌入式或 RTOS 移植

## 示例：最小化 SocketCAN 驱动

以下示例演示如何实现并注册一个基础 SocketCAN 后端。

### 第 1 步—实现驱动回调

```
#include <unistd.h>
#include <cstring>
#include <sys/socket.h>
#include <linux/can.h>
#include <linux/can/raw.h>
#include <net/if.h>
#include <sys/ioctl.h>

#include "xynova/c/can_api.h"

struct SocketCanHandle
{
    int fd{-1};
};

static void* sc_create(const XynovaCommunicationCanConfig* cfg)
{
    auto* h = new SocketCanHandle{};
    return h;
}

static void sc_destroy(void* handle)
{
    delete static_cast<SocketCanHandle*>(handle);
}

static XynovaErrorCode sc_open(void* handle)
{
    auto* h = static_cast<SocketCanHandle*>(handle);

    h->fd = socket(PF_CAN, SOCK_RAW, CAN_RAW);
    if (h->fd < 0)
        return XYNOVA_ERROR_CODE_CAN_OPEN_FAILED;

    return XYNOVA_ERROR_CODE_OK;
}
```

(续下页)

(接上页)

```

}

static XynovaErrorCode sc_close(void* handle)
{
    auto* h = static_cast<SocketCanHandle*>(handle);

    if (h->fd >= 0)
        close(h->fd);

    return XYNOVA_ERROR_CODE_OK;
}

static bool sc_isOpen(void* handle)
{
    auto* h = static_cast<SocketCanHandle*>(handle);
    return h->fd >= 0;
}

static XynovaErrorCode sc_send(
    void* handle,
    XynovaCommunicationCanFrameType,
    uint32_t id,
    const uint8_t* data,
    size_t len,
    uint32_t)
{
    auto* h = static_cast<SocketCanHandle*>(handle);

    struct can_frame frame{};
    frame.can_id = id;
    frame.can_dlc = len;
    std::memcpy(frame.data, data, len);

    write(h->fd, &frame, sizeof(frame));

    return XYNOVA_ERROR_CODE_OK;
}

static XynovaErrorCode sc_recv(
    void* handle,
    XynovaCommunicationCanFrameType*,
    uint32_t* id,
    uint8_t* data,
    size_t* len,
    uint32_t)
{
    auto* h = static_cast<SocketCanHandle*>(handle);

    struct can_frame frame{};
    read(h->fd, &frame, sizeof(frame));

    *id = frame.can_id;

```

(续下页)

(接上页)

```

    *len = frame.can_dlc;
    std::memcpy(data, frame.data, *len);

    return XYNova_ERROR_CODE_OK;
}

static XynovaErrorCode sc_setFilters(
    void*, uint32_t, const XynovaCommunicationCanRecvFilter*)
{
    return XYNova_ERROR_CODE_OK;
}

```

## 第 2 步—创建 VTable

```

static XynovaCommunicationCanVTable g_socketcan_vtable = {
    .create = sc_create,
    .destroy = sc_destroy,
    .open = sc_open,
    .close = sc_close,
    .isOpen = sc_isOpen,
    .setRecvFilters = sc_setFilters,
    .send = sc_send,
    .recv = sc_recv
};

```

## 第 3 步—注册驱动

```

XynovaCommunicationCan_registerDriver(
    "socketcan",
    &g_socketcan_vtable);

```

## 第 4 步—在管理器中使用驱动

```

master_cfg.comm.type = CommType::Can;
master_cfg.comm.sub_type = "socketcan";    // match registered name

```

## 最佳实践

- 实时系统使用 **one-shot** 模式
- 避免在 send/recv 中阻塞
- 尽可能保持回调无锁
- 在驱动层处理超时
- 始终校验缓冲区大小

## 总结

集成新的 CAN 后端:

1. 实现回调
2. 填充 XynovaCommunicationCanVTable
3. 使用 registerDriver() 注册
4. 在配置中按名称引用驱动

SDK 将自动将所有 CAN 操作路由到你的驱动。

### 4.1.3.1.3 升级固件

本示例演示如何通过通信管理器升级从站设备的固件，使用：

- Manager::upgradeFirmware(master\_name, slave\_id, file)

## 前置条件

- 开始升级前，管理器必须已初始化且主站正在运行（参见[初始化通信管理器](#)）。

## 示例代码

```
using namespace xynova;
using namespace xynova::communication;

#define CHECK_EC(expr) \
do { \
    ec = (expr); \
    if (ec != XYNova_ERROR_CODE_OK) { \
        std::cerr << "[ERROR] " << #expr \
        << " failed, ec=" << ec << std::endl; \
        return -1; \
    } \
} while (0)

XynovaErrorCode ec;

// =====
// Manager must be initialized and running first
// (see the "Initialize Communication Manager" example)
// =====

auto& mgr = Manager::instance();

// =====
// Upgrade the firmware of the target slave
// =====

const uint32_t slave_id = static_cast<uint32_t>(-1); // ID = 0xFFFFFFFF
const std::string firmware_file = "/path/to/firmware.bin";

CHECK_EC(mgr.upgradeFirmware(
    master.name,          // master name
    slave_id,             // target slave identifier
    firmware_file));      // firmware file path
```

## 说明

- `master_name` 必须与已配置且正在运行的主站名称一致。
- `slave_id` 指定目标从站；请按实际设备配置调整。
- 固件文件格式及升级期间设备的行为以设备侧定义为准。
- 完整的管理器初始化与配置流程参见[初始化通信管理器](#)。

### 4.1.3.2 手部设备 (Hand Device)

#### 4.1.3.2.1 运行灵巧手设备

本指南介绍如何通过通信管理器启动与停止 **灵巧手设备**。

重点介绍当前可供灵巧手用户使用的接口：

- `run()` / `stop()`
- 句柄校验 `valid()`

## 工作流程

1. 配置 CAN 主站与从站时序参数
2. 注册设备
3. 初始化并运行管理器
4. 获取设备句柄
5. 使用 `run()` 启动手部设备通信
6. 使用 `stop()` 停止

## 配置说明

完整的管理器/主站/从站配置（含 `general` 与 `fast` 时序配置）参见[初始化通信管理器](#)。本示例复用该配置，仅突出设备启动流程。

## 示例代码

```
using namespace xynova;
using namespace xynova::communication;

#define CHECK_EC(expr) \
do { \
    ec = (expr); \
    if (ec != XYNOVA_ERROR_CODE_OK) { \
        std::cerr << "[ERROR] " << #expr \
        << " failed, ec=" << ec << std::endl; \
        return -1; \
    } \
} while (0)

// =====
// Configure master / slave / device / manager
// (see the "Initialize Communication Manager" example)
// =====
```

(续下页)

(接上页)

```

MasterConfig master{};
master.name = "test_master";
master.cpu_index = -1;
master.period_us = 1000;

master.comm.type = CommType::Can;
master.comm.sub_type = "ChuangXinCAN";

master.comm.can.iface          = "0.0";
master.comm.can.bitrate        = 1'000'000;
master.comm.can.dbitrate       = 5'000'000;
master.comm.can.sample_point   = 0.75;
master.comm.can.dsampling_point = 0.75;

SlaveConfig slave{};
slave.type = "DexterousHandSlave";
slave.id   = static_cast<uint32_t>(-1); // ID = 0xFFFFFFFF (all Fs)

DeviceConfig device_cfg{};
device_cfg.type      = "DexterousHandDevice";
device_cfg.name      = "test_hand";
device_cfg.master_name = master.name;
device_cfg.slaves.push_back(slave);

ManagerConfig cfg;
cfg.masters.push_back(master);
cfg.devices.push_back(device_cfg);

// =====
// Initialize and run
// =====

XynovaErrorCode ec;

auto& mgr = Manager::instance();

CHECK_EC(mgr.init(cfg));
CHECK_EC(mgr.run(master.name));

// =====
// Acquire and start device
// =====

auto hand = mgr.getDevice<xynova::hand::dexterous_hand::Device>(
    master.name,
    device_cfg.type,
    device_cfg.name);

if (!hand || !hand->valid())
{
    std::cerr << "[ERROR] device invalid" << std::endl;
    return -1;
}

CHECK_EC(hand->run());

```

(续下页)

(接上页)

```
// ... control the hand ...

CHECK_EC(hand->stop());
```

## 说明

- 管理器拥有所配置的主站与设备。
- 使用前务必检查 `device->valid()`。
- 当前灵巧手接口集限于上文列出的 API。
- 完整的管理器初始化与配置流程参见[初始化通信管理器](#)。

### 4.1.3.2.2 产品信息

本示例演示如何读取灵巧手设备的 **产品相关信息**，包括：

- `getProductModel()` — 产品型号字符串
- `getProductSN()` — 产品序列号
- `getHardwareVersion()` — 硬件版本字符串
- `getSoftwareVersion()` — 软件版本字符串
- `getHandType()` — 左手 / 右手手型

这些接口用于设备识别、版本跟踪以及在启动控制流程前进行兼容性检查。

## 工作流程

1. 通过通信管理器获取设备句柄
2. 使用 `run()` 启动设备
3. 读取产品型号与序列号
4. 读取硬件 / 软件版本字符串
5. 读取手型并映射为 `RIGHT / LEFT`

## 概念说明

使用以下设备方法：

- `getProductModel(std::string &model)`
- `getProductSN(std::string &sn)`
- `getHardwareVersion(std::string &version)`
- `getSoftwareVersion(std::string &version)`
- `getHandType(HandType &type)`

手型以 `HandType` 值上报：

- `XYNOVA_DEXTEROUS_HAND_TYPE_UNKNOWN` — 未知手型
- `XYNOVA_DEXTEROUS_HAND_TYPE_RIGHT` — 右手

- XYNOVA\_DEXTEROUS\_HAND\_TYPE\_LEFT 一左手

所有版本与型号字符串通过调用方提供的 `std::string` 缓冲区返回。

### 示例代码

```
using namespace xynova::hand::dexterous_hand;

#define CHECK_EC(expr) \
do { \
    ec = (expr); \
    if (ec != XYNOVA_ERROR_CODE_OK) { \
        std::cerr << "[ERROR] " << #expr \
            << " failed, ec=" << ec << std::endl; \
        return -1; \
    } \
} while (0)

XynovaErrorCode ec;

// =====
// Read product model and serial number
// =====

std::string product_model;
std::string product_sn;

CHECK_EC(device->getProductModel(product_model));
CHECK_EC(device->getProductSN(product_sn));

std::cout << "Product model : " << product_model << std::endl;
std::cout << "Serial number : " << product_sn << std::endl;

// =====
// Read hardware / software versions
// =====

std::string hw_version;
std::string sw_version;

CHECK_EC(device->getHardwareVersion(hw_version));
CHECK_EC(device->getSoftwareVersion(sw_version));

std::cout << "Hardware ver  : " << hw_version << std::endl;
std::cout << "Software ver  : " << sw_version << std::endl;

// =====
// Read hand type
// =====

HandType hand_type = XYNOVA_DEXTEROUS_HAND_TYPE_UNKNOWN;

CHECK_EC(device->getHandType(hand_type));

switch (hand_type)
{
case XYNOVA_DEXTEROUS_HAND_TYPE_RIGHT:
    std::cout << "Hand type      : RIGHT" << std::endl;
```

(续下页)

(接上页)

```

        break;
    case XYNOVA_DEXTEROUS_HAND_TYPE_LEFT:
        std::cout << "Hand type      : LEFT" << std::endl;
        break;
    default:
        std::cout << "Hand type      : UNKNOWN" << std::endl;
        break;
}

```

## 说明

- 在调用产品信息接口前，必须先获取并启动设备句柄。
- 始终检查返回的 `XynovaErrorCode`，以便发现通信失败或设备未就绪的情况。
- 完整的管理器初始化与设备启动流程参见[运行灵巧手设备](#)。

### 4.1.3.2.3 关节控制示例

本示例演示当前支持的灵巧手 **关节级控制**流程，包括：

- 设置控制模式
- 读取当前关节状态
- 构造关节角度命令
- 发送同步角度命令

## 工作流程

1. 将关节控制模式设置为 `ANGLE`。
2. 读取当前关节角度。
3. 构造新的 `AngleControl` 命令。
4. 使用 `setJointAngle()` 发送命令。

## 概念说明

当前实现暴露以下受支持的设备方法：

- `run() / stop()`
- `setJointControlMode(mode)`
- `setJointAngle(angle)`
- `setJointAngleAsync(angle)`
- `getJointAngle(states, timestamp)`

每个关节值存储在 `AngleControl` 或 `AngleStates` 对象中。测试代码使用简单循环为所有关节填充值。

## 示例代码

```
using namespace xynova::hand::dexterous_hand;

#define CHECK_EC(expr) \
do { \
    ec = (expr); \
    if (ec != XYNOVA_ERROR_CODE_OK) { \
        std::cerr << "[ERROR] " << #expr \
        << " failed, ec=" << ec << std::endl; \
        return -1; \
    } \
} while (0)

XynovaErrorCode ec;

ControlMode mode = XYNOVA_DEXTEROUS_HAND_CONTROL_MODE_ANGLE;

// setJointControlMode() applies to all joints
CHECK_EC(device->setJointControlMode(mode));

AngleStates states{};
uint64_t ts = 0;

CHECK_EC(device->getJointAngle(states, ts));

AngleControl pos{};
for (size_t i = 0; i < JOINT_NUM; ++i)
{
    pos[i].setId(static_cast<JointID>(i + 1));
    pos[i].setValue(0.5236f);
}

CHECK_EC(device->setJointAngle(pos));
```

## 说明

- 进行关节控制前，必须先获取并启动设备句柄。
- setJointControlMode() 作用于全部关节。
- 完整的管理器初始化与设备启动流程参见运行灵巧手设备。

### 4.1.3.2.4 异步关节控制

本示例演示当前支持的灵巧手异步控制流程。

当前实现提供以下相关方法：

- setJointControlMode(mode)
- setJointAngleAsync(angle)
- getJointAngle(states, timestamp)

本示例与仓库中的测试用例采用相同模式。

## 工作流程

1. 将关节控制模式设置为 ANGLE 。
2. 读取当前关节状态。
3. 构造新的 AngleControl 命令。
4. 使用 setJointAngleAsync() 发送。

## 示例代码

```
using namespace xynova::hand::dexterous_hand;

#define CHECK_EC(expr) \
do { \
    ec = (expr); \
    if (ec != XYNOVA_ERROR_CODE_OK) { \
        std::cerr << "[ERROR] " << #expr \
        << " failed, ec=" << ec << std::endl; \
        return -1; \
    } \
} while (0)

XynovaErrorCode ec;

ControlMode mode = XYNOVA_DEXTEROUS_HAND_CONTROL_MODE_ANGLE;

CHECK_EC(device->setJointControlMode(mode));

AngleControl angle;
for (size_t i = 0; i < JOINT_NUM; ++i)
{
    angle[i].setId(static_cast<JointID>(i + 1));
    angle[i].setValue(-0.3491f);
}

CHECK_EC(device->setJointAngleAsync(angle));

std::this_thread::sleep_for(std::chrono::seconds(1));

AngleStates states{};
uint64_t ts = 0;

CHECK_EC(device->getJointAngle(states, ts));
```

## 说明

- 进行关节控制前，必须先获取并启动设备句柄。
- setJointAngleAsync() 将命令入队后立即返回；执行由通信后端处理。
- 完整的管理器初始化与设备启动流程参见运行灵巧手设备。

#### 4.1.3.2.5 触觉传感

本示例演示如何读取灵巧手设备的 **触觉力 / 力矩反馈**，包括：

- `enableTactileWrench(bool)` 一启用或禁用触觉 **wrench** 上报
- `getTactileWrench(TactileWrenchStates &states, uint64_t &timestamp_us)` 一读取所有触觉区域的最近 **wrench** 数据

这些接口用于接触力监控、触觉反馈以及力控操作。

#### 工作流程

1. 通过通信管理器获取设备句柄
2. 使用 `run()` 启动设备
3. 使用 `enableTactileWrench(true)` 启用触觉 **wrench** 上报
4. 使用 `getTactileWrench()` 周期性读取 **wrench** 状态
5. 解析每个区域的力 / 力矩分量

#### 概念说明

灵巧手暴露 `TACTILE_NUM` (15) 个触觉区域。每个区域由 `TactileID` 值标识，例如：

- `XYNOVA_DEXTEROUS_HAND_TACTILE_ID_TT1` 一拇指指尖
- `XYNOVA_DEXTEROUS_HAND_TACTILE_ID_FT1` 一食指指尖
- `XYNOVA_DEXTEROUS_HAND_TACTILE_ID_PT1` 一手掌

按区域的 **wrench** 维度：

- **6D 区域** (`XYNOVA_DEXTEROUS_HAND_WRENCH_DIMENSION_6D`，完整力 + 力矩)：
  - `TT1` (拇指指尖)、`FT1` (食指指尖)、`MT1` (中指指尖)、`RT1` (无名指指尖)、`LT1` (小指指尖)、`PT1` (手掌)
- **1D 区域** (`XYNOVA_DEXTEROUS_HAND_WRENCH_DIMENSION_1D`，仅法向力 **Fz**)：
  - `TT2` (拇指指腹)
  - `FT2` (食指中节)、`FT3` (食指近节)
  - `MT2` (中指中节)、`MT3` (中指近节)
  - `RT2` (无名指中节)、`RT3` (无名指近节)
  - `LT2` (小指中节)、`LT3` (小指近节)

每个 `TactileWrench` 条目包含：

- `timestamp_us` 一采样时间戳 (微秒，系统单调时钟)
- `region` 一触觉区域标识
- `dimension` 一有效 **wrench** 维度，取值之一： - `XYNOVA_DEXTEROUS_HAND_WRENCH_DIMENSION_1D` (1) - `XYNOVA_DEXTEROUS_HAND_WRENCH_DIMENSION_3D` (3) - `XYNOVA_DEXTEROUS_HAND_WRENCH_DIMENSION_6D` (6)
- `wrench` 一6 分量力 / 力矩值数组

**wrench** 向量定义为：

```
[Fx, Fy, Fz, Mx, My, Mz]
```

其中  $F^*$  为力分量, 单位牛顿 (N);  $M^*$  为力矩分量, 单位牛·米 (Nm)。

### 备注

必须先调用 `enableTactileWrench(true)` 启用触觉 `wrench` 上报, `getTactileWrench()` 才会返回数据。

### 示例代码

```
using namespace xynova::hand::dexterous_hand;

#define CHECK_EC(expr) \
do { \
    ec = (expr); \
    if (ec != XYNOVA_ERROR_CODE_OK) { \
        std::cerr << "[ERROR] " << #expr \
        << " failed, ec=" << ec << std::endl; \
        return -1; \
    } \
} while (0)

XynovaErrorCode ec;

// =====
// Enable tactile wrench reporting
// =====

CHECK_EC(device->enableTactileWrench(true));

// =====
// Periodically read wrench states
// =====

TactileWrenchStates states{};
uint64_t ts = 0;

CHECK_EC(device->getTactileWrench(states, ts));

// =====
// Parse force / torque of each tactile region
// =====

for (size_t i = 0; i < TACTILE_NUM; ++i)
{
    const auto &w = states[i];

    const float fx = w.wrench.value()[0];
    const float fy = w.wrench.value()[1];
    const float fz = w.wrench.value()[2];
    const float mx = w.wrench.value()[3];
    const float my = w.wrench.value()[4];
    const float mz = w.wrench.value()[5];

    std::cout << "region=" << static_cast<uint32_t>(w.region)
```

(续下页)

(接上页)

```
<< " dim=" << static_cast<uint32_t>(w.dimension)
<< " F=( " << fx << ", " << fy << ", " << fz << " ) N"
<< " M=( " << mx << ", " << my << ", " << mz << " ) Nm"
<< std::endl;
}
```

说明

- 启用触觉 wrench 上报前，必须先获取并启动设备句柄。
- 必须先调用 enableTactileWrench(true)，wrench 数据才可用。
- 返回的 wrench 数组始终有 6 个分量；未使用的分量（取决于 dimension）应忽略。
- 对于 1D 区域，仅 wrench[0]（Fz，法向力）包含有效数据，其余分量应忽略；对于 6D 区域，6 个分量均有效。
- 完整的管理器初始化与设备启动流程参见运行灵巧手设备。

4.1.3.2.6 触觉敏感元感知

本示例演示如何读取灵巧手设备的 详细触觉敏感元（taxel）数据，包括：

- enableTactileTaxel(bool) 一启用或禁用触觉taxel 上报
- getTactileTaxel(TactileTaxelStates &states, uint64\_t &timestamp\_us) 一读取所有触觉区域的敏感元级力数据

与区域级的触觉传感（wrench）反馈不同，taxel 数据提供触觉阵列中 每个独立敏感元测得的力。

传感器阵列总览

全手触觉阵列由覆盖五指与手掌的 15 个模块组成，共 312 个 taxel：

| 区域 ID | 模块  | 部位    | 力类型 | Taxel 数 |
|-------|-----|-------|-----|---------|
| 0x01  | TT1 | 拇指指尖  | 3D  | 33      |
| 0x02  | TT2 | 拇指指腹  | 1D  | 12      |
| 0x03  | FT1 | 食指指尖  | 3D  | 24      |
| 0x04  | FT2 | 食指中节  | 1D  | 8       |
| 0x05  | FT3 | 食指近节  | 1D  | 8       |
| 0x06  | MT1 | 中指指尖  | 3D  | 24      |
| 0x07  | MT2 | 中指中节  | 1D  | 8       |
| 0x08  | MT3 | 中指近节  | 1D  | 8       |
| 0x09  | RT1 | 无名指指尖 | 3D  | 24      |
| 0x0A  | RT2 | 无名指中节 | 1D  | 8       |
| 0x0B  | RT3 | 无名指近节 | 1D  | 8       |
| 0x0C  | LT1 | 小指指尖  | 3D  | 19      |
| 0x0D  | LT2 | 小指中节  | 1D  | 8       |
| 0x0E  | LT3 | 小指近节  | 1D  | 8       |
| 0x0F  | PT1 | 手掌    | 3D  | 112     |

命名规则：T = 拇指，F = 食指，M = 中指，R = 无名指，L = 小指，P = 手掌；01 = 指尖，02 = 指节，03 = 近节。

左右手镜像

产品阵列共 **16 个模块 / 424 个 taxel**：手掌传感区由两个独立模块（左手掌与右手掌，各 112 个 taxel）提供。SDK 对当前连接的设备暴露 **15 个区域 / 312 个 taxel**，手掌通过单一 PT1 区域（0x0F）上报；右手掌模块不单独暴露。

空间布局

下图展示各模块内 taxel 的物理排列。编号采用产品侧 1 起始编号，用于在物理阵列上定位 taxel；与设备接口返回的 taxel\_id 值的对应关系由设备本身上报。

编号规则

| 模块类型                           | 起始 taxel | 编号方向      | 行优先顺序                   |
|--------------------------------|----------|-----------|-------------------------|
| 指尖模块（3D 力）                     | 右下角      | 自右向左、自下而上 | 从右下角开始，每行自右向左填充，然后移至上一行 |
| 指腹 / 指节 / 指根 / 手掌模块（1D 或 3D 力） | 左上角      | 自左向右、自上而下 | 从左上角开始，每行自左向右填充，然后移至下一行 |

指尖模块

TT1 拇指指尖（0x01，33 个 taxel，7 行 × 9 列网格）：

```
Row 1:      33 32 31
Row 2:      30 29 28
Row 3:      27 26 25 24
Row 4:      23 22 21 20 19
Row 5:      18 17 16 15 14 13
Row 6:      12 11 10  9  8  7
Row 7:       6  5  4  3  2  1  <- #1 bottom-right
```

FT1 / MT1 / RT1 食指 / 中指 / 无名指指尖（0x03 / 0x06 / 0x09，各 24 个 taxel，7 行 × 6 列网格）：

```
Row 1:      24 23
Row 2:      22 21 20
Row 3:      19 18 17 16
Row 4:      15 14 13 12
Row 5:      11 10  9  8
Row 6:       7  6  5  4
Row 7:       3  2  1  <- #1 bottom-right
```

LT1 小指指尖（0x0C，19 个 taxel，6 行 × 6 列网格）：

```
Row 1:      19 18
Row 2:      17 16 15
Row 3:      14 13 12
Row 4:      11 10  9  8
Row 5:       7  6  5  4
Row 6:       3  2  1  <- #1 bottom-right
```

## 指腹 / 指节 / 指根模块

TT2 拇指指腹 (0x02, 12 个 taxel, 2 行 × 6 列网格):

```
Row 1:  1  2  3  4  5  6
Row 2:  7  8  9 10 11 12  <- #1 top-left
```

指节 / 指根模块 (FT2 / FT3 / MT2 / MT3 / RT2 / RT3 / LT2 / LT3, 0x04 ~ 0x0E, 各 8 个 taxel, 2 行 × 4 列网格, 各模块布局相同):

```
Row 1:  1  2  3  4
Row 2:  5  6  7  8  <- #1 top-left
```

## 手掌模块

PT1 左手掌 (0x0F, 112 个 taxel, 20 行 × 17 列网格, . = 空网格位置):

```
      1  2  3  4  5  6  7  8  9  10 11 12 13 14 15 16 17  <- column
r 1:  .  .  .  .  1  2  .  .  .  .  .  .  .  .  .  .  .
r 2:  3  4  5  6  7  8  9  10 .  .  .  .  .  .  .  .
r 3: 11 12 13 14 15 16 .  .  .  .  .  .  .  .  .  .
r 4: 17 18 19 20 21 .  .  .  .  .  .  .  .  .  .  .
r 5:  . 22 23 24 25 .  .  .  .  .  .  .  .  .  .  .
r 6:  .  . 26 27 28 29 .  .  .  .  .  .  .  .  .  .
r 7:  .  .  .  . 30 31 .  .  .  .  .  .  .  .  .  .
r 8:  .  .  .  .  . 32 33 34 .  .  .  .  .  .  .  .
r 9:  .  .  .  .  .  . 35 36 .  .  .  .  .  .  .  .
r10: .  .  .  .  .  .  . 37 .  .  .  .  .  .  .  .
r11: .  .  .  .  .  .  . 38 39 .  .  .  .  .  .  .  .
r12: .  .  .  .  .  .  .  . 40 41 42 43 44 .  .  .  .
r13: .  .  .  .  .  .  .  . 45 46 47 48 49 .  .  .  .
r14: .  .  .  .  .  .  .  . 50 51 52 53 54 55 .  .  .  .
r15: .  .  .  .  .  .  .  . 56 57 58 59 60 61 62 .  .  .  .
r16: .  .  .  .  .  .  .  . 63 64 65 66 67 68 69 70 .  .  .  .
r17: .  .  .  .  .  .  .  . 71 72 73 74 75 76 77 78 79 .  .  .  .
r18: .  .  .  .  .  .  .  . 80 81 82 83 84 85 86 87 88 89 90 .  .  .  .
r19: .  .  .  .  .  .  .  . 91 92 93 94 95 96 97 98 99 100 101 .  .  .  .
r20: .  .  .  .  .  .  .  . 102 103 104 105 106 107 108 109 110 111 112 .  .  .  .
```

右手掌 (112 个 taxel, 20 行 × 17 列网格)。右手掌是左手掌的水平镜像: 每行在镜像列位置包含相同数量的 taxel, 行内自左向右编号 (#1 起始于右上区域):

```
      1  2  3  4  5  6  7  8  9  10 11 12 13 14 15 16 17  <- column
r 1:  .  .  .  .  .  .  .  .  .  .  .  1  2  .  .  .  .
r 2:  .  .  .  .  .  .  .  .  .  .  3  4  5  6  7  8  9  10
r 3:  .  .  .  .  .  .  .  .  .  .  .  11 12 13 14 15 16
r 4:  .  .  .  .  .  .  .  .  .  .  .  .  17 18 19 20 21
r 5:  .  .  .  .  .  .  .  .  .  .  .  .  22 23 24 25 .
r 6:  .  .  .  .  .  .  .  .  .  .  .  .  26 27 28 29 .
r 7:  .  .  .  .  .  .  .  .  .  .  .  .  30 31 .  .  .  .
r 8:  .  .  .  .  .  .  .  .  .  .  32 33 34 .  .  .  .
r 9:  .  .  .  .  .  .  .  .  .  35 36 .  .  .  .
r10: .  .  .  .  .  .  .  .  37 .  .  .  .
r11: .  .  .  .  .  .  .  38 39 .  .  .  .
r12: .  .  .  . 40 41 42 43 44 .  .  .  .
r13: .  .  .  . 45 46 47 48 49 .  .  .  .
r14: .  .  . 50 51 52 53 54 55 .  .  .  .
```

(续下页)

(接上页)

```

r15:  .  .  .  56 57 58 59 60 61 62 .  .  .  .  .  .  .
r16:  .  .  63 64 65 66 67 68 69 70 .  .  .  .  .  .  .
r17:  .  71 72 73 74 75 76 77 78 79 .  .  .  .  .  .  .
r18:  80 81 82 83 84 85 86 87 88 89 90 .  .  .  .  .  .  .
r19:  91 92 93 94 95 96 97 98 99 100 101 .  .  .  .  .  .  .
r20:  102 103 104 105 106 107 108 109 110 111 112 .  .  .  .  .  .  .

```

## 力数据

每个 **taxel** 通过 **wrench** 字段上报力值，该字段为固定 6 分量数组，单位 **N（牛顿）**：

- **1D taxel**（非指尖区域）：单个法向力分量，存于 **wrench[0]**；**wrench[1..5]** 为零。
- **3D taxel**（指尖与手掌区域）：三个正交分量 **wrench[0..2] = [Fx, Fy, Fz]**，其中 **Fz** 为法向力，**Fx/Fy** 为剪切分量；**wrench[3..5]** 为零。

**taxel** 反馈仅包含力数据——不含力矩（扭矩）分量。**6D wrench** 输出的 **Mx / My / Mz** 力矩（参见**触觉传感**）是区域级合力矩，而非单敏感元输出。

力由设备以原始幅值转换为牛顿上报；确切的量程与分辨率以设备侧定义为准。

## 工作流程

1. 通过通信管理器获取设备句柄
2. 使用 **run()** 启动设备
3. 使用 **enableTactileTaxel(true)** 启用触觉 **taxel** 上报
4. 使用 **getTactileTaxel()** 周期性读取 **taxel** 状态
5. 解析每个 **taxel** 的区域、维度与力分量
6. 完成后使用 **enableTactileTaxel(false)** 禁用上报

## 示例代码

```

using namespace xynova::hand::dexterous_hand;

#define CHECK_EC(expr) \
do { \
    ec = (expr); \
    if (ec != XYNova_ERROR_CODE_OK) { \
        std::cerr << "[ERROR] " << #expr \
        << " failed, ec=" << ec << std::endl; \
        return -1; \
    } \
} while (0)

XynovaErrorCode ec;

// =====
// Enable tactile taxel reporting
// =====

CHECK_EC(device->enableTactileTaxel(true));

```

(续下页)

(接上页)

```

// Give the device time to start publishing taxel data.
std::this_thread::sleep_for(std::chrono::milliseconds(100));

// =====
// Periodically read taxel states
//
// The C++ wrapper resizes `states` to the number of taxels
// reported by the device (up to TACTILE_TOTAL_TAXEL_NUM).
// =====

TactileTaxelStates states;
uint64_t ts = 0;

CHECK_EC(device->getTactileTaxel(states, ts));

// =====
// Parse force of each taxel
// =====

for (const auto &t : states)
{
    std::cout << "region=" << static_cast<uint32_t>(t.region)
                << " taxel=" << t.taxel_id
                << " dim=" << static_cast<uint32_t>(t.dimension);

    if (t.dimension == XYNOVA_DEXTEROUS_HAND_WRENCH_DIMENSION_1D)
    {
        std::cout << " Fz=" << t.wrench.value()[0];
    }
    else
    {
        std::cout << " F=(" << t.wrench.value()[0] << ", "
                    << t.wrench.value()[1] << ", "
                    << t.wrench.value()[2] << ") ";
    }

    std::cout << std::endl;
}

// =====
// Disable tactile taxel reporting
// =====

CHECK_EC(device->enableTactileTaxel(false));

```

## 说明

- 启用触觉 taxel 上报前，必须先获取并启动设备句柄。
- 必须先调用 enableTactileTaxel(true)，taxel 数据才可用。
- 若无新数据到达，getTactileTaxel() 返回 XYNOVA\_ERROR\_CODE\_DATA\_OUT\_OF\_DATE，但缓存数据仍会拷贝到输出缓冲区。
- 每个 taxel 带有各自的 timestamp\_us 采样时间戳（微秒，系统单调时钟）。
- taxel\_id 由设备上报提供。

- taxel 数据按区域 ID 升序返回。
- 完整的管理器初始化与设备启动流程参见运行灵巧手设备；区域级 wrench 反馈参见触觉传感。

#### 4.1.3.2.7 接近觉测量

本示例演示如何读取灵巧手设备的 **接近觉 (DToF) 距离测量**，包括：

- enableRange (bool) 一启用或禁用接近觉上报
- getRange (RangeStates &states, uint64\_t &timestamp\_us) 一读取最新的距离图 (range map)

这些接口用于接近检测、避障以及手势相关感知。

#### 距离图布局

距离数据以 RANGE\_POINT\_NUM (16) 个点构成的 4×4 二维图返回，按行优先存储：

- index = y \* width + x
- 每个点的 id 是遵循传输顺序的 1 起始序号 (id = index + 1)

传输顺序 (行优先，图中为 id)：

|        | col 0 | col 1 | col 2 | col 3 |                |
|--------|-------|-------|-------|-------|----------------|
| row 0: | id 1  | id 2  | id 3  | id 4  | <- index 0~3   |
| row 1: | id 5  | id 6  | id 7  | id 8  | <- index 4~7   |
| row 2: | id 9  | id 10 | id 11 | id 12 | <- index 8~11  |
| row 3: | id 13 | id 14 | id 15 | id 16 | <- index 12~15 |

物理传感器布局 (产品规格，第 0 行朝向指尖方向)：

```

+---+---+---+---+
| 0 | 4 | 8 | 12 | <- row 0 (top, toward fingertips)
+---+---+---+---+
| 1 | 5 | 9 | 13 | <- row 1
+---+---+---+---+
| 2 | 6 | 10 | 14 | <- row 2
+---+---+---+---+
| 3 | 7 | 11 | 15 | <- row 3 (bottom, toward wrist)
+---+---+---+---+

```

传输顺序与物理布局的映射关系：

- points[i] 位于 (row = i / 4, col = i % 4)
- points[i] 的物理像素编号为 (i % 4) \* 4 + (i / 4) (物理编号 = col \* 4 + row)
- API 返回的 id 是传输序号，**不是**物理像素编号

左右手套 (从手心方向观察) 的传感器安装方向一致，因此上述布局适用于左右手，无需镜像。

#### 数据布局

- RangeStates: width/height/count/points; SDK 当前上报 16 个点构成的 4×4 距离图。
- RangePoint 字段: - id 一 1 起始的点序号 - status 一 测量状态 (见下文) - distance 一 距离，单位米 (m) - timestamp\_us 一 时间戳 (微秒，系统单调时钟)

RangePointStatus 取值：

| 状态 (Status) | 含义 (Meaning) |
|-------------|--------------|
| NO_TARGET   | 未检测到目标       |
| WEAK_SIGNAL | 信号弱，测量不可靠    |
| VALID       | 有效测量         |
| INVALID     | 无效测量         |

工作流程

- 1. 通过通信管理器获取设备句柄
- 2. 使用 run() 启动设备
- 3. 使用 enableRange(true) 启用接近觉上报
- 4. 使用 getRange() 周期性读取距离图
- 5. 遍历网格，根据每个点的 status 与 distance 进行评估
- 6. 完成后使用 enableRange(false) 禁用上报

示例代码

```
using namespace xynova::hand::dexterous_hand;

#define CHECK_EC(expr) \
do { \
    ec = (expr); \
    if (ec != XYNOVA_ERROR_CODE_OK) { \
        std::cerr << "[ERROR] " << #expr \
            << " failed, ec=" << ec << std::endl; \
        return -1; \
    } \
} while (0)

XynovaErrorCode ec;

// =====
// Enable range reporting
// =====

CHECK_EC(device->enableRange(true));

// Give the device time to start publishing range data.
std::this_thread::sleep_for(std::chrono::milliseconds(100));

// =====
// Periodically read the range map
// =====

RangeStates states(RANGE_POINT_NUM);
uint64_t ts = 0;

CHECK_EC(device->getRange(states, ts));

// =====
// Iterate the grid in row-major order
```

(续下页)

(接上页)

```
// =====
for (uint16_t y = 0; y < states.height; ++y)
{
    for (uint16_t x = 0; x < states.width; ++x)
    {
        const auto &p = states.points[y * states.width + x];

        if (p.status == XYNOVA_DEXTEROUS_HAND_RANGE_POINT_STATUS_VALID)
        {
            std::cout << "(" << x << ", " << y << ") id=" << p.id
                << " distance=" << p.distance << " m"
                << std::endl;
        }
        else
        {
            std::cout << "(" << x << ", " << y << ") id=" << p.id
                << " status="
                << static_cast<uint32_t>(p.status)
                << std::endl;
        }
    }
}

// =====
// Disable range reporting
// =====

CHECK_EC(device->enableRange(false));
```

## 说明

- 启用接近觉上报前，必须先获取并启动设备句柄。
- 必须先调用 `enableRange(true)`，距离数据才可用。
- 若无新数据到达，`getRange()` 返回 `XYNOVA_ERROR_CODE_DATA_OUT_OF_DATE`，但缓存数据仍会拷贝到输出缓冲区。
- 点缓冲区由调用方提供；C++ 封装会自动管理缓冲区。
- 完整的管理器初始化与设备启动流程参见运行灵巧手设备。

### 4.1.3.2.8 错误状态

本示例演示如何读取灵巧手设备的 **设备错误状态**，包括：

- `getErrorState()` 一读取最新的全局告警、全局故障与关节错误信息
- 这些接口用于运行时健康监控、故障检测与安全停机处理。

#### 错误状态结构

错误状态通过 `ErrorState` 类型上报：

- `timestamp_us` 一错误状态时间戳（微秒）
- `warning_flags` 一全局告警标志

- `fault_flags` 一全局故障标志
- `joint_error_codes` 一每个关节的错误码（共 `JOINT_NUM` 项）

便捷辅助方法:

- `hasWarning()` 一存在任一全局告警时返回 `true`
- `hasFault()` 一存在任一全局故障时返回 `true`
- `joint(id)` 一查询指定关节的错误状态

每个关节错误码是由 `JointError` 枚举定义的一组位标志，多个错误可同时存在:

- `StallProtection` 一电机堵转保护
- `OverTemperature` 一过温保护
- `OverCurrent` 一过流保护
- `MotorAbnormal` 一电机异常
- `Communication` 一通信错误
- `EncoderAbnormal` 一编码器异常

## 工作流程

1. 通过通信管理器获取设备句柄
2. 使用 `run()` 启动设备
3. 使用 `getErrorState()` 读取错误状态
4. 检查全局告警 / 故障标志
5. 检查上报错误的关节的错误码

## 示例代码

```
using namespace xynova::hand::dexterous_hand;

#define CHECK_EC(expr) \
do { \
    ec = (expr); \
    if (ec != XYNova_ERROR_CODE_OK) { \
        std::cerr << "[ERROR] " << #expr \
        << " failed, ec=" << ec << std::endl; \
        return -1; \
    } \
} while (0)

XynovaErrorCode ec;

// =====
// Read the device error state
// =====

ErrorState state{};

CHECK_EC(device->getErrorState(state));
```

(续下页)

(接上页)

```

// =====
// Check global warning / fault flags
// =====

if (state.hasFault())
{
    std::cout << "Global fault : 0x"
                << std::hex << state.fault_flags << std::dec
                << std::endl;
}

if (state.hasWarning())
{
    std::cout << "Global warn : 0x"
                << std::hex << state.warning_flags << std::dec
                << std::endl;
}

// =====
// Inspect joint error codes (print only joints with errors)
// =====

for (size_t i = 0; i < JOINT_NUM; ++i)
{
    const auto &joint_error = state.joint_error_codes[i];

    if (joint_error.error_code.ok())
        continue;

    std::cout << "Joint "
                << static_cast<uint32_t>(joint_error.id)
                << " error:";

    if (joint_error.error_code.has(JointError::StallProtection))
        std::cout << " stall-protection";
    if (joint_error.error_code.has(JointError::OverTemperature))
        std::cout << " over-temperature";
    if (joint_error.error_code.has(JointError::OverCurrent))
        std::cout << " over-current";
    if (joint_error.error_code.has(JointError::MotorAbnormal))
        std::cout << " motor-abnormal";
    if (joint_error.error_code.has(JointError::Communication))
        std::cout << " communication";
    if (joint_error.error_code.has(JointError::EncoderAbnormal))
        std::cout << " encoder-abnormal";

    std::cout << std::endl;
}

if (!state.hasFault() && !state.hasWarning())
    std::cout << "No warning or fault reported." << std::endl;

```

## 说明

- 读取错误状态前，必须先获取并启动设备句柄。
- `warning_flags` 与 `fault_flags` 是设备上报的位标志；各比特位的含义以设备侧定义为准。
- 关节错误码是位标志，可能同时表示多个错误。
- 设备错误状态与 API 函数返回的 `XynovaErrorCode` 不同：前者描述设备健康状态，后者描述 API 调用本身的结果。
- 完整的管理器初始化与设备启动流程参见运行灵巧手设备。

### 4.1.3.3 日志 (Logging)

#### 4.1.3.3.1 创建日志实例

本示例演示如何配置 **Xynova 日志系统**，并为不同模块（主站与设备）挂接专用的实时 logger。

#### 日志架构

日志系统由以下部分组成：

- **Writer** → 输出目标（文件 / 控制台）
- **Sink** → 对多个 writer 分组
- **Logger** → 供模块用于输出日志消息

#### 使用场景

在需要以下能力时使用自定义 logger：

- 每个模块独立的日志文件
- 日志文件轮转
- 异步日志
- 高频日志且不阻塞控制线程

#### 工作流程

1. 初始化日志运行时
2. 创建文件 writer
3. 创建 logger 实例
4. 将 logger 名称绑定到主站/从站
5. 初始化并运行管理器
6. 停止主站通信循环

#### 示例代码

```
using namespace xynova::infra::log;
using namespace xynova::communication;

#define CHECK_EC(expr) \
```

(续下页)

(接上页)

```
do {
    ec = (expr);
    if (ec != XYNOVA_ERROR_CODE_OK) {
        std::cerr << "[ERROR] " << #expr
            << " failed, ec=" << ec << std::endl;
        return -1;
    }
} while (0)
```

---

XynovaErrorCode ec;

```
// =====
// 1. Initialize logging runtime (REQUIRED)
//     Must be called before creating any writers or loggers
// =====
```

XynovaLogGlobalLogger\_init(XYNOVA\_LOG\_LEVEL\_DEBUG, nullptr);  
LogManager::instance().initDefaultObjects();

```
// =====
// 2. Create file writers
//     Each writer corresponds to one output file
// =====
```

FileWriterConfiguration file\_conf{};  
file\_conf.base\_conf.formater = "default";  
file\_conf.base\_conf.auto\_flush = true;  
file\_conf.base\_conf.flush\_interval\_ms = 500;  
file\_conf.max\_file\_size = 100 \* 1024 \* 1024;  
file\_conf.max\_roll\_files = 3;

```
// master log file
file_conf.file_path = "/var/xynova/hand/log/master.log";
CHECK_EC(LogManager::instance().createFileWriter("default", "master", file_conf));

// hand log file
file_conf.file_path = "/var/xynova/hand/log/hand.log";
CHECK_EC(LogManager::instance().createFileWriter("default", "hand", file_conf));
```

```
// =====
// 3. Create logger instances
//     A logger may bind one or more writers
// =====
```

LoggerConfiguration logger\_conf{};  
logger\_conf.sink\_name = "default";  
logger\_conf.level = XYNOVA\_LOG\_LEVEL\_TRACE;  
logger\_conf.lb\_type = "default";

logger\_conf.lb\_config.pool\_size = 50;  
logger\_conf.lb\_config.block\_size = 2048;  
logger\_conf.lb\_config.message\_size = 1024;  
logger\_conf.lb\_config.max\_continuous\_repeated\_log = 10;

```
// logger for master
logger_conf.writer_name_list = {"master"};
```

(续下页)

(接上页)

```

CHECK_EC(LogManager::instance().createLogger("master", logger_conf));

// logger for device
logger_conf.writer_name_list = {"hand"};
CHECK_EC(LogManager::instance().createLogger("hand", logger_conf));

// =====
// 4. Configure manager and bind logger names
//     Modules will emit logs through the assigned logger
// =====

ManagerConfig manager_cfg{};

MasterConfig master_cfg{};
master_cfg.name = "test_master";
master_cfg.cpu_index = -1;
master_cfg.period_us = 1000;

master_cfg.comm.type = CommType::Can;
master_cfg.comm.sub_type = "ChuangXinCAN";
master_cfg.comm.can.iface = "0.0";
master_cfg.comm.can.bitrate = 1'000'000;
master_cfg.comm.can.dbitrate = 5'000'000;

master_cfg.comm.logger_name = "master";

SlaveConfig slave_cfg{};
slave_cfg.type = "DexterousHandSlave";
slave_cfg.id = static_cast<uint32_t>(-1); // ID = 0xFFFFFFFF (all Fs)
slave_cfg.logger_name = "hand";

DeviceConfig device_cfg{};
device_cfg.type = "DexterousHandDevice";
device_cfg.name = "test_hand";
device_cfg.master_name = master_cfg.name;
device_cfg.slaves.push_back(slave_cfg);

manager_cfg.masters.push_back(master_cfg);
manager_cfg.devices.push_back(device_cfg);

// =====
// 5. Initialize and run manager
// =====

auto& mgr = Manager::instance();

CHECK_EC(mgr.init(manager_cfg));
CHECK_EC(mgr.run(master_cfg.name));

// =====
// 6. Stop the master communication loop on exit
//     (the logging subsystem flushes writers asynchronously
//     and is shut down with the process)
// =====

CHECK_EC(mgr.stop(master_cfg.name));

```

### 4.1.3.4 C API

#### 4.1.3.4.1 灵巧手控制 (C API)

本示例演示使用 **C API** 完成完整控制流程：管理器初始化、设备获取、关节控制与资源释放。它是 **C++** 示例（如运行灵巧手设备）的 **C** 对应版本。

**C API** 要求手动管理设备句柄：通过 `XynovaCommunicationManager_getDevice()` 获取的句柄必须使用 `XynovaCommunicationManager_freeDevice()` 释放。

#### 工作流程

1. 配置主站、从站与设备
2. 初始化并运行管理器
3. 获取设备句柄
4. 启动手部设备并发送关节角度命令
5. 停止手部设备
6. 释放设备句柄并停止管理器

#### 示例代码

```
#include <stdint.h>
#include <stdio.h>

#include "xynova/c/comm_api.h"
#include "xynova/c/comm_types.h"
#include "xynova/c/dexterous_hand_api.h"
#include "xynova/c/dexterous_hand_types.h"

#define CHECK_EC(expr) \
do { \
    ec = (expr); \
    if (ec != XYNova_ERROR_CODE_OK) { \
        fprintf(stderr, "[ERROR] %s failed, ec=%d\n", \
            #expr, ec); \
        return -1; \
    } \
} while (0)

int main(void)
{
    XynovaErrorCode ec;

    // =====
    // Configure master (CAN)
    // =====

    XynovaCommunicationMasterConfig master = {0};
    master.name = "test_master";
    master.cpu_index = -1;
    master.period_us = 1000;

    master.comm.type = XYNova_COMMUNICATION_COMM_TYPE_CAN;
    master.comm.sub_type = "ChuangXinCAN";
```

(续下页)

(接上页)

```

master.comm.config.can.iface = "0.0";
master.comm.config.can.bitrate = 1000000;
master.comm.config.can.dbitrate = 5000000;
master.comm.config.can.sample_point = 0.75f;
master.comm.config.can.dsampling_point = 0.75f;

// =====
// Configure slave and device
// =====

XynovaCommunicationSlaveConfig slave = {0};
slave.type = "DexterousHandSlave";
slave.id = 0xFFFFFFFF; // ID = 0xFFFFFFFF (all Fs)

XynovaCommunicationDeviceConfig device_cfg = {0};
device_cfg.type = "DexterousHandDevice";
device_cfg.name = "test_hand";
device_cfg.master_name = master.name;
device_cfg.slaves = &slave;
device_cfg.slave_count = 1;

XynovaCommunicationManagerConfig manager_cfg = {0};
manager_cfg.masters = &master;
manager_cfg.master_count = 1;
manager_cfg.devices = &device_cfg;
manager_cfg.device_count = 1;

// =====
// Initialize and run the manager
// =====

CHECK_EC(XynovaCommunicationManager_init(&manager_cfg));
CHECK_EC(XynovaCommunicationManager_run(master.name));

// =====
// Acquire and start the device
// =====

XynovaDeviceHandle *device =
    XynovaCommunicationManager_getDevice(
        master.name, device_cfg.type, device_cfg.name);

if (device == NULL)
{
    fprintf(stderr, "[ERROR] device not found\n");
    return -1;
}

CHECK_EC(XynovaDexterousHand_run(device));

// =====
// Set control mode and send a joint angle command
// =====

CHECK_EC(XynovaDexterousHand_setJointControlAllMode(
    device,

```

(续下页)

(接上页)

```

        XYNOVA_DEXTEROUS_HAND_CONTROL_MODE_ANGLE));

XynovaDexterousHandAngleControl angle = {0};
for (size_t i = 0; i < XYNOVA_DEXTEROUS_HAND_JOINT_NUM; ++i)
{
    angle.values[i].id = (uint16_t)(i + 1);
    angle.values[i].data.type = XYNOVA_DATA_TYPE_FLOAT;
    angle.values[i].data.value.f32 = 0.5236f;
}

CHECK_EC(XynovaDexterousHand_setJointAngle(device, &angle));

// =====
// Stop the device and release resources
// =====

CHECK_EC(XynovaDexterousHand_stop(device));

XynovaCommunicationManager_freeDevice(device);
CHECK_EC(XynovaCommunicationManager_stop(master.name));

return 0;
}

```

## 说明

- 通过 `XynovaCommunicationManager_getDevice()` 获取的设备句柄必须使用 `XynovaCommunicationManager_freeDevice()` 释放。
- 使用前务必检查设备句柄是否为 `NULL`。
- C API 结构体与其 C++ 对应类型的语义一致；配置字段说明参见[初始化通信管理器](#)。

## 4.1.4 完整示例（Full Examples）

端到端示例，演示包含管理器初始化、设备启动、控制与日志在内的完整工作流。

### 4.1.4.1 灵巧手—伺服控制

#### 4.1.4.1.1 概述

本指南使用 **Xynova C++ API** 演示灵巧手的 **最小端到端控制工作流**。

它遵循灵巧手测试用例相同的控制路径，仅展示当前对用户可用的接口。

#### 4.1.4.1.2 功能

本手册涵盖：

- 日志系统初始化
- 管理器 / 主站 / 设备配置
- CAN 通信配置
- 管理器运行时启动与日志绑定
- 控制模式选择

- 同步角度控制

#### 4.1.4.1.3 前置条件

- Linux 系统
- 已安装 CAN 驱动（SocketCAN 或厂商驱动）
- 建议以 root 权限运行以获得实时调度

#### 备注

实时调度（SCHED\_FIFO）需要 root 或 CAP\_SYS\_NICE 权限。

#### 4.1.4.1.4 执行流程

工作流包含以下步骤：

1. 初始化日志
2. 创建管理器配置
3. 启动主站通信循环
4. 创建并运行设备
5. 为控制线程设置实时调度
6. 切换至角度控制模式
7. 发送角度命令
8. 停止设备并释放资源

#### 4.1.4.1.5 源代码

```

1  #include <atomic>
2  #include <iostream>
3  #include <thread>
4
5  #include <errno.h>
6  #include <pthread.h>
7  #include <sched.h>
8  #include <stdio.h>
9  #include <string.h>
10 #include <time.h>
11 #include <stdint.h>
12
13 #include "xynova/c/log_api.h"
14 #include "xynova/c/version.h"
15
16 #include "xynova/cpp/communication/Manager.hpp"
17 #include "xynova/cpp/hand/dexterous_hand/Device.hpp"
18 #include "xynova/cpp/infra/log/LogManager.hpp"
19
20 using namespace xynova::hand::dexterous_hand;
21
22 // =====

```

(续下页)

(接上页)

```

23 // helpers
24 // =====
25
26 #define CHECK_EC(expr) \
27     do { \
28         ec = (expr); \
29         if (ec != XYNOVA_ERROR_CODE_OK) { \
30             std::cerr << "[ERROR] " << #expr \
31                 << " failed, ec=" << ec << std::endl; \
32             return -1; \
33         } \
34     } while (0)
35
36 bool set_current_thread_realtime(int priority)
37 {
38     if (priority < 1 || priority > 99) {
39         return false;
40     }
41
42     struct sched_param param{};
43     param.sched_priority = priority;
44
45     int ret = pthread_setschedparam(pthread_self(), SCHED_FIFO, &param);
46     if (ret != 0)
47     {
48         std::cerr << "[WARN] set realtime failed: " << strerror(ret) << std::endl;
49         return false;
50     }
51
52     return true;
53 }
54
55 // =====
56 // main
57 // =====
58
59
60 int main()
61 {
62     XynovaErrorCode ec;
63
64     // -----
65     // Version info
66     // -----
67     std::cout << "SDK Version : " << xynovaGetSdkVersion() << "\n"
68         << "Git Branch   : " << xynovaGetSdkGitBranch() << "\n"
69         << "Git Commit    : " << xynovaGetSdkGitCommit() << "\n"
70         << "Build Time   : " << xynovaGetSdkBuildTime() << "\n\n";
71
72     // -----
73     // Logging
74     // -----
75     XynovaLogGlobalLogger_init(XYNOVA_LOG_LEVEL_DEBUG, nullptr);
76
77     auto &log_mgr = xynova::infra::log::LogManager::instance();
78     log_mgr.initDefaultObjects();

```

(续下页)

(接上页)

```

79
80     xynova::infra::log::FileWriterConfiguration file_conf;
81     file_conf.base_conf.formater = "default";
82     file_conf.base_conf.auto_flush = true;
83     file_conf.base_conf.flush_interval_ms = 500;
84     file_conf.file_path = "/var/xynova/hand/log/master.log";
85     file_conf.max_file_size = 100 * 1024 * 1024;
86     file_conf.max_roll_files = 3;
87
88     CHECK_EC(log_mgr.createFileWriter("default", "master", file_conf));
89
90     file_conf.file_path = "/var/xynova/hand/log/hand.log";
91     CHECK_EC(log_mgr.createFileWriter("default", "hand", file_conf));
92
93     xynova::infra::log::LoggerConfiguration logger_conf;
94     logger_conf.sink_name = "default";
95     logger_conf.writer_name_list = {"master"};
96     logger_conf.level = XYNOVA_LOG_LEVEL_TRACE;
97     logger_conf.lb_type = "default";
98     logger_conf.lb_config.pool_size = 50;
99     logger_conf.lb_config.block_size = 2048;
100    logger_conf.lb_config.message_size = 1024;
101    logger_conf.lb_config.max_continuous_repeated_log = 10;
102
103    CHECK_EC(log_mgr.createLogger("master", logger_conf));
104
105    logger_conf.writer_name_list = {"hand"};
106    CHECK_EC(log_mgr.createLogger("hand", logger_conf));
107
108    // -----
109    // Manager config
110    // -----
111    xynova::communication::ManagerConfig manager_cfg;
112
113    xynova::communication::MasterConfig master_cfg;
114    master_cfg.name = "test_master";
115    master_cfg.cpu_index = -1;
116    master_cfg.period_us = 1000;
117    master_cfg.comm.type = xynova::communication::CommType::Can;
118    master_cfg.comm.sub_type = "ChuangXinCAN";
119    master_cfg.comm.can.iface = "0.0";
120    master_cfg.comm.can.bitrate = 1000000;
121    master_cfg.comm.can.dbitrate = 5000000;
122    master_cfg.comm.can.sample_point = 0.75;
123    master_cfg.comm.can.dsampling_point = 0.75;
124    // master_cfg.comm.can.manager_can = true;
125    master_cfg.comm.logger_name = "master";
126
127    xynova::communication::SlaveConfig slave_cfg;
128    slave_cfg.type = "DexterousHandSlave";
129    slave_cfg.id = static_cast<uint32_t>(-1); // ID = 0xFFFFFFFF (all Fs)
130    slave_cfg.logger_name = "hand";
131
132    slave_cfg.general_time_setting.user_max_timeout_us      = 50 * 1000;
133    slave_cfg.general_time_setting.user_poll_interval_us    = 1 * 1000;
134    slave_cfg.general_time_setting.execute_timeout_us       = 5 * 1000;

```

(续下页)

(接上页)

```

135 slave_cfg.general_time_setting.execute_poll_interval_us = 10 * 1000;
136 slave_cfg.general_time_setting.execute_max_retry      = 3;
137
138 slave_cfg.fast_time_setting.user_max_timeout_us       = 5 * 1000;
139 slave_cfg.fast_time_setting.user_poll_interval_us     = 100;
140 slave_cfg.fast_time_setting.execute_timeout_us        = 3 * 1000;
141 slave_cfg.fast_time_setting.execute_poll_interval_us  = 3 * 1000;
142 slave_cfg.fast_time_setting.execute_max_retry        = 0;
143
144 xynova::communication::DeviceConfig device_cfg;
145 device_cfg.type = "DexterousHandDevice";
146 device_cfg.name = "test_hand";
147 device_cfg.master_name = master_cfg.name;
148 device_cfg.slaves.push_back(slave_cfg);
149
150 manager_cfg.masters.push_back(master_cfg);
151 manager_cfg.devices.push_back(device_cfg);
152
153 // -----
154 // Manager start
155 // -----
156 auto &mgr = xynova::communication::Manager::instance();
157
158 CHECK_EC(mgr.init(manager_cfg));
159 CHECK_EC(mgr.run(master_cfg.name));
160
161 // -----
162 // Device
163 // -----
164 auto device = mgr.getDevice<Device>(
165     master_cfg.name,
166     device_cfg.type,
167     device_cfg.name);
168
169 if (!device || !device->valid())
170 {
171     std::cerr << "[ERROR] device invalid\n";
172     return -1;
173 }
174
175 // Switch the calling (control) thread to realtime scheduling.
176 // Note: the SDK's master communication threads are configured with
177 // realtime scheduling internally; this call only affects the current
178 // thread and requires root or CAP_SYS_NICE.
179 set_current_thread_realtime(80);
180
181 CHECK_EC(device->run());
182
183 // -----
184 // Control example
185 // -----
186 ControlMode mode = XYNOVA_DEXTEROUS_HAND_CONTROL_MODE_ANGLE;
187 CHECK_EC(device->setJointControlMode(mode));
188
189 AngleControl angle;
190 for (size_t i = 0; i < JOINT_NUM; ++i)

```

(续下页)

(接上页)

```

191     {
192         angle[i].setId(static_cast<JointID>(i + 1));
193         angle[i].setValue(0.5236f);
194     }
195
196     CHECK_EC(device->setJointAngle(angle));
197
198     std::cout << "Control command sent successfully.\n";
199
200     // -----
201     // Stop the device and release resources
202     // -----
203     CHECK_EC(device->stop());
204     CHECK_EC(mgr.stop(master_cfg.name));
205
206     return 0;
207 }

```

### 4.1.5 使用说明

- 除非另有说明，示例均基于 C++ 高级 API 编写。
- 对应实现位于 communication、dexterous hand 与 infrastructure 源码树中。
- 示例力求精简，便于适配真实硬件。

## 4.2 API 参考

本节提供 Xynova SDK 在本仓库中暴露的编程接口。

SDK 提供两层 API:

- **C API** — ABI 稳定、轻量，适用于嵌入式系统或跨语言绑定。
- **C++ API** — 在 C 层之上构建的面向对象封装，开发更安全、更方便。

当前实现覆盖通信协议栈、灵巧手控制模块、日志工具与 CAN 驱动集成。你可以根据需求选择：需要 ABI 稳定性用 C API，需要更友好的 C++ 体验用 C++ API。

### 4.2.1 概述

Xynova SDK 包含以下功能模块:

- **通信模块 (Communication)** 管理器初始化、主站/设备生命周期与 CAN 传输配置。
- **手部控制模块 (Hand control)** 控制与监控灵巧手设备，包括关节驱动、状态上报与运行时控制。
- **基础设施工具 (Infrastructure)** 日志框架、错误处理与通用数据类型，保证代码健壮且易于维护。

### 4.2.2 API 分类

#### 4.2.2.1 C API

C API 包含以下模块:

#### 4.2.2.1.1 C API 参考

**C API** 为 Xynova 设备与基础设施提供稳定的 ABI。它轻量且适用于嵌入式系统、跨语言绑定与性能敏感型应用。

##### 概述

C API 包含以下模块：

- **通信 (Communication)**：管理主站、设备与 CAN 传输层。
- **手部控制 (Hand Control)**：灵巧手设备的控制与监控。
- **基础设施 (Infrastructure)**：日志、错误码与通用类型。

##### 说明

- 所有 C API 函数均以 *Xynova* 前缀命名。
- 部分对象需要手动管理内存。
- 如需更高级的抽象与 RAII，请参见 C++ API。

#### 4.2.2.1.2 通信模块 (C API)

##### 概述

**通信模块**提供用于管理 Xynova 通信主站、设备与 CAN 传输接口的公共 C API。它暴露稳定的 ABI 入口，供应用与绑定层初始化通信管理器、操作主站、访问设备以及注册 CAN 后端。

##### 公共头文件

本模块对外暴露的 C 头文件包括：

- `xynova/c/comm_api.h` — 通信管理器生命周期与设备访问 API
- `xynova/c/comm_types.h` — 管理器、主站、设备与从站配置类型

CAN 驱动注册入口见[CAN 传输模块](#) 页面：

- `xynova/c/can_api.h` — CAN 驱动注册入口
- `xynova/c/can_types.h` — CAN 传输配置与回调类型

##### API 参考

Communication manager C API for the Xynova SDK.

Provides public APIs for initializing, running, stopping and managing communication managers and connected devices.

➡ 参见

*XynovaCommunicationManagerConfig*

 参见

*XynovaDeviceHandle*

**Version**

1.0.0

**Date**

2025-10-31

**Author**

Xynova

**Typedefs**

typedef struct *XynovaDeviceHandle* **XynovaDeviceHandle**

Opaque handle representing a connected device.

The handle is obtained from *XynovaCommunicationManager\_getDevice()* and released using *XynovaCommunicationManager\_freeDevice()*.

**Functions**

*XynovaErrorCode* **XynovaCommunicationManager\_init** (const *XynovaCommunicationManagerConfig* \*config)

Initialize communication manager.

Creates communication masters and device instances according to the configuration.

**Parameters**

**config** -- Communication manager configuration.

**Returns**

*XynovaErrorCode*.

*XynovaErrorCode* **XynovaCommunicationManager\_run** (const char \*master\_name)

Start communication manager.

Starts communication on the specified master.

**Parameters**

**master\_name** -- Master name.

**Returns**

*XynovaErrorCode*.

*XynovaErrorCode* **XynovaCommunicationManager\_stop** (const char \*master\_name)

Stop communication manager.

Stops communication on the specified master.

**Parameters**

**master\_name** -- Master name.

**Returns**

*XynovaErrorCode*.

*XynovaDeviceHandle* **XynovaCommunicationManager\_getDevice** (const char \*master\_name, const char \*device\_type, const char \*device\_name)

Get device handle.

Retrieves a device handle by master name, device type and device name.

#### Parameters

- **master\_name** -- Master name.
- **device\_type** -- Device type.
- **device\_name** -- Device name.

#### Returns

Device handle, or NULL if not found.

void **XynovaCommunicationManager\_freeDevice** (*XynovaDeviceHandle* \*device)

Free device handle.

Releases a device handle obtained from *XynovaCommunicationManager\_getDevice()*.

#### Parameters

**device** -- Device handle.

*XynovaErrorCode* **XynovaCommunicationManager\_upgradeFirmware** (const char \*master\_name, uint32\_t slave\_id, const char \*file)

Upgrade slave firmware.

#### Parameters

- **master\_name** -- Master name.
- **slave\_id** -- Slave identifier.
- **file** -- Firmware file path.

#### Returns

*XynovaErrorCode*.

Core communication type definitions for the Xynova SDK.

Defines communication configuration types including communication bus, master, slave, device and manager topology.

#### ➡ 参见

*XynovaCommunicationCanConfig*

#### ➡ 参见

*XynovaCommunicationManagerConfig*

#### Version

1.0.0

**Date**

2025-10-29

**Author**

Xynova

**Typedefs**

```
typedef uint8_t XynovaCommunicationCommType
```

Communication type.

Stored as uint8\_t to guarantee ABI compatibility.

```
typedef struct XynovaCommunicationCommConfig XynovaCommunicationCommConfig
```

Communication configuration.

```
typedef struct XynovaCommunicationMasterConfig XynovaCommunicationMasterConfig
```

Master configuration.

```
typedef struct XynovaCommunicationResponseTimeSetting XynovaCommunicationResponseTimeSetting
```

Response timing configuration.

```
typedef struct XynovaCommunicationSlaveConfig XynovaCommunicationSlaveConfig
```

Slave configuration.

```
typedef struct XynovaCommunicationSchedulingCommand XynovaCommunicationSchedulingCommand
```

Scheduling command configuration.

Defines commands periodically sent to a device with response timing settings.

```
typedef struct XynovaCommunicationDeviceConfig XynovaCommunicationDeviceConfig
```

Device configuration.

```
typedef struct XynovaCommunicationManagerConfig XynovaCommunicationManagerConfig
```

Communication manager configuration.

**Enums**

```
enum [anonymous]
```

Communication types.

*Values:*

```
enumerator XYNOVA_COMMUNICATION_COMM_TYPE_CAN
```

CAN communication

```
struct XynovaCommunicationCommConfig
```

*#include <comm\_types.h>* Communication configuration.

**Public Members***XynovaCommunicationCommType* **type**

Communication type

const char \***sub\_type**

Communication driver subtype

*XynovaCommunicationCanConfig* **can**

CAN configuration

union *XynovaCommunicationCommConfig*::[anonymous] **config**const char \***logger\_name**

Logger name

struct **XynovaCommunicationMasterConfig***#include <comm\_types.h>* Master configuration.**Public Members**const char \***name**

Master name

*XynovaCommunicationCommConfig* **comm**

Communication configuration

uint64\_t **period\_us**

Control loop period in microseconds

int32\_t **cpu\_index**

CPU affinity index, -1 disables binding

struct **XynovaCommunicationResponseTimeSetting***#include <comm\_types.h>* Response timing configuration.**Public Members**uint32\_t **user\_max\_timeout\_us**

Maximum user timeout

uint32\_t **user\_poll\_interval\_us**

User polling interval

uint32\_t **execute\_timeout\_us**

Backend request timeout

uint32\_t **execute\_max\_retry**

Backend retry count

uint32\_t **execute\_poll\_interval\_us**

Backend polling interval

struct **XynovaCommunicationSlaveConfig**

*#include <comm\_types.h>* Slave configuration.

### Public Members

const char \***type**

Slave type

uint32\_t **id**

Slave identifier

*XynovaCommunicationResponseTimeSetting* **general\_time\_setting**

General response timing

*XynovaCommunicationResponseTimeSetting* **fast\_time\_setting**

Fast response timing

const char \***logger\_name**

Logger name

struct **XynovaCommunicationSchedulingCommand**

*#include <comm\_types.h>* Scheduling command configuration.

Defines commands periodically sent to a device with response timing settings.

### Public Members

const char \***command**

Command name

*XynovaCommunicationResponseTimeSetting* **response\_time\_setting**

Response timing

struct **XynovaCommunicationDeviceConfig**

*#include <comm\_types.h>* Device configuration.

**Public Members**`const char *type`

Device type

`const char *name`

Device name

`const char *master_name`

Associated master name

*XynovaCommunicationSlaveConfig* \*slaves

Slave configuration array

`size_t slave_count`

Number of slaves

*XynovaCommunicationSchedulingCommand* \*commands

Scheduling command array

`size_t command_count`

Number of commands

`struct XynovaCommunicationManagerConfig`*#include <comm\_types.h>* Communication manager configuration.**Public Members***XynovaCommunicationMasterConfig* \*masters

Master configuration array

`size_t master_count`

Number of masters

*XynovaCommunicationDeviceConfig* \*devices

Device configuration array

`size_t device_count`

Number of devices

**说明**

- 在启动主站循环或访问设备之前，应完成所有句柄与配置对象的初始化。
- 详细的函数签名与类型布局请参考生成的 Doxygen 文档。

4.2.2.1.3 CAN 传输模块 (C API)

概述

CAN 传输模块提供与 Xynova 设备 CAN 总线交互的底层 API，支持实时发送、接收与配置 CAN 消息。该模块以 C 实现，属于通信 API 的一部分。

API 参考

CAN driver registration API for the Xynova SDK.

This header provides the interface to register custom CAN driver implementations with the Xynova SDK. Drivers must implement the *XynovaCommunicationCanVTable* structure and provide it to the SDK via this registration function.

Example:

```
XynovaCommunicationCanVTable my_driver = { ... };
XynovaCommunicationCan_registerDriver("socketcan", &my_driver);
```

➡ 参见

*XynovaCommunicationCanVTable*

**Version**  
1.0.0

**Date**  
2025-10-31

**Author**  
Xynova

**Functions**

*XynovaErrorCode* **XynovaCommunicationCan\_registerDriver** (const char \*type, const *XynovaCommunicationCanVTable* \*vtable)

Register a CAN driver implementation.

The driver must provide a valid *XynovaCommunicationCanVTable*.

- Parameters**
- **type** -- Driver type name, e.g. "socketcan".
  - **vtable** -- Driver virtual function table.

**Return values**

**XYNOVA\_ERROR\_CODE\_OK** -- Driver registered successfully.

**Returns**

*XynovaErrorCode*.

Core CAN type definitions for the Xynova SDK communication subsystem.

Defines fundamental CAN communication types including frame types, identifier masks, interface configuration, receive filters, and driver callback interfaces.

➡ 参见

[XynovaCommunicationCanConfig](#)

➡ 参见

[XynovaCommunicationCanVTable](#)

**Version**  
1.0.0

**Date**  
2025-10-31

**Author**  
Xynova

**Typedefs**

`typedef uint8_t XynovaCommunicationCanFrameType`  
CAN frame identifier type.  
Stored as `uint8_t` to guarantee ABI compatibility.

`typedef uint32_t XynovaCommunicationCanMask`  
CAN identifier mask type.  
Stored as `uint32_t` to guarantee ABI compatibility.

`typedef struct XynovaCommunicationCanConfig XynovaCommunicationCanConfig`  
CAN interface configuration.

`typedef struct XynovaCommunicationCanRecvFilter XynovaCommunicationCanRecvFilter`  
CAN receive filter.  
A frame is accepted when: `(received_id & mask) == (id & mask)`

`typedef struct XynovaCommunicationCanVTable XynovaCommunicationCanVTable`  
Virtual function table (VTable) for a CAN driver implementation.  
Each driver must provide a filled instance of this structure, implementing the basic CAN interface operations such as `create`, `open`, `send`, and `recv`. This enables the SDK to dynamically load and manage different CAN backends.  
Example registration:

```
XynovaCommunicationCanVTable my_driver = {
    .create = MyDriver_create,
    .destroy = MyDriver_destroy,
    .open = MyDriver_open,
    .close = MyDriver_close,
    .isOpen = MyDriver_isOpen,
    .setRecvFilters = MyDriver_setFilters,
    .send = MyDriver_send,
    .recv = MyDriver_recv
};
registerCanDriver("socketcan", my_driver);
```

## Enums

enum **[anonymous]**

CAN frame identifier types.

*Values:*

enumerator **XYNOVA\_COMMUNICATION\_CAN\_FRAME\_TYPE\_STANDARD**

Standard 11-bit CAN identifier

enumerator **XYNOVA\_COMMUNICATION\_CAN\_FRAME\_TYPE\_EXTENDED**

Extended 29-bit CAN identifier

enum **[anonymous]**

CAN identifier masks.

*Values:*

enumerator **XYNOVA\_COMMUNICATION\_CAN\_MASK\_STANDARD**

Standard CAN identifier mask

enumerator **XYNOVA\_COMMUNICATION\_CAN\_MASK\_EXTENDED**

Extended CAN identifier mask

struct **XynovaCommunicationCanConfig**

*#include <can\_types.h>* CAN interface configuration.

## Public Members

const char \***iface**

CAN interface name, e.g. "can0"

bool **manager\_can**

Whether SDK manages CAN interface

uint32\_t **bitrate**

Arbitration bitrate

uint32\_t **dbitrate**

CAN-FD data bitrate

float **sample\_point**

Arbitration sample point, range 0.0 ~ 1.0

float **dsample\_point**

CAN-FD data sample point, range 0.0 ~ 1.0

struct **XynovaCommunicationCanRecvFilter**

*#include <can\_types.h>* CAN receive filter.

A frame is accepted when: (received\_id & mask) == (id & mask)

## Public Members

*XynovaCommunicationCanFrameType* **type**

CAN frame identifier type

uint32\_t **id**

CAN identifier

*XynovaCommunicationCanMask* **mask**

CAN identifier mask

struct **XynovaCommunicationCanVTable**

*#include <can\_types.h>* Virtual function table (VTable) for a CAN driver implementation.

Each driver must provide a filled instance of this structure, implementing the basic CAN interface operations such as `create`, `open`, `send`, and `recv`. This enables the SDK to dynamically load and manage different CAN backends.

Example registration:

```
XynovaCommunicationCanVTable my_driver = {
    .create = MyDriver_create,
    .destroy = MyDriver_destroy,
    .open = MyDriver_open,
    .close = MyDriver_close,
    .isOpen = MyDriver_isOpen,
    .setRecvFilters = MyDriver_setFilters,
    .send = MyDriver_send,
    .recv = MyDriver_recv
};
registerCanDriver("socketcan", my_driver);
```

## Public Members

void **(\*create)**(const *XynovaCommunicationCanConfig* \*config)

Create driver instance.

**Param config**

CAN configuration.

**Return**

Driver handle, or NULL on failure.

void **(\*destroy)**(void \*handle)

Destroy driver instance.

**Param handle**

Driver handle.

*XynovaErrorCode* **(\*open)**(void \*handle)

Open CAN interface.

**Param handle**

Driver handle.

**Return**

XynovaErrorCode.

*XynovaErrorCode* **(\*close)**(void \*handle)

Close CAN interface.

**Param handle**

Driver handle.

**Return**

XynovaErrorCode.

bool **(\*isOpen)**(void \*handle)

Check CAN interface state.

**Param handle**

Driver handle.

**Return**

true if opened.

*XynovaErrorCode* **(\*setRecvFilters)**(void \*handle, uint32\_t num, const *XynovaCommunicationCanRecvFilter* \*filters)

Configure CAN receive filters.

**Param handle**

Driver handle.

**Param num**

Number of filters.

**Param filters**

Filter array.

**Return**

XynovaErrorCode.

*XynovaErrorCode* (\***send**)(void \*handle, *XynovaCommunicationCanFrameType* type, uint32\_t id, const uint8\_t \*data, size\_t len, uint32\_t timeout\_us)

Send CAN frame.

**Param handle**

Driver handle.

**Param type**

Frame identifier type.

**Param id**

CAN identifier.

**Param data**

Data buffer.

**Param len**

Data length.

**Param timeout\_us**

Timeout in microseconds.

**Return**

XynovaErrorCode.

*XynovaErrorCode* (\***recv**)(void \*handle, *XynovaCommunicationCanFrameType* \*type, uint32\_t \*id, uint8\_t \*data, size\_t \*len, uint32\_t timeout\_us)

Receive CAN frame.

**Param handle**

Driver handle.

**Param type**

Output frame type.

**Param id**

Output CAN identifier.

**Param data**

Receive buffer.

**Param len**

Buffer size and received length.

**Param timeout\_us**

Timeout in microseconds.

**Return**

XynovaErrorCode.

**4.2.2.1.4 手部控制模块 (C API)**

本节介绍 Xynova C API 中的 **手部控制模块**。

C API 提供用于控制灵巧手设备的公共底层入口，包括启动、关闭、关节控制与关节状态查询。

## 公共头文件

本模块对外暴露的头文件包括：

- `xynova/c/dexterous_hand_api.h` — 设备生命周期与关节控制 API
- `xynova/c/dexterous_hand_types.h` — 命令与状态类型定义

## API 参考

本模块的完整 API 参考参见灵巧手（C API）。

### 4.2.2.1.5 灵巧手（C API）

本节介绍 Xynova C API 中的 灵巧手设备 API。

它提供当前已实现的灵巧手底层函数，包括：

- 设备生命周期管理（`XynovaDexterousHand_run / stop`）
- 产品信息（型号、序列号、硬件/软件版本、手型）
- 错误状态监控（`XynovaDexterousHand_getErrorState`）
- 关节控制模式配置（`XynovaDexterousHand_setJointControlAllMode`）
- 同步关节角度命令（`XynovaDexterousHand_setJointAngle`）
- 异步关节角度命令（`XynovaDexterousHand_setJointAngle_async`）
- 关节角度状态查询（`XynovaDexterousHand_getJointAngle`）
- 触觉反馈（`wrench` 与 `taxel` 的启用/查询函数）
- 接近觉反馈（`XynovaDexterousHand_enableRange / getRange`）

## 概述

**灵巧手模块**允许直接控制 Xynova 灵巧手设备。所有函数遵循 C API 约定：

- 通过返回 `XynovaErrorCode` 进行错误处理。
- 操作从通信管理器获取的 `XynovaDeviceHandle`。
- 使用前需完成正确初始化。

## API 函数

C API for Xynova Dexterous Hand control and monitoring.

This header provides the public C interface for controlling and monitoring Xynova Dexterous Hand devices, including device lifecycle management, joint control, and joint state acquisition.

### Functions

*XynovaErrorCode* **XynovaDexterousHand\_run** (*XynovaDeviceHandle* \*device)

Start the dexterous hand device.

Initializes and starts device communication.

#### Parameters

**device** -- Device handle obtained from *XynovaCommunicationManager\_getDevice*.

**Returns***XynovaErrorCode*

- XYNOVA\_ERROR\_CODE\_OK on success
- Other codes indicate startup failure.

*XynovaErrorCode* **XynovaDexterousHand\_stop** (*XynovaDeviceHandle* \*device)

Stop the dexterous hand device.

**Parameters****device** -- Device handle.**Returns***XynovaErrorCode**XynovaErrorCode* **XynovaDexterousHand\_getProductModel** (*XynovaDeviceHandle* \*device, char \*product\_model, uint32\_t size)

Get product model.

Reads the product model string from the device.

**Parameters**

- **device** -- Device handle.
- **product\_model** -- Output buffer for product model string.
- **size** -- Size of `product_model` in bytes.

**Returns***XynovaErrorCode**XynovaErrorCode* **XynovaDexterousHand\_getProductSN** (*XynovaDeviceHandle* \*device, char \*product\_sn, uint32\_t size)

Get product serial number.

Reads the product serial number from the device.

**Parameters**

- **device** -- Device handle.
- **product\_sn** -- Output buffer for serial number string.
- **size** -- Size of `product_sn` in bytes.

**Returns***XynovaErrorCode**XynovaErrorCode* **XynovaDexterousHand\_getHardwareVersion** (*XynovaDeviceHandle* \*device, char \*version, uint32\_t size)

Get hardware version.

Reads the hardware version string.

**Parameters**

- **device** -- Device handle.
- **version** -- Output buffer for hardware version.
- **size** -- Size of `version` in bytes.

**Returns***XynovaErrorCode*

*XynovaErrorCode* **XynovaDexterousHand\_getSoftwareVersion** (*XynovaDeviceHandle* \*device, char \*version, uint32\_t size)

Get software version.

Reads the software version string.

**Parameters**

- **device** -- Device handle.
- **version** -- Output buffer for software version.
- **size** -- Size of version in bytes.

**Returns***XynovaErrorCode*

*XynovaErrorCode* **XynovaDexterousHand\_getHandType** (*XynovaDeviceHandle* \*device, *XynovaDexterousHandType* \*type)

Get hand type.

Reads the hand type from the device.

**Parameters**

- **device** -- Device handle.
- **type** -- Output hand type.

**Returns***XynovaErrorCode*

*XynovaErrorCode* **XynovaDexterousHand\_getErrorState** (*XynovaDeviceHandle* \*device, *XynovaDexterousHandErrorState* \*state)

Get current device error state.

Retrieves the latest global warning, global fault and joint error information from the dexterous hand.

The returned data contains:

- Global warning flags.
- Global fault flags.
- Joint error codes for all joints.

**Parameters**

- **device** -- Device handle.
- **state** -- Output error state.

**Returns***XynovaErrorCode*

- XYNOVA\_ERROR\_CODE\_OK on success.

*XynovaErrorCode* **XynovaDexterousHand\_setJointControlAllMode** (*XynovaDeviceHandle* \*device, *XynovaDexterousHandControlMode* mode)

Set control mode for all joints.

#### Parameters

- **device** -- Device handle.
- **mode** -- Joint control mode.

#### Returns

*XynovaErrorCode*

*XynovaErrorCode* **XynovaDexterousHand\_enableTactileWrench** (*XynovaDeviceHandle* \*device, bool enable)

Enable or disable tactile wrench reporting.

Controls whether the device publishes tactile wrench data. When disabled, tactile wrench feedback is not transmitted.

#### Parameters

- **device** -- Device handle.
- **enable** --
  - true: Enable tactile wrench reporting.
  - false: Disable tactile wrench reporting.

#### Returns

*XynovaErrorCode*

*XynovaErrorCode* **XynovaDexterousHand\_enableTactileTaxel** (*XynovaDeviceHandle* \*device, bool enable)

Enable or disable tactile taxel reporting.

Controls whether the device publishes detailed tactile taxel data. When disabled, tactile taxel feedback is not transmitted.

#### Parameters

- **device** -- Device handle.
- **enable** --
  - true: Enable tactile taxel reporting.
  - false: Disable tactile taxel reporting.

#### Returns

*XynovaErrorCode*

*XynovaErrorCode* **XynovaDexterousHand\_enableRange** (*XynovaDeviceHandle* \*device, bool enable)

Enable or disable range reporting.

Controls whether the device publishes range measurement data. When disabled, range feedback is not transmitted.

#### Parameters

- **device** -- Device handle.
- **enable** --
  - true: Enable range reporting.

– false: Disable range reporting.

#### Returns

*XynovaErrorCode*

*XynovaErrorCode* **XynovaDexterousHand\_setJointAngle** (*XynovaDeviceHandle* \*device, const *XynovaDexterousHandAngleControl* \*angle)

Set joint angle control command.

Sends joint angle commands and waits until execution is completed.

#### Parameters

- **device** -- Device handle.
- **angle** -- Joint angle command.

#### Returns

*XynovaErrorCode*

*XynovaErrorCode* **XynovaDexterousHand\_setJointAngle\_async** (*XynovaDeviceHandle* \*device, const *XynovaDexterousHandAngleControl* \*angle)

Asynchronously set joint angle control command.

Queues the joint command and returns immediately. The execution is handled internally by the communication backend.

#### Parameters

- **device** -- Device handle.
- **angle** -- Joint angle command.

#### Returns

*XynovaErrorCode*

*XynovaErrorCode* **XynovaDexterousHand\_getJointAngle** (*XynovaDeviceHandle* \*device, *XynovaDexterousHandAngleStates* \*states, uint64\_t \*timestamp\_us)

Get current joint angle states.

Retrieves the latest joint state data from the device.

#### Parameters

- **device** -- Device handle.
- **states** -- Output joint angle states.
- **timestamp\_us** -- Output timestamp in microseconds. Can be NULL when timestamp is not required.

#### Returns

*XynovaErrorCode*

*XynovaErrorCode* **XynovaDexterousHand\_getTactileWrench** (*XynovaDeviceHandle* \*device, *XynovaDexterousHandTactileWrenchStates* \*states, uint64\_t \*timestamp\_us)

Get tactile wrench states.

Retrieves the latest tactile wrench data from the dexterous hand.

Each tactile region contains:

- **timestamp\_us**: Sampling timestamp in microseconds.
- **region**: Tactile region identifier.
- **dimension**: Valid wrench dimension.
- **wrench**: Force/torque values.

The wrench vector is defined as:

$[F_x, F_y, F_z, M_x, M_y, M_z]$

where:

- **F\***: Force components. Unit: N (Newtons)
- **M\***: Torque components. Unit: Nm (Newton-meters)

#### Parameters

- **device** -- Device handle.
- **states** -- Output tactile wrench states.
- **timestamp\_us** -- Output timestamp in microseconds. Can be NULL when timestamp is not required.

#### Returns

*XynovaErrorCode*

- XYNOVA\_ERROR\_CODE\_OK on success.

*XynovaErrorCode* **XynovaDexterousHand\_getTactileTaxel** (*XynovaDeviceHandle* \*device, *XynovaDexterousHandTactileTaxelStates* \*states, uint64\_t \*timestamp\_us)

Get tactile taxel states.

Retrieves detailed tactile taxel data from the dexterous hand.

The caller provides the taxel buffer. The buffer size must be sufficient for the requested data.

#### Parameters

- **device** -- Device handle.
- **states** -- Output tactile taxel states.
- **timestamp\_us** -- Output timestamp in microseconds. Can be NULL when timestamp is not required.

#### Returns

*XynovaErrorCode*

*XynovaErrorCode* **XynovaDexterousHand\_getRange** (*XynovaDeviceHandle* \*device, *XynovaDexterousHandRangeStates* \*states, uint64\_t \*timestamp\_us)

Get range measurement states.

Retrieves the latest range measurement data from the dexterous hand.

The caller provides the range point buffer. The buffer size must be sufficient for the requested data.

Points are stored in row-major order:

```
index = y * width + x
```

Each point contains:

- **timestamp\_us**: Sampling timestamp.
- **id**: Point identifier.
- **status**: Measurement status.
- **distance**: Measured distance in meters.

#### Parameters

- **device** -- Device handle.
- **states** -- Output range measurement states.
- **timestamp\_us** -- Output timestamp in microseconds. Represents the latest update time of the returned range states. Can be NULL when timestamp is not required.

#### Returns

*XynovaErrorCode*

Xynova Dexterous Hand type definitions.

This header defines fundamental data types used by the Xynova Dexterous Hand API, including joint identifiers, control modes, joint commands, and feedback state structures.

### Defines

#### **XYNOVA\_DEXTEROUS\_HAND\_JOINT\_NUM**

Number of joints supported by Xynova Dexterous Hand.

#### **XYNOVA\_DEXTEROUS\_HAND\_TACTILE\_NUM**

Number of tactile regions supported by Xynova Dexterous Hand.

#### **XYNOVA\_DEXTEROUS\_HAND\_TACTILE\_TOTAL\_TAXEL\_NUM**

Total number of tactile taxels supported by Xynova Dexterous Hand.

#### **XYNOVA\_DEXTEROUS\_HAND\_RANGE\_POINT\_NUM**

Number of range points supported by the default range sensor.

### Typedefs

**typedef uint16\_t XynovaDexterousHandErrorCode**

Dexterous hand error code type.

Stored as uint16\_t to guarantee ABI compatibility.

Each bit represents a joint error condition.

**typedef uint32\_t XynovaDexterousHandWarningFlags**

Dexterous hand global warning flags.

Stored as uint32\_t to guarantee ABI compatibility.

Each bit represents a global warning condition.

**typedef uint32\_t XynovaDexterousHandFaultFlags**

Dexterous hand global fault flags.

Stored as uint32\_t to guarantee ABI compatibility.

Each bit represents a global fault condition.

**typedef struct *XynovaDexterousHandErrorState* XynovaDexterousHandErrorState**

Dexterous hand error state.

Provides global warning, global fault and joint error information.

**typedef uint8\_t XynovaDexterousHandType**

Dexterous hand type.

Stored as uint8\_t to guarantee ABI compatibility.

**typedef uint8\_t XynovaDexterousHandJointID**

Dexterous hand joint identifier type.

Stored as uint8\_t to guarantee ABI compatibility.

**typedef uint8\_t XynovaDexterousHandTactileID**

Dexterous hand tactile region identifier type.

Stored as uint8\_t to guarantee ABI compatibility.

**typedef uint8\_t XynovaDexterousHandWrenchDimension**

Tactile force dimension type.

Stored as uint8\_t to guarantee ABI compatibility.

**typedef uint8\_t XynovaDexterousHandControlMode**

Dexterous hand control mode type.

Stored as uint8\_t to guarantee ABI compatibility.

**typedef uint8\_t XynovaDexterousHandRangePointStatus**

Range point status type.

Stored as uint8\_t to guarantee ABI compatibility.

**typedef struct *XynovaDexterousHandControlModeCommand* XynovaDexterousHandControlModeCommand**

Joint control mode command.

Each element contains:

- id: Joint identifier
- type: *XYNOVA\_DATA\_TYPE\_U8*
- value.u8: *XynovaDexterousHandControlMode*

typedef struct *XynovaDexterousHandAngleControl* **XynovaDexterousHandAngleControl**

Joint angle control command.

Each element contains:

- id: Joint identifier
- type: *XYNOVA\_DATA\_TYPE\_FLOAT*
- value.f32: Joint angle in radians

typedef struct *XynovaDexterousHandAngleStates* **XynovaDexterousHandAngleStates**

Joint angle feedback states.

Each element contains:

- id: Joint identifier
- type: *XYNOVA\_DATA\_TYPE\_FLOAT*
- value.f32: Joint angle in radians

typedef struct *XynovaDexterousHandTactileWrench* **XynovaDexterousHandTactileWrench**

Tactile resultant force.

typedef struct *XynovaDexterousHandTactileWrenchStates* **XynovaDexterousHandTactileWrenchStates**

Tactile resultant force states.

typedef struct *XynovaDexterousHandTactileTaxel* **XynovaDexterousHandTactileTaxel**

Tactile taxel force.

typedef struct *XynovaDexterousHandTactileTaxelStates* **XynovaDexterousHandTactileTaxelStates**

Tactile taxel states.

The taxels buffer is managed by the caller.

typedef struct *XynovaDexterousHandRangePoint* **XynovaDexterousHandRangePoint**

Range point measurement.

Distance unit: meter.

typedef struct *XynovaDexterousHandRangeStates* **XynovaDexterousHandRangeStates**

Range measurement states.

The points buffer is managed by the caller.

Points are stored in row-major order:  $\text{index} = y * \text{width} + x$

## Enums

enum **[anonymous]**

Dexterous hand joint error flags.

*Values:*

enumerator **XYNOVA\_DEXTEROUS\_HAND\_ERROR\_NONE**

No error

enumerator **XYNOVA\_DEXTEROUS\_HAND\_ERROR\_STALL\_PROTECTION**

Motor stall protection

enumerator **XYNOVA\_DEXTEROUS\_HAND\_ERROR\_OVER\_TEMPERATURE**

Over temperature protection

enumerator **XYNOVA\_DEXTEROUS\_HAND\_ERROR\_OVER\_CURRENT**

Over current protection

enumerator **XYNOVA\_DEXTEROUS\_HAND\_ERROR\_MOTOR\_ABNORMAL**

Motor abnormal

enumerator **XYNOVA\_DEXTEROUS\_HAND\_ERROR\_COMMUNICATION**

Communication error

enumerator **XYNOVA\_DEXTEROUS\_HAND\_ERROR\_ENCODER\_ABNORMAL**

Encoder abnormal

enum **[anonymous]**

Dexterous hand global warning flags.

*Values:*

enumerator **XYNOVA\_DEXTEROUS\_HAND\_WARNING\_NONE**

No warning

enumerator **XYNOVA\_DEXTEROUS\_HAND\_WARNING\_ANGLE\_SENSOR\_COMMUNICATION**

Angle sensor communication abnormal

enumerator **XYNOVA\_DEXTEROUS\_HAND\_WARNING\_TACTILE\_SENSOR\_COMMUNICATION**

Tactile sensor communication abnormal

enumerator **XYNOVA\_DEXTEROUS\_HAND\_WARNING\_RANGE\_SENSOR\_COMMUNICATION**

Range (dToF) sensor communication abnormal

enumerator **XYNOVA\_DEXTEROUS\_HAND\_WARNING\_MAINBOARD\_OVER\_TEMPERATURE**

Main control board over temperature

enumerator **XYNOVA\_DEXTEROUS\_HAND\_WARNING\_MAINBOARD\_UNDER\_TEMPERATURE**

Main control board under temperature

enumerator **XYNOVA\_DEXTEROUS\_HAND\_WARNING\_SUBBOARD\_OVER\_TEMPERATURE**

Sub board over temperature

enumerator **XYNOVA\_DEXTEROUS\_HAND\_WARNING\_SUBBOARD\_UNDER\_TEMPERATURE**

Sub board under temperature

enumerator **XYNOVA\_DEXTEROUS\_HAND\_WARNING\_CANFD1\_STATUS**

CAN FD1 status warning

enumerator **XYNOVA\_DEXTEROUS\_HAND\_WARNING\_CANFD2\_STATUS**

CAN FD2 status warning

enumerator **XYNOVA\_DEXTEROUS\_HAND\_WARNING\_FIRMWARE\_UPGRADE**

Firmware upgrade warning

enumerator **XYNOVA\_DEXTEROUS\_HAND\_WARNING\_PDM\_FRAME\_LOSS**

PDM frame loss

enumerator **XYNOVA\_DEXTEROUS\_HAND\_WARNING\_CONTROL\_LOOP\_TIMEOUT**

Control loop timeout

enum **[anonymous]**

Dexterous hand global fault flags.

*Values:*

enumerator **XYNOVA\_DEXTEROUS\_HAND\_FAULT\_NONE**

No fault

enumerator **XYNOVA\_DEXTEROUS\_HAND\_FAULT\_SUBBOARD\_COMMUNICATION**

Sub board communication abnormal

enumerator **XYNOVA\_DEXTEROUS\_HAND\_FAULT\_ACTUATOR\_COMMUNICATION**

Actuator communication abnormal

enumerator **XYNOVA\_DEXTEROUS\_HAND\_FAULT\_ANGLE\_SENSOR\_COMMUNICATION**

Angle sensor communication abnormal

enumerator **XYNOVA\_DEXTEROUS\_HAND\_FAULT\_TACTILE\_SENSOR\_COMMUNICATION**

Tactile sensor communication abnormal

enumerator **XYNOVA\_DEXTEROUS\_HAND\_FAULT\_RANGE\_SENSOR\_COMMUNICATION**

Range (dToF) sensor communication abnormal

enumerator **XYNOVA\_DEXTEROUS\_HAND\_FAULT\_MAINBOARD\_OVER\_TEMPERATURE**

Main control board over temperature

enumerator **XYNOVA\_DEXTEROUS\_HAND\_FAULT\_MAINBOARD\_UNDER\_TEMPERATURE**

Main control board under temperature

enumerator **XYNOVA\_DEXTEROUS\_HAND\_FAULT\_SUBBOARD\_OVER\_TEMPERATURE**

Sub board over temperature

enumerator **XYNOVA\_DEXTEROUS\_HAND\_FAULT\_SUBBOARD\_UNDER\_TEMPERATURE**

Sub board under temperature

enumerator **XYNOVA\_DEXTEROUS\_HAND\_FAULT\_MAINBOARD\_48V\_OVER\_VOLTAGE**

Main board 48V over voltage

enumerator **XYNOVA\_DEXTEROUS\_HAND\_FAULT\_MAINBOARD\_24V\_OVER\_VOLTAGE**

Main board 24V over voltage

enumerator **XYNOVA\_DEXTEROUS\_HAND\_FAULT\_MAINBOARD\_48V\_UNDER\_VOLTAGE**

Main board 48V under voltage

enumerator **XYNOVA\_DEXTEROUS\_HAND\_FAULT\_MAINBOARD\_24V\_UNDER\_VOLTAGE**

Main board 24V under voltage

enumerator **XYNOVA\_DEXTEROUS\_HAND\_FAULT\_MAINBOARD\_OVER\_CURRENT1**

Main board over current channel 1

enumerator **XYNOVA\_DEXTEROUS\_HAND\_FAULT\_MAINBOARD\_OVER\_CURRENT2**

Main board over current channel 2

enumerator **XYNOVA\_DEXTEROUS\_HAND\_FAULT\_MAINBOARD\_OVER\_CURRENT3**

Main board over current channel 3

enumerator **XYNOVA\_DEXTEROUS\_HAND\_FAULT\_CANFD1\_STATUS**

CAN FD1 status fault

enumerator **XYNOVA\_DEXTEROUS\_HAND\_FAULT\_CANFD2\_STATUS**

CAN FD2 status fault

enumerator **XYNOVA\_DEXTEROUS\_HAND\_FAULT\_PARAMETER\_STORAGE**

Parameter storage fault

enumerator **XYNOVA\_DEXTEROUS\_HAND\_FAULT\_ACTUATOR\_OVER\_TEMPERATURE**

Actuator over temperature

enumerator **XYNOVA\_DEXTEROUS\_HAND\_FAULT\_ACTUATOR\_OVER\_CURRENT**

Actuator over current

enumerator **XYNOVA\_DEXTEROUS\_HAND\_FAULT\_ACTUATOR\_UNDER\_VOLTAGE**

Actuator under voltage

enumerator **XYNOVA\_DEXTEROUS\_HAND\_FAULT\_ACTUATOR\_OVER\_VOLTAGE**

Actuator over voltage

enumerator **XYNOVA\_DEXTEROUS\_HAND\_FAULT\_PDM\_FRAME\_LOSS**

PDM frame loss

enumerator **XYNOVA\_DEXTEROUS\_HAND\_FAULT\_CONTROL\_LOOP\_TIMEOUT**

Control loop timeout

enum [**anonymous**]

Dexterous hand types.

*Values:*

enumerator **XYNOVA\_DEXTEROUS\_HAND\_TYPE\_UNKNOWN**

Unknown hand type

enumerator **XYNOVA\_DEXTEROUS\_HAND\_TYPE\_LEFT**

Left hand

enumerator **XYNOVA\_DEXTEROUS\_HAND\_TYPE\_RIGHT**

Right hand

enum [**anonymous**]

Dexterous hand joint identifiers.

*Values:*

enumerator **XYNOVA\_DEXTEROUS\_HAND\_JOINT\_ID\_FF1**

Index fingertip flex (Passive)

enumerator **XYNOVA\_DEXTEROUS\_HAND\_JOINT\_ID\_FF2**

Index middle flex

enumerator **XYNOVA\_DEXTEROUS\_HAND\_JOINT\_ID\_FF3**  
Index base flex

enumerator **XYNOVA\_DEXTEROUS\_HAND\_JOINT\_ID\_FF4**  
Index abduction

enumerator **XYNOVA\_DEXTEROUS\_HAND\_JOINT\_ID\_MF1**  
Middle fingertip flex (Passive)

enumerator **XYNOVA\_DEXTEROUS\_HAND\_JOINT\_ID\_MF2**  
Middle middle flex

enumerator **XYNOVA\_DEXTEROUS\_HAND\_JOINT\_ID\_MF3**  
Middle base flex

enumerator **XYNOVA\_DEXTEROUS\_HAND\_JOINT\_ID\_MF4**  
Middle abduction

enumerator **XYNOVA\_DEXTEROUS\_HAND\_JOINT\_ID\_RF1**  
Ring fingertip flex (Passive)

enumerator **XYNOVA\_DEXTEROUS\_HAND\_JOINT\_ID\_RF2**  
Ring middle flex

enumerator **XYNOVA\_DEXTEROUS\_HAND\_JOINT\_ID\_RF3**  
Ring base flex

enumerator **XYNOVA\_DEXTEROUS\_HAND\_JOINT\_ID\_RF4**  
Ring abduction

enumerator **XYNOVA\_DEXTEROUS\_HAND\_JOINT\_ID\_LF1**  
Little fingertip flex (Passive)

enumerator **XYNOVA\_DEXTEROUS\_HAND\_JOINT\_ID\_LF2**  
Little middle flex

enumerator **XYNOVA\_DEXTEROUS\_HAND\_JOINT\_ID\_LF3**  
Little base flex

enumerator **XYNOVA\_DEXTEROUS\_HAND\_JOINT\_ID\_LF4**  
Little abduction

enumerator **XYNOVA\_DEXTEROUS\_HAND\_JOINT\_ID\_LF5**  
Little rotation

enumerator **XYNOVA\_DEXTEROUS\_HAND\_JOINT\_ID\_TH1**

Thumb tip flex

enumerator **XYNOVA\_DEXTEROUS\_HAND\_JOINT\_ID\_TH2**

Thumb middle flex

enumerator **XYNOVA\_DEXTEROUS\_HAND\_JOINT\_ID\_TH3**

Thumb base flex

enumerator **XYNOVA\_DEXTEROUS\_HAND\_JOINT\_ID\_TH4**

Thumb abduction

enumerator **XYNOVA\_DEXTEROUS\_HAND\_JOINT\_ID\_WR1**

Wrist pitch

enumerator **XYNOVA\_DEXTEROUS\_HAND\_JOINT\_ID\_WR2**

Wrist yaw

enum **[anonymous]**

Dexterous hand tactile region identifiers.

*Values:*

enumerator **XYNOVA\_DEXTEROUS\_HAND\_TACTILE\_ID\_TT1**

Thumb fingertip

enumerator **XYNOVA\_DEXTEROUS\_HAND\_TACTILE\_ID\_TT2**

Thumb pad

enumerator **XYNOVA\_DEXTEROUS\_HAND\_TACTILE\_ID\_FT1**

Index fingertip

enumerator **XYNOVA\_DEXTEROUS\_HAND\_TACTILE\_ID\_FT2**

Index middle phalanx

enumerator **XYNOVA\_DEXTEROUS\_HAND\_TACTILE\_ID\_FT3**

Index proximal phalanx

enumerator **XYNOVA\_DEXTEROUS\_HAND\_TACTILE\_ID\_MT1**

Middle fingertip

enumerator **XYNOVA\_DEXTEROUS\_HAND\_TACTILE\_ID\_MT2**

Middle middle phalanx

enumerator **XYNOVA\_DEXTEROUS\_HAND\_TACTILE\_ID\_MT3**

Middle proximal phalanx

enumerator **XYNOVA\_DEXTEROUS\_HAND\_TACTILE\_ID\_RT1**

Ring fingertip

enumerator **XYNOVA\_DEXTEROUS\_HAND\_TACTILE\_ID\_RT2**

Ring middle phalanx

enumerator **XYNOVA\_DEXTEROUS\_HAND\_TACTILE\_ID\_RT3**

Ring proximal phalanx

enumerator **XYNOVA\_DEXTEROUS\_HAND\_TACTILE\_ID\_LT1**

Little fingertip

enumerator **XYNOVA\_DEXTEROUS\_HAND\_TACTILE\_ID\_LT2**

Little middle phalanx

enumerator **XYNOVA\_DEXTEROUS\_HAND\_TACTILE\_ID\_LT3**

Little proximal phalanx

enumerator **XYNOVA\_DEXTEROUS\_HAND\_TACTILE\_ID\_PT1**

Palm

enum **[anonymous]**

Tactile wrench dimensions.

*Values:*

enumerator **XYNOVA\_DEXTEROUS\_HAND\_WRENCH\_DIMENSION\_1D**

One-dimensional wrench

enumerator **XYNOVA\_DEXTEROUS\_HAND\_WRENCH\_DIMENSION\_3D**

Three-dimensional wrench

enumerator **XYNOVA\_DEXTEROUS\_HAND\_WRENCH\_DIMENSION\_6D**

Six-dimensional wrench

enum **[anonymous]**

Dexterous hand control modes.

*Values:*

enumerator **XYNOVA\_DEXTEROUS\_HAND\_CONTROL\_MODE\_DISABLE**

Control disabled

enumerator **XYNOVA\_DEXTEROUS\_HAND\_CONTROL\_MODE\_ANGLE**

Angle control

enum [**anonymous**]

Range point measurement status.

*Values:*

enumerator **XYNOVA\_DEXTEROUS\_HAND\_RANGE\_POINT\_STATUS\_NO\_TARGET**

No target detected

enumerator **XYNOVA\_DEXTEROUS\_HAND\_RANGE\_POINT\_STATUS\_WEAK\_SIGNAL**

Weak signal

enumerator **XYNOVA\_DEXTEROUS\_HAND\_RANGE\_POINT\_STATUS\_VALID**

Valid measurement

enumerator **XYNOVA\_DEXTEROUS\_HAND\_RANGE\_POINT\_STATUS\_INVALID**

Invalid measurement

struct **XynovaDexterousHandErrorState**

*#include <dexterous\_hand\_types.h>* Dexterous hand error state.

Provides global warning, global fault and joint error information.

### Public Members

uint64\_t **timestamp\_us**

Error timestamp (us): system monotonic clock

*XynovaDexterousHandWarningFlags* **warning\_flags**

Global warning flags

*XynovaDexterousHandFaultFlags* **fault\_flags**

Global fault flags

*XynovaIdValue* **joint\_error\_code**[XYNOVA\_DEXTEROUS\_HAND\_JOINT\_NUM]

Joint error codes

struct **XynovaDexterousHandControlModeCommand**

*#include <dexterous\_hand\_types.h>* Joint control mode command.

Each element contains:

- id: Joint identifier
- type: *XYNOVA\_DATA\_TYPE\_U8*
- value.u8: *XynovaDexterousHandControlMode*

**Public Members**

*XynovaIdValue* **values**[XYNOVA\_DEXTEROUS\_HAND\_JOINT\_NUM]

Joint control mode values

struct **XynovaDexterousHandAngleControl**

*#include <dexterous\_hand\_types.h>* Joint angle control command.

Each element contains:

- id: Joint identifier
- type: *XYNOVA\_DATA\_TYPE\_FLOAT*
- value.f32: Joint angle in radians

**Public Members**

*XynovaIdValue* **values**[XYNOVA\_DEXTEROUS\_HAND\_JOINT\_NUM]

Joint angle commands

struct **XynovaDexterousHandAngleStates**

*#include <dexterous\_hand\_types.h>* Joint angle feedback states.

Each element contains:

- id: Joint identifier
- type: *XYNOVA\_DATA\_TYPE\_FLOAT*
- value.f32: Joint angle in radians

**Public Members**

*XynovaIdValue* **values**[XYNOVA\_DEXTEROUS\_HAND\_JOINT\_NUM]

Joint angle states

struct **XynovaDexterousHandTactileWrench**

*#include <dexterous\_hand\_types.h>* Tactile resultant force.

**Public Members**

uint64\_t **timestamp\_us**

Sampling timestamp (us): system monotonic clock

*XynovaDexterousHandTactileID* **region**

Tactile region

*XynovaDexterousHandWrenchDimension* **dimension**

Force dimension

*XynovaTypedValueArray6* **wrench**

Force/torque values: force unit: N, torque unit: Nm

struct **XynovaDexterousHandTactileWrenchStates**

*#include <dexterous\_hand\_types.h>* Tactile resultant force states.

### Public Members

*XynovaDexterousHandTactileWrench* **values**[XYNOVA\_DEXTEROUS\_HAND\_TACTILE\_NUM]

struct **XynovaDexterousHandTactileTaxel**

*#include <dexterous\_hand\_types.h>* Tactile taxel force.

### Public Members

uint64\_t **timestamp\_us**

Sampling timestamp (us): system monotonic clock

*XynovaDexterousHandTactileID* **region**

Tactile region

uint16\_t **taxel\_id**

Taxel identifier

*XynovaDexterousHandWrenchDimension* **dimension**

Force dimension

uint8\_t **reserved**

Reserved

*XynovaTypedValueArray6* **wrench**

Force/torque values: force unit: N, torque unit: Nm

struct **XynovaDexterousHandTactileTaxelStates**

*#include <dexterous\_hand\_types.h>* Tactile taxel states.

The taxels buffer is managed by the caller.

### Public Members

uint32\_t **count**

Number of taxels

*XynovaDexterousHandTactileTaxel* \***taxels**

Taxel array

```
struct XynovaDexterousHandRangePoint
```

*#include <dexterous\_hand\_types.h>* Range point measurement.

Distance unit: meter.

### Public Members

```
uint64_t timestamp_us
```

Measurement timestamp (us): system monotonic clock

```
uint16_t id
```

Point identifier

```
XynovaDexterousHandRangePointStatus status
```

Measurement status

```
uint8_t reserved
```

Reserved

```
float distance
```

Distance in meter

```
struct XynovaDexterousHandRangeStates
```

*#include <dexterous\_hand\_types.h>* Range measurement states.

The points buffer is managed by the caller.

Points are stored in row-major order:  $\text{index} = y * \text{width} + x$

### Public Members

```
uint16_t width
```

Range map width

```
uint16_t height
```

Range map height

```
uint32_t count
```

Number of range points

```
XynovaDexterousHandRangePoint *points
```

Range point buffer

4.2.2.1.6 基础设施 (C API)

**基础设施模块**提供 Xynova SDK 各处使用的基础工具，包括日志、错误处理与通用值辅助，为设备与通信层的一致、可靠行为提供支持。

主要特性:

- 日志系统 (sink、writer、formatter 与后端)
- 标准化错误码与结果类型
- 用于跨模块数据交换的通用值与结果封装
- 兼容 ABI 的基础类型与符号可见性宏
- SDK 版本信息

公共头文件

本模块对外暴露的 C 头文件包括:

- xynova/c/base\_types.h —兼容 ABI 的基础类型 (XynovaBool)
- xynova/c/abi.h —符号可见性与 ABI 控制宏
- xynova/c/error\_code.h —标准化错误码定义 (另见错误码 参考)
- xynova/c/result.h —结果封装辅助
- xynova/c/value\_types.h —类型化值与标识符容器类型
- xynova/c/version.h —SDK 版本信息

日志入口见日志 页面:

- xynova/c/log\_api.h —全局与对象日志入口
- xynova/c/log\_types.h —日志级别、记录、sink、writer 与 logger 配置类型

API 参考

Fundamental ABI-safe type definitions for the Xynova SDK.  
Defines common scalar types shared by all public C APIs.  
All public types use fixed-width integers to guarantee ABI compatibility.

**Version**  
1.0.0  
**Date**  
2025-10-29  
**Author**  
Xynova

**Typedefs**  
  
typedef uint8\_t **XynovaBool**  
ABI-safe boolean type.

Stored as `uint8_t` for cross-platform compatibility.

Valid values are `XYNOVA_FALSE` and `XYNOVA_TRUE`.

## Enums

enum **[anonymous]**

Boolean constants.

*Values:*

enumerator **XYNOVA\_FALSE**

Boolean false.

enumerator **XYNOVA\_TRUE**

Boolean true.

Definition of symbol visibility and ABI control macros for the Xynova SDK.

This header defines platform-independent macros that control symbol visibility, ABI stability, and linkage behavior across different compilers and operating systems. It ensures that public SDK interfaces are exported properly, while internal symbols remain hidden to maintain binary compatibility.

The macros in this file provide consistent export/import rules for both Windows and POSIX-based systems (Linux/macOS), making shared library integration straightforward.

## Defines

**XYNOVA\_API**

When ABI stability control is disabled, all symbols are public.

**XYNOVA\_LOCAL**

**XYNOVA\_IMPL\_API**

**XYNOVA\_INLINE\_API**

Defines inline functions with hidden visibility for internal usage.

Used to prevent unnecessary symbol export from inline helper functions. On GCC/Clang, adds `__attribute__((visibility("hidden")))`.

### 备注

C++17 `constexpr` functions are implicitly inline.

**XYNOVA\_NOEXCEPT**

Provides a consistent `noexcept` macro for function declarations.

**XYNOVA\_CONSTEXPR**

Provides a consistent `constexpr` macro for function or variable declarations.

**XYNOVA\_EXPORT\_TEMPLATE** (class\_name, ...)

Explicitly exports template class instantiations for shared library usage.

```
template class XYNova_API MyTemplate<int>;
```

**Parameters**

- **class\_name** -- The name of the template class.
- ... -- The template type arguments.

Definition of Xynova SDK error codes and conversion utilities.

Provides stable error code definitions and string conversion support.

Error codes are maintained through X-Macro for centralized management.

**Version**

1.0.0

**Date**

2025-10-29

**Author**

Xynova

**Defines****XYNOVA\_ERROR\_CODE\_LIST**

Central definition list of SDK error codes.

Format: X(NAME, VALUE, DESCRIPTION)

**X** (name, value, desc)

desc

**X** (name, value, desc)

desc

**Typedefs**

typedef int32\_t **XynovaErrorCode**

Xynova SDK error code type.

Uses int32\_t to guarantee ABI compatibility.

## Enums

enum [**anonymous**]

Xynova SDK error code constants.

Generated from XYNOVA\_ERROR\_CODE\_LIST.

*Values:*

enumerator **XYNOVA\_ERROR\_CODE\_LIST**

## Functions

static inline const char \***xynovaErrorCodeToString** (*XynovaErrorCode* code)

Convert error code to string.

### Parameters

**code** -- Error code.

### Returns

Static string describing the error.

Definition of *XynovaResult* structure for representing operation outcomes.

Defines *XynovaResult* for returning operation status and optional message. Designed for stable C ABI usage across shared libraries.

Features:

- Standardized operation result representation.
- Combines *XynovaErrorCode* with an optional message.
- Compatible with C and C++ callers.

 参见

xynova/c/error\_code.h

### Version

1.0.0

### Date

2025-10-09

### Author

Xynova

## Typedefs

typedef struct *XynovaResult* **XynovaResult**

struct **XynovaResult**

*#include <result.h>* Represents the result of an SDK operation.

Contains an error code and optional descriptive message.

#### 备注

The message pointer does not transfer ownership. The pointed string must remain valid while the result is used.

### Public Members

*XynovaErrorCode* **code**

Operation result code.

const char \***message**

Optional descriptive message.

Generic value type definitions for the Xynova C API.

Defines generic scalar and fixed-size vector value containers used by the Xynova SDK. All public structures use fixed-width integer types to maintain ABI compatibility.

### Version

1.0.0

### Date

2026-07-17

### Author

Xynova

### Typedefs

typedef uint8\_t **XynovaDataType**

Value type identifier.

Stored as uint8\_t to guarantee ABI compatibility.

typedef union *XynovaValue* **XynovaValue**

Generic scalar value storage.

The active member is selected by *XynovaDataType*.

typedef struct *XynovaTypedValue* **XynovaTypedValue**

Typed scalar value.

typedef struct *XynovaTypedValueArray2* **XynovaTypedValueArray2**

Typed two-element value.

Commonly used for 2-D vector data.

```
typedef struct XynovaTypedValueArray3 XynovaTypedValueArray3
```

Typed three-element value.

Commonly used for 3-D vector data.

```
typedef struct XynovaTypedValueArray4 XynovaTypedValueArray4
```

Typed four-element value.

Commonly used for 4-D vector data.

```
typedef struct XynovaTypedValueArray5 XynovaTypedValueArray5
```

Typed five-element value.

Commonly used for 5-D vector data.

```
typedef struct XynovaTypedValueArray6 XynovaTypedValueArray6
```

Typed six-element value.

Commonly used for 6-DoF vector data.

```
typedef struct XynovaIdValue XynovaIdValue
```

Identifier with typed scalar value.

```
typedef struct XynovaIdValueArray2 XynovaIdValueArray2
```

Identifier with typed two-element value.

```
typedef struct XynovaIdValueArray3 XynovaIdValueArray3
```

Identifier with typed three-element value.

```
typedef struct XynovaIdValueArray4 XynovaIdValueArray4
```

Identifier with typed four-element value.

```
typedef struct XynovaIdValueArray5 XynovaIdValueArray5
```

Identifier with typed five-element value.

```
typedef struct XynovaIdValueArray6 XynovaIdValueArray6
```

Identifier with typed six-element value.

## Enums

```
enum [anonymous]
```

Value type identifiers.

Values are explicitly assigned for protocol and ABI compatibility.

*Values:*

enumerator **XYNOVA\_DATA\_TYPE\_NONE**

Undefined type

enumerator **XYNOVA\_DATA\_TYPE\_BOOL**

Boolean value

enumerator **XYNOVA\_DATA\_TYPE\_U8**

Unsigned 8-bit integer

enumerator **XYNOVA\_DATA\_TYPE\_S8**

Signed 8-bit integer

enumerator **XYNOVA\_DATA\_TYPE\_U16**

Unsigned 16-bit integer

enumerator **XYNOVA\_DATA\_TYPE\_S16**

Signed 16-bit integer

enumerator **XYNOVA\_DATA\_TYPE\_U32**

Unsigned 32-bit integer

enumerator **XYNOVA\_DATA\_TYPE\_S32**

Signed 32-bit integer

enumerator **XYNOVA\_DATA\_TYPE\_U64**

Unsigned 64-bit integer

enumerator **XYNOVA\_DATA\_TYPE\_S64**

Signed 64-bit integer

enumerator **XYNOVA\_DATA\_TYPE\_FLOAT**

32-bit floating point

enumerator **XYNOVA\_DATA\_TYPE\_DOUBLE**

64-bit floating point

union **XynovaValue**

*#include <value\_types.h>* Generic scalar value storage.

The active member is selected by *XynovaDataType*.

## Public Members

uint8\_t **u8**

Unsigned 8-bit value

**int8\_t s8**  
Signed 8-bit value

**uint16\_t u16**  
Unsigned 16-bit value

**int16\_t s16**  
Signed 16-bit value

**uint32\_t u32**  
Unsigned 32-bit value

**int32\_t s32**  
Signed 32-bit value

**uint64\_t u64**  
Unsigned 64-bit value

**int64\_t s64**  
Signed 64-bit value

**float f32**  
32-bit floating point value

**double f64**  
64-bit floating point value

struct **XynovaTypedValue**  
*#include <value\_types.h>* Typed scalar value.

### Public Members

*XynovaDataType* **type**  
Value type

*XynovaValue* **value**  
Scalar value

struct **XynovaTypedValueArray2**  
*#include <value\_types.h>* Typed two-element value.  
Commonly used for 2-D vector data.

**Public Members***XynovaDataType* **type**

Value type

*XynovaValue* **values[2]**

Value elements

struct **XynovaTypedValueArray3***#include <value\_types.h>* Typed three-element value.

Commonly used for 3-D vector data.

**Public Members***XynovaDataType* **type**

Value type

*XynovaValue* **values[3]**

Value elements

struct **XynovaTypedValueArray4***#include <value\_types.h>* Typed four-element value.

Commonly used for 4-D vector data.

**Public Members***XynovaDataType* **type**

Value type

*XynovaValue* **values[4]**

Value elements

struct **XynovaTypedValueArray5***#include <value\_types.h>* Typed five-element value.

Commonly used for 5-D vector data.

**Public Members***XynovaDataType* **type**

Value type

*XynovaValue* **values[5]**

Value elements

struct **XynovaTypedValueArray6**

*#include <value\_types.h>* Typed six-element value.

Commonly used for 6-DoF vector data.

### Public Members

*XynovaDataType* **type**

Value type

*XynovaValue* **values**[6]

Value elements

struct **XynovaIdValue**

*#include <value\_types.h>* Identifier with typed scalar value.

### Public Members

uint16\_t **id**

Value identifier

*XynovaTypedValue* **data**

Typed scalar value

struct **XynovaIdValueArray2**

*#include <value\_types.h>* Identifier with typed two-element value.

### Public Members

uint16\_t **id**

Value identifier

*XynovaTypedValueArray2* **data**

Typed two-element value

struct **XynovaIdValueArray3**

*#include <value\_types.h>* Identifier with typed three-element value.

### Public Members

uint16\_t **id**

Value identifier

*XynovaTypedValueArray3* **data**

Typed three-element value

```
struct XynovaIdValueArray4
```

*#include <value\_types.h>* Identifier with typed four-element value.

### Public Members

```
uint16_t id
```

Value identifier

```
XynovaTypedValueArray4 data
```

Typed four-element value

```
struct XynovaIdValueArray5
```

*#include <value\_types.h>* Identifier with typed five-element value.

### Public Members

```
uint16_t id
```

Value identifier

```
XynovaTypedValueArray5 data
```

Typed five-element value

```
struct XynovaIdValueArray6
```

*#include <value\_types.h>* Identifier with typed six-element value.

### Public Members

```
uint16_t id
```

Value identifier

```
XynovaTypedValueArray6 data
```

Typed six-element value

SDK version and build metadata interface for Xynova SDK.

This header provides functions to query the SDK version, Git branch, Git commit hash, and build timestamp. The version information is automatically injected at build time by CMake.

### Defines

```
XYNOVA_SDK_VERSION
```

SDK semantic version string injected at build time.

```
XYNOVA_SDK_LOCAL_VERSION
```

SDK semantic local version string injected at build time.

**XYNOVA\_SDK\_GIT\_BRANCH**

Git branch name of the SDK build.

**XYNOVA\_SDK\_GIT\_COMMIT**

Git commit hash corresponding to this SDK build.

**XYNOVA\_SDK\_BUILD\_TIME**

ISO 8601 timestamp of the SDK build.

**Functions**

const char **\*xynovaGetSdkVersion()** noexcept

Get the SDK semantic version string.

**Returns**

Null-terminated string (e.g., "1.4.0").

const char **\*xynovaGetSdkLocalVersion()** noexcept

Get the SDK semantic local version string.

**Returns**

Null-terminated string (e.g., "#1").

const char **\*xynovaGetSdkGitBranch()** noexcept

Get the Git branch name used for this build.

**Returns**

Null-terminated string (e.g., "main" or "develop").

const char **\*xynovaGetSdkGitCommit()** noexcept

Get the Git commit hash of the build.

**Returns**

Null-terminated string (e.g., "a1b2c3d4e5f6").

const char **\*xynovaGetSdkBuildTime()** noexcept

Get the build timestamp in ISO 8601 format.

**Returns**

Null-terminated string (e.g., "2025-10-29T12:34:56Z").

**4.2.2.1.7 日志 (C API)**

**日志模块**为 Xynova 设备与 SDK 组件提供灵活且高性能的日志基础设施。

支持:

- 多种 sink (线程池、文件、控制台)
- 自定义 formatter
- 可配置缓冲池、刷新闻隔与消息过滤的 logger 后端
- 基于级别的日志记录

以下头文件定义了 C API:

Public C API for the Xynova SDK logging subsystem.

Provides global logging control and object-based logging management APIs. Supports configurable log levels, callbacks, sinks, writers, formatters and logger backends.

#### Version

1.0.0

#### Date

2025-10-29

#### Author

Xynova

### Functions

void **XynovaLogGlobalLogger\_init** (*XynovaLogLevel* log\_level, *XynovaLogLogHandler* log\_handler)

Initialize global logging subsystem.

#### Parameters

- **log\_level** -- Initial log level.
- **log\_handler** -- Log callback handler.

*XynovaLogLevel* **XynovaLogGlobalLogger\_setLogLevel** (*XynovaLogLevel* level)

Set global logging level.

#### Parameters

**level** -- New log level.

#### Returns

Previous log level.

*XynovaLogLevel* **XynovaLogGlobalLogger\_getLogLevel** (void)

Get global logging level.

#### Returns

Current log level.

*XynovaErrorCode* **XynovaLogObjectLogger\_initDefaultObjects** (void)

Initialize default logging objects.

Creates default sinks, writers, formatters and logger backend.

#### Returns

XynovaErrorCode.

*XynovaBool* **XynovaLogObjectLogger\_isInitialized** (void)

Check whether object logger subsystem is initialized.

#### Returns

XYNOVA\_TRUE if initialized, otherwise XYNOVA\_FALSE.

*XynovaErrorCode* **XynovaLogObjectLogger\_createSinkTP** (const char \*type, const char \*name, const *XynovaLogSinkTPConfiguration* \*conf)

Create thread-pool based sink.

#### Parameters

- **type** -- Sink type.
- **name** -- Sink instance name.
- **conf** -- Sink configuration.

**Returns**

XynovaErrorCode.

*XynovaErrorCode* **XynovaLogObjectLogger\_createFileWriter** (const char \*type, const char \*name, const *XynovaLogFileWriterConfiguration* \*conf)

Create file writer.

**Parameters**

- **type** -- Writer type.
- **name** -- Writer instance name.
- **conf** -- Writer configuration.

**Returns**

XynovaErrorCode.

*XynovaErrorCode* **XynovaLogObjectLogger\_createConsoleWriter** (const char \*type, const char \*name, const *XynovaLogConsoleWriterConfiguration* \*conf)

Create console writer.

**Parameters**

- **type** -- Writer type.
- **name** -- Writer instance name.
- **conf** -- Writer configuration.

**Returns**

XynovaErrorCode.

*XynovaErrorCode* **XynovaLogObjectLogger\_createFormatter** (const char \*type, const char \*name, const *XynovaLogFormatterConfiguration* \*conf)

Create formatter.

**Parameters**

- **type** -- Formatter type.
- **name** -- Formatter instance name.
- **conf** -- Formatter configuration.

**Returns**

XynovaErrorCode.

*XynovaErrorCode* **XynovaLogObjectLogger\_createLogger** (const char \*name, const *XynovaLogLoggerConfiguration* \*conf)

Create logger backend.

**Parameters**

- **name** -- Logger instance name.

- **conf** -- Logger configuration.

**Returns**

XynovaErrorCode.

*XynovaBool* **XynovaLogObjectLogger\_hasSink** (const char \*name)

Check whether sink exists.

**Parameters**

**name** -- Sink instance name.

**Returns**

XYNOVA\_TRUE if exists, otherwise XYNOVA\_FALSE.

*XynovaBool* **XynovaLogObjectLogger\_hasWriter** (const char \*name)

Check whether writer exists.

**Parameters**

**name** -- Writer instance name.

**Returns**

XYNOVA\_TRUE if exists, otherwise XYNOVA\_FALSE.

*XynovaBool* **XynovaLogObjectLogger\_hasFormatter** (const char \*name)

Check whether formatter exists.

**Parameters**

**name** -- Formatter instance name.

**Returns**

XYNOVA\_TRUE if exists, otherwise XYNOVA\_FALSE.

*XynovaBool* **XynovaLogObjectLogger\_hasLogger** (const char \*name)

Check whether logger backend exists.

**Parameters**

**name** -- Logger backend instance name.

**Returns**

XYNOVA\_TRUE if exists, otherwise XYNOVA\_FALSE.

*XynovaErrorCode* **XynovaLogObjectLogger\_setLoggerLevel** (const char \*name, *XynovaLogLevel* level)

Set logger backend log level.

**Parameters**

- **name** -- Logger backend instance name.
- **level** -- New logger level.

**Returns**

XynovaErrorCode.

Core data types for the Xynova SDK logging subsystem.

Defines logging structures, severity levels, callbacks and configuration types used by the Xynova SDK logging module.

**Version**

1.0.0

**Date**

2025-10-29

**Author**

Xynova

**Typedefs**typedef uint8\_t **XynovaLogLevel**

Logging severity level type.

Stored as uint8\_t to guarantee ABI compatibility.

typedef struct *XynovaLogDateTimeRaw* **XynovaLogDateTimeRaw**

Raw timestamp representation.

Uses UNIX epoch time with microsecond precision.

typedef struct *XynovaLogLogRecordMeta* **XynovaLogLogRecordMeta**

Metadata of a log record.

typedef struct *XynovaLogLogRecord* **XynovaLogLogRecord**

Complete log record.

Delivered to user callback handlers.

typedef void (\***XynovaLogLogHandler**)(const *XynovaLogLogRecord* \*record)

Log record callback type.

typedef struct *XynovaLogSinkConfiguration* **XynovaLogSinkConfiguration**

Common sink configuration.

Defines queue and worker loop timing parameters.

typedef struct *XynovaLogSinkTPConfiguration* **XynovaLogSinkTPConfiguration**

Thread-pool sink configuration.

typedef struct *XynovaLogWriterConfiguration* **XynovaLogWriterConfiguration**

Base writer configuration.

typedef struct *XynovaLogFileWriterConfiguration* **XynovaLogFileWriterConfiguration**

File writer configuration.

typedef struct *XynovaLogConsoleWriterConfiguration* **XynovaLogConsoleWriterConfiguration**

Console writer configuration.

typedef struct *XynovaLogFormatterConfiguration* **XynovaLogFormatterConfiguration**

Formatter configuration.

Reserved for future extension.

```
typedef struct XynovaLogLoggerBackendConfiguration XynovaLogLoggerBackendConfiguration
    Logger backend configuration.
```

```
typedef struct XynovaLogLoggerConfiguration XynovaLogLoggerConfiguration
    Logger creation configuration.
    Defines the relationship between sink, writers and backend logger.
```

## Enums

```
enum [anonymous]
    Logging severity levels.
    Values:

    enumerator XYNova_LOG_LEVEL_TRACE
        Fine-grained diagnostic tracing.

    enumerator XYNova_LOG_LEVEL_DEBUG
        Debug information.

    enumerator XYNova_LOG_LEVEL_INFO
        Normal information messages.

    enumerator XYNova_LOG_LEVEL_WARN
        Warning messages.

    enumerator XYNova_LOG_LEVEL_ERROR
        Error messages.

    enumerator XYNova_LOG_LEVEL_FATAL
        Fatal failures.
```

```
struct XynovaLogDateTimeRaw
    #include <log_types.h> Raw timestamp representation.
    Uses UNIX epoch time with microsecond precision.
```

## Public Members

```
uint64_t seconds
    Seconds since UNIX epoch UTC.

uint32_t microseconds
    Microsecond offset.
```

struct **XynovaLogLogRecordMeta**

*#include <log\_types.h>* Metadata of a log record.

### Public Members

*XynovaLogDateTimeRaw* **timestamp**

Log timestamp.

*XynovaLogLevel* **level**

Log severity level.

**uint64\_t thread\_id**

Thread identifier.

const char \***file**

Source file name.

**uint32\_t line**

Source line number.

const char \***func**

Function name.

struct **XynovaLogLogRecord**

*#include <log\_types.h>* Complete log record.

Delivered to user callback handlers.

### Public Members

*XynovaLogLogRecordMeta* **meta**

Log metadata.

const char \***message**

UTF-8 message string.

struct **XynovaLogSinkConfiguration**

*#include <log\_types.h>* Common sink configuration.

Defines queue and worker loop timing parameters.

### Public Members

**uint64\_t block\_obtain\_timeout\_us**

Block obtain timeout.

`uint64_t writing_task_push_timeout_us`

Write task push timeout.

`uint64_t writing_task_wait_timeout_us`

Write task wait timeout.

`uint64_t min_looping_task_idle_interval_us`

Minimum idle interval.

`uint64_t max_looping_task_idle_interval_us`

Maximum idle interval.

`uint32_t max_continuous_push_retry_time_ms`

Maximum retry duration.

struct **XynovaLogSinkTPConfiguration**

*#include <log\_types.h>* Thread-pool sink configuration.

### Public Members

*XynovaLogSinkConfiguration* **base\_conf**

Common sink configuration.

`uint32_t num_threads`

Worker thread count.

`uint32_t max_queue_size`

Maximum queue size.

struct **XynovaLogWriterConfiguration**

*#include <log\_types.h>* Base writer configuration.

### Public Members

`const char *formater`

Formatter instance name.

*XynovaBool* **auto\_flush**

Enable automatic flush.

`uint32_t flush_interval_ms`

Flush interval in milliseconds.

```
struct XynovaLogFileWriterConfiguration
    #include <log_types.h> File writer configuration.
```

### Public Members

*XynovaLogWriterConfiguration* **base\_conf**  
Base writer configuration.

const char \***file\_path**  
Output file path.

uint64\_t **max\_file\_size**  
Maximum file size before rotation.

uint32\_t **max\_roll\_files**  
Number of retained files.

```
struct XynovaLogConsoleWriterConfiguration
    #include <log_types.h> Console writer configuration.
```

### Public Members

*XynovaLogWriterConfiguration* **base\_conf**  
Base writer configuration.

*XynovaBool* **use\_stderr**  
Output to stderr when true.

```
struct XynovaLogFormatterConfiguration
    #include <log_types.h> Formatter configuration.
    Reserved for future extension.
```

### Public Members

uint32\_t **reserved**  
Reserved field.

```
struct XynovaLogLoggerBackendConfiguration
    #include <log_types.h> Logger backend configuration.
```

### Public Members

uint32\_t **pool\_size**  
Log memory pool size.

`uint32_t block_size`

Records per block.

`uint32_t message_size`

Maximum message size.

`uint32_t max_continuous_repeated_log`

Maximum repeated logs.

`uint32_t flush_interval_us`

Periodic flush interval.

`float queue_size_multiple`

Queue size multiplier.

struct **XynovaLogLoggerConfiguration**

*#include <log\_types.h>* Logger creation configuration.

Defines the relationship between sink, writers and backend logger.

### Public Members

`const char *sink_name`

Sink instance name.

`const char **writer_name_list`

NULL-terminated writer names.

`const char *lb_type`

Backend logger type.

*XynovaLogLevel* `level`

Minimum output level.

*XynovaLogLoggerBackendConfiguration* `lb_config`

Backend configuration.

#### 4.2.2.1.8 错误码

**XynovaErrorCode** 是公共 C 与 C++ API 函数返回的标准错误码类型。它以 `int32_t` 存储，以保证 ABI 兼容。可使用 `xynovaErrorCodeToString()` 将错误码转换为可读字符串。

## 通用 (General)

| 常量 (Constant)                         | 值 (Value) | 说明 (Description) |
|---------------------------------------|-----------|------------------|
| XYNOVA_ERROR_CODE_OK                  | 0         | 无错误              |
| XYNOVA_ERROR_CODE_UNKNOWN             | -1        | 未知错误             |
| XYNOVA_ERROR_CODE_INVALID_ARG         | -2        | 无效参数             |
| XYNOVA_ERROR_CODE_NOT_IMPLEMENTED     | -3        | 功能未实现            |
| XYNOVA_ERROR_CODE_TIMEOUT             | -4        | 操作超时             |
| XYNOVA_ERROR_CODE_NOT_FOUND           | -5        | 资源未找到            |
| XYNOVA_ERROR_CODE_ALREADY_EXISTS      | -6        | 资源已存在            |
| XYNOVA_ERROR_CODE_NO_MEMORY           | -7        | 内存分配失败           |
| XYNOVA_ERROR_CODE_IO                  | -8        | 输入/输出错误          |
| XYNOVA_ERROR_CODE_BUSY                | -9        | 资源忙              |
| XYNOVA_ERROR_CODE_INTERRUPTED         | -10       | 操作被中断            |
| XYNOVA_ERROR_CODE_OUT_OF_RANGE        | -11       | 值超出允许范围          |
| XYNOVA_ERROR_CODE_PERMISSION_DENIED   | -12       | 权限被拒绝            |
| XYNOVA_ERROR_CODE_UNSUPPORTED         | -13       | 操作不支持            |
| XYNOVA_ERROR_CODE_ALREADY_INITIALIZED | -14       | 已初始化             |
| XYNOVA_ERROR_CODE_DATA_OUT_OF_DATE    | -15       | 数据已过期            |
| XYNOVA_ERROR_CODE_NO_DATA_UPDATED     | -16       | 数据未更新            |
| XYNOVA_ERROR_CODE_INTERNAL_ERROR      | -17       | SDK 内部配置或逻辑错误    |

## 日志 (Logging)

| 常量 (Constant)                                  | 值 (Value) | 说明 (Description) |
|------------------------------------------------|-----------|------------------|
| XYNOVA_ERROR_CODE_LOG_SINK_ALREADY_EXISTS      | -1000     | 日志 sink 已存在      |
| XYNOVA_ERROR_CODE_LOG_WRITER_ALREADY_EXISTS    | -1001     | 日志 writer 已存在    |
| XYNOVA_ERROR_CODE_LOG_FORMATTER_ALREADY_EXISTS | -1002     | 日志 formatter 已存在 |
| XYNOVA_ERROR_CODE_LOG_LOGGER_ALREADY_EXISTS    | -1003     | 日志 logger 已存在    |
| XYNOVA_ERROR_CODE_LOG_SINK_NOT_FOUND           | -1010     | 日志 sink 未找到      |
| XYNOVA_ERROR_CODE_LOG_WRITER_NOT_FOUND         | -1011     | 日志 writer 未找到    |
| XYNOVA_ERROR_CODE_LOG_FORMATTER_NOT_FOUND      | -1012     | 日志 formatter 未找到 |
| XYNOVA_ERROR_CODE_LOG_LOGGER_NOT_FOUND         | -1013     | 日志 logger 未找到    |
| XYNOVA_ERROR_CODE_LOG_LOGGER_CREATION_FAILED   | -1014     | 日志 logger 创建失败   |

## 通信 (Communication)

| 常量 (Constant)                              | 值 (Value) | 说明 (Description) |
|--------------------------------------------|-----------|------------------|
| XYNOVA_ERROR_CODE_COMM_INIT_FAILED         | -2000     | 通信初始化失败          |
| XYNOVA_ERROR_CODE_COMM_SHUTDOWN_FAILED     | -2001     | 通信关闭失败           |
| XYNOVA_ERROR_CODE_COMM_INVALID_CONFIG      | -2002     | 无效的通信配置          |
| XYNOVA_ERROR_CODE_COMM_ALREADY_INITIALIZED | -2003     | 通信已初始化           |
| XYNOVA_ERROR_CODE_COMM_NOT_INITIALIZED     | -2004     | 通信未初始化           |
| XYNOVA_ERROR_CODE_COMM_NOT_LOOPING         | -2005     | 通信循环未运行          |
| XYNOVA_ERROR_CODE_COMM_NO_MAPPING          | -2006     | 未找到通信映射          |
| XYNOVA_ERROR_CODE_COMM_RT_LOOP_TIMEOUT     | -2007     | 实时循环停滞或未运行       |

## CAN

| 常量 (Constant)                                   | 值 (Value) | 说明 (Description)  |
|-------------------------------------------------|-----------|-------------------|
| XYNOVA_ERROR_CODE_CAN_DRIVER_NOT_FOUND          | -3000     | CAN 驱动未找到         |
| XYNOVA_ERROR_CODE_CAN_DRIVER_ALREADY_REGISTERED | -3001     | CAN 驱动已注册         |
| XYNOVA_ERROR_CODE_CAN_INVALID_VTABLE            | -3002     | 无效的 CAN 驱动 vtable |
| XYNOVA_ERROR_CODE_CAN_OPEN_FAILED               | -3003     | 打开 CAN 接口失败       |
| XYNOVA_ERROR_CODE_CAN_CLOSE_FAILED              | -3004     | 关闭 CAN 接口失败       |
| XYNOVA_ERROR_CODE_CAN_SEND_FAILED               | -3005     | 发送 CAN 消息失败       |
| XYNOVA_ERROR_CODE_CAN_RECV_FAILED               | -3006     | 接收 CAN 消息失败       |
| XYNOVA_ERROR_CODE_CAN_SET_FILTERS_FAILED        | -3007     | 设置 CAN 过滤器失败      |
| XYNOVA_ERROR_CODE_CAN_NOT_OPEN                  | -3008     | CAN 接口未打开         |
| XYNOVA_ERROR_CODE_CAN_ALREADY_OPEN              | -3009     | CAN 接口已打开         |
| XYNOVA_ERROR_CODE_CAN_INVALID_CONFIG            | -3010     | 无效的 CAN 配置        |
| XYNOVA_ERROR_CODE_CAN_NO_DATA                   | -3011     | 无 CAN 数据可用        |
| XYNOVA_ERROR_CODE_CAN_INVALID_VTABLE_NAME       | -3012     | 无效的 CAN vtable 名称 |
| XYNOVA_ERROR_CODE_CAN_SET_BUFFER_FAILED         | -3013     | 设置接收/发送缓冲区失败      |
| XYNOVA_ERROR_CODE_CAN_NO_TX_BUFFERS             | -3014     | 发送消息时无可用缓冲区       |
| XYNOVA_ERROR_CODE_CAN_REQUEST_FAILED            | -3015     | CAN 请求失败          |
| XYNOVA_ERROR_CODE_CAN_REQUEST_VERIFY_FAILED     | -3016     | CAN 请求校验失败        |
| XYNOVA_ERROR_CODE_CAN_ERROR_FRAME               | -3017     | 收到 CAN 错误帧        |
| XYNOVA_ERROR_CODE_CAN_BUS_OFF                   | -3018     | CAN 总线关闭          |
| XYNOVA_ERROR_CODE_CAN_BUS_ERROR                 | -3019     | 通用 CAN 总线错误       |

## 从站 (Slave)

| 常量 (Constant)                                       | 值 (Value) | 说明 (Description) |
|-----------------------------------------------------|-----------|------------------|
| XYNOVA_ERROR_CODE_SLAVE_CREATE_FAILED               | -4000     | 创建从站失败           |
| XYNOVA_ERROR_CODE_SLAVE_ADD_SCHEDULE_COMMAND_FAILED | -4001     | 添加调度命令失败         |
| XYNOVA_ERROR_CODE_SLAVE_INIT_FAILED                 | -4002     | 初始化从站失败          |

## 设备 (Device)

| 常量 (Constant)                                         | 值 (Value) | 说明 (Description) |
|-------------------------------------------------------|-----------|------------------|
| XYNOVA_ERROR_CODE_DEVICE_CREATE_FAILED                | -5000     | 创建设备失败           |
| XYNOVA_ERROR_CODE_DEVICE_CREATE_SLAVE_FAILED          | -5001     | 为设备创建从站失败        |
| XYNOVA_ERROR_CODE_DEVICE_INIT_SCHEDULE_COMMAND_FAILED | -5002     | 初始化调度命令失败        |

## 灵巧手 (Dexterous Hand)

| 常量 (Constant)                                   | 值 (Value) | 说明 (Description) |
|-------------------------------------------------|-----------|------------------|
| XYNOVA_ERROR_CODE_HAND_SET_MODE_FAILED          | -6000     | 设置手部模式失败         |
| XYNOVA_ERROR_CODE_HAND_SET_SYSTEM_ENABLEMENT_FA | -6001     | 设置系统使能失败         |
| XYNOVA_ERROR_CODE_HAND_SET_BACK_TO_ZERO_FAILED  | -6002     | 复位手部至零位失败        |
| XYNOVA_ERROR_CODE_HAND_CLEAR_ERRORS_FAILED      | -6003     | 清除手部错误失败         |
| XYNOVA_ERROR_CODE_HAND_NO_DATA_UPDATED          | -6004     | 手部数据未更新          |
| XYNOVA_ERROR_CODE_HAND_DATA_OUT_OF_DATE         | -6005     | 手部数据已过期          |
| XYNOVA_ERROR_CODE_HAND_CREATE_SLAVE_FAILED      | -6006     | 创建手部从站设备失败       |

## 协议栈 (Protocol Stack)

| 常量 (Constant)                                | 值 (Value) | 说明 (Description) |
|----------------------------------------------|-----------|------------------|
| XYNOVA_ERROR_CODE_PROTOCOL_STACK_INIT_FAILED | -7000     | 初始化协议栈失败         |

### 另请参阅

- 基础设施 (C API)
- 日志 (C API)
- 术语
- 示例
- 灵巧手 (C API)

### 4.2.2.2 C++ API

C++ API 包含以下模块:

#### 4.2.2.2.1 C++ API 参考

C++ API 提供围绕 C API 的现代面向对象封装, 包括:

- 基于 RAII 的资源管理
- 高级设备与日志类
- 类型安全与便捷函数

### 概述

C++ API 包含以下模块:

- 通信 (Communication): *Manager* 单例与 *DeviceHandle* RAII 封装。
- 手部控制 (Hand Control): 用于关节级控制与状态上报的 *dexterous\_hand::Device*。
- 基础设施 / 日志 (Infrastructure / Logging): *LogManager* 及相关配置类型。

## 说明

- C++ API 类大多是相应 C 结构与函数的薄封装。
- 内存与设备句柄管理由 RAII 自动完成。
- C++ API 与 C API 保持一致的命名与模块层级，便于交叉参考。

### 4.2.2.2.2 通信 (C++ API)

**通信模块**提供用于管理 Xynova 通信接口、主站与设备的高级 C++ 封装。

它构建在 C API 之上，提供：

- 设备句柄的 RAII 管理
- 用于主站/设备初始化与运行时控制的单例 Manager
- 同步与异步设备操作的类型安全接口
- 通过专用辅助类型提供 CAN 与传输支持

## 公共头文件

本模块对外暴露的 C++ 头文件包括：

- `xynova/cpp/communication/Manager.hpp` — 通信管理器单例与设备句柄封装
- `xynova/cpp/communication/types.h` — C++ 通信配置类型

CAN 传输辅助类型见 [CAN 通信](#) 页面：

- `xynova/cpp/communication/can/types.h` — CAN 传输辅助类型

## API 参考

C++ wrapper for Xynova Communication Manager.

Provides RAII resource management and type-safe C++ interfaces for the Xynova communication manager C API.

The module supports:

- Communication manager initialization
- Master runtime control
- Device handle management
- Firmware upgrade

➡ 参见

*XynovaCommunicationManagerConfig*

 参见

*XynovaCommunicationManager\_init*

#### Version

1.0.0

#### Date

2025-10-29

#### Author

Xynova

namespace **xynova**

namespace **communication**

class **DeviceHandle**

*#include <Manager.hpp>* RAII wrapper for C device handle.

Automatically releases the underlying C API handle when the object is destroyed.

#### Public Functions

inline explicit **DeviceHandle** (*XynovaDeviceHandle* \*handle = nullptr)

inline **~DeviceHandle** ()

**DeviceHandle** (const *DeviceHandle*&) = delete

*DeviceHandle* &**operator=** (const *DeviceHandle*&) = delete

inline **DeviceHandle** (*DeviceHandle* &&other) noexcept

inline *DeviceHandle* &**operator=** (*DeviceHandle* &&other) noexcept

inline *XynovaDeviceHandle* \***get** () const noexcept

inline explicit **operator bool** () const noexcept

#### Private Members

*XynovaDeviceHandle* \***i\_handle** = {nullptr}

class **Manager**

*#include <Manager.hpp>* Communication manager singleton.

Provides high-level C++ interfaces for communication lifecycle, device discovery and firmware operations.

## Public Functions

**Manager** (const *Manager*&) = delete

*Manager* &**operator=** (const *Manager*&) = delete

inline *XynovaErrorCode* **init** (const *ManagerConfig* &config)

Initialize communication manager.

Converts C++ configuration objects into C-compatible structures and initializes communication masters and devices.

### 备注

Temporary storage is maintained during initialization to guarantee C API pointer validity.

### Parameters

**config** -- C++ manager configuration.

### Returns

*XynovaErrorCode*.

inline *XynovaErrorCode* **run** (const std::string &master\_name)

Start communication loop.

### Parameters

**master\_name** -- Master name.

### Returns

*XynovaErrorCode*.

inline *XynovaErrorCode* **stop** (const std::string &master\_name)

Stop communication loop.

### Parameters

**master\_name** -- Master name.

### Returns

*XynovaErrorCode*.

template<typename T>

inline std::shared\_ptr<T> **getDevice** (const std::string &master\_name, const std::string &type, const std::string &device\_name)

Get device instance.

Wraps the C API device handle into a C++ RAII object.

### Template Parameters

**T** -- Device wrapper type.

### Parameters

- **master\_name** -- Master name.
- **type** -- Device type.
- **device\_name** -- Device name.

### Returns

Shared pointer to device instance. Returns nullptr when device is unavailable.

inline *XynovaErrorCode* **upgradeFirmware** (const std::string &master\_name, uint32\_t slave\_id, const std::string &file)

Upgrade slave firmware.

### Parameters

- **master\_name** -- Master name.

- **slave\_id** -- Slave identifier.
- **file** -- Firmware file path.

**Returns***XynovaErrorCode.***Public Static Functions**static inline *Manager* &**instance** ()

Get singleton instance.

**Returns***Manager* instance.**Private Functions****Manager** () = default~**Manager** () = default

C++ wrapper definitions for Xynova communication system configuration.

This header provides modern C++ representations of Xynova communication configuration structures, bridging the low-level C API (*xynova/c/comm\_types.h*) with STL-based and type-safe C++ structures.

```
namespace xynova
```

```
    namespace communication
```

**Enums**enum class **CommType** : uint32\_t

Supported communication transport types.

*Values:*enumerator **Can**

Controller Area Network

struct **CommConfig***#include <types.h>* Communication interface configuration.

Provides a common configuration layer for different communication backends.

**Public Members***CommType* **type**

Communication type

std::string **sub\_type**

Communication driver subtype, e.g. "SockCAN"

*can::CanConfig* **can**  
CAN specific configuration

std::string **logger\_name**  
Logger name

struct **MasterConfig**

*#include <types.h>* Configuration of a communication master.

A master represents a communication controller managing one or more devices.

### Public Members

std::string **name**  
Master name

*CommConfig* **comm**  
Communication configuration

uint64\_t **period\_us**  
Control loop period in microseconds

int32\_t **cpu\_index**  
CPU affinity index, -1 disables binding

struct **ResponseTimeSetting**

*#include <types.h>* Communication response timeout and retry configuration.

This structure corresponds to *XynovaCommunicationResponseTimeSetting*.

### Public Members

uint32\_t **user\_max\_timeout\_us**  
Maximum timeout for user API calls

uint32\_t **user\_poll\_interval\_us**  
User polling interval

uint32\_t **execute\_timeout\_us**  
Backend request timeout

uint32\_t **execute\_max\_retry**  
Backend retry count

```
uint32_t execute_poll_interval_us
```

Backend polling interval

```
struct SlaveConfig
```

*#include <types.h>* Configuration of a communication slave device.

### Public Members

```
std::string type
```

Slave type or model name

```
uint32_t id
```

Slave identifier

```
ResponseTimeSetting general_time_setting
```

General response timing

```
ResponseTimeSetting fast_time_setting
```

Fast response timing

```
std::string logger_name
```

Logger name

```
struct SchedulingCommand
```

*#include <types.h>* Periodic command scheduling configuration.

Defines commands that are periodically sent to or reported from devices.

### Public Members

```
std::string command
```

Command name

```
ResponseTimeSetting response_time_setting
```

Response timing configuration

```
struct DeviceConfig
```

*#include <types.h>* Logical device configuration.

A device may contain multiple communication slaves and scheduled commands.

### Public Members

```
std::string type
```

Device type

```

std::string name
    Device name

std::string master_name
    Associated master name

std::vector<SlaveConfig> slaves
    Slave configuration list

std::vector<SchedulingCommand> commands
    Scheduled command list

```

struct **ManagerConfig**

*#include <types.h>* Complete communication manager configuration.

Contains all communication masters and logical devices required to initialize the communication subsystem.

 参见

*xynova::communication::Manager*

### Public Members

```

std::vector<MasterConfig> masters
    Master configuration list

std::vector<DeviceConfig> devices
    Device configuration list

```

#### 4.2.2.2.3 CAN 通信 (C++ API)

**CAN 模块**为 Xynova SDK 内的 CAN 通信提供类型与工具，主要用于 *Manager* 与设备接口的 CAN 帧收发。

本模块定义：

- CAN 消息结构
- CAN 错误码
- CAN 设备辅助

头文件

```

namespace xynova

    namespace communication

```

namespace **can**

## Enums

enum class **FrameType** : uint32\_t

CAN frame identifier type.

*Values:*

enumerator **Standard**

Standard 11-bit CAN identifier

enumerator **Extended**

Extended 29-bit CAN identifier

enum class **CanMask** : uint32\_t

CAN identifier mask.

*Values:*

enumerator **Standard**

Standard 11-bit identifier mask

enumerator **Extended**

Extended 29-bit identifier mask

struct **CanConfig**

*#include <types.h>* CAN interface configuration.

Supports both classical CAN and CAN-FD configuration.

## Public Members

std::string **iface**

CAN interface name, e.g. "can0"

bool **manager\_can** = false

Whether SDK manages CAN interface setup

uint32\_t **bitrate** = 0

Arbitration bitrate in bits per second

uint32\_t **dbitrate** = 0

CAN-FD data bitrate in bits per second

```
float sample_point = 0.75f
    Arbitration sample point ratio

float dsample_point = 0.75f
    CAN-FD data sample point ratio

struct RecvFilter
    #include <types.h> CAN receive filter.
    Filters CAN frames using identifier and mask:
```

```
(received_id & mask) == (id & mask)
```

### Public Members

*FrameType* **type**

CAN frame type

uint32\_t **id**

CAN identifier

uint32\_t **mask**

CAN identifier mask

#### 4.2.2.2.4 手部控制 (C++ API)

##### 概述

**手部控制模块 (C++ API)** 为控制 Xynova 灵巧手设备提供现代、面向对象的封装。

该层构建在稳定的 **C ABI** 之上，提供：

- RAPII 设备生命周期管理
- 强类型状态结构
- 更安全的接口（引用而非裸指针）
- 更清晰的同步 / 异步 API
- 与现代 C++ 项目的更好集成

与 **C API** 相比，该层：

- 减少样板代码
- 提升可读性
- 防止常见误用
- 保持零运行时开销

开发 C++ 机器人应用时请使用该模块。

## 与 C API 的关系

内部实现中，所有 C++ 接口都委托给对应的 C 函数：

示例：

```
XynovaErrorCode Device::getJointAngle(
    AngleStates& joints,
    uint64_t& ts)
{
    return XynovaDexterousHand_getJointAngle(...);
}
```

因此：

- C API → ABI 稳定 / 跨语言
- C++ API → 易用封装

请根据项目需求选择。

### 4.2.2.2.5 灵巧手 (C++ API)

#### 概述

**灵巧手 C++ 模块**提供当前可用的 Xynova 灵巧手设备控制 C++ 封装。

当前实现暴露以下设备级操作：

- 设备生命周期: `run()` / `stop()` / `valid()`
- 产品信息: `getProductModel()` / `getProductSN()` / `getHardwareVersion()` / `getSoftwareVersion()` / `getHandType()`
- 错误状态: `getErrorState()`
- 控制模式设置 (全部关节): `setJointControlMode()`
- 同步角度命令: `setJointAngle()`
- 异步角度命令: `setJointAngleAsync()`
- 关节反馈: `getJointAngle()`
- 触觉反馈: `enableTactileWrench()` / `getTactileWrench()` / `enableTactileTaxel()` / `getTactileTaxel()`
- 接近觉反馈: `enableRange()` / `getRange()`

#### 设备 (Device)

高级设备封装类。

提供：

- 异步关节控制
- 反馈查询
- 产品信息
- 错误状态监控
- 触觉反馈

- 接近觉反馈
- 设备生命周期管理

C++ wrapper for Xynova Dexterous Hand device.

This header provides a high-level C++ interface for controlling and monitoring Xynova Dexterous Hand devices. It wraps the C API and provides RAII-based resource management together with strongly typed interfaces.

```
namespace xynova
```

```
    namespace hand
```

```
        namespace dexterous_hand
```

```
            class Device
```

*#include <Device.hpp>* RAII wrapper for Xynova Dexterous Hand device.

This class provides a type-safe C++ interface over the C API. The underlying device handle is automatically released when the object is destroyed.

### Public Types

```
using DeviceHandle = communication::DeviceHandle
```

### Public Functions

```
inline explicit Device (DeviceHandle &&handle) noexcept
```

Construct a device wrapper.

#### Parameters

**handle** -- *Device* handle.

```
Device (const Device&) = delete
```

```
Device &operator= (const Device&) = delete
```

```
Device (Device&&) noexcept = delete
```

```
Device &operator= (Device&&) noexcept = delete
```

```
inline bool valid () const noexcept
```

Check whether the device handle is valid.

#### Returns

true if valid, otherwise false.

```
inline XynovaErrorCode run ()
```

Start dexterous hand device.

#### Returns

*XynovaErrorCode*

```
inline XynovaErrorCode stop ()
```

Stop dexterous hand device.

#### Returns

*XynovaErrorCode*

```

inline XynovaErrorCode getProductModel (std::string &product_model)
    Get product model.
    Parameters
        product_model -- Output product model string.
    Returns
        XynovaErrorCode

inline XynovaErrorCode getProductSN (std::string &product_sn)
    Get product serial number.
    Parameters
        product_sn -- Output serial number string.
    Returns
        XynovaErrorCode

inline XynovaErrorCode getHardwareVersion (std::string &version)
    Get hardware version.
    Parameters
        version -- Output hardware version.
    Returns
        XynovaErrorCode

inline XynovaErrorCode getSoftwareVersion (std::string &version)
    Get software version.
    Parameters
        version -- Output software version.
    Returns
        XynovaErrorCode

inline XynovaErrorCode getHandType (HandType &type)
    Get hand type.
    Parameters
        type -- Output hand type.
    Returns
        XynovaErrorCode

inline XynovaErrorCode getErrorState (ErrorState &state)
    Get current device error state.

    Retrieves the latest global warning, global fault and joint error information from the dex-
    terous hand.
    Parameters
        state -- Output error state.
    Returns
        XynovaErrorCode

inline XynovaErrorCode setJointControlMode (ControlMode mode)
    Set control mode for all joints.
    Parameters
        mode -- Joint control mode.
    Returns
        XynovaErrorCode

inline XynovaErrorCode enableTactileWrench (bool enable)
    Enable or disable tactile wrench reporting.

```

Controls whether tactile wrench feedback is reported by the device.

**Parameters**

**enable** --

- true: Enable tactile wrench reporting.
- false: Disable tactile wrench reporting.

**Returns**

*XynovaErrorCode*

inline *XynovaErrorCode* **enableTactileTaxel** (bool enable)

Enable or disable tactile taxel reporting.

Controls whether tactile taxel feedback is reported by the device.

**Parameters**

**enable** --

- true: Enable tactile taxel reporting.
- false: Disable tactile taxel reporting.

**Returns**

*XynovaErrorCode*

inline *XynovaErrorCode* **enableRange** (bool enable)

Enable or disable range reporting.

Controls whether range feedback is reported by the device.

**Parameters**

**enable** --

- true: Enable range reporting.
- false: Disable range reporting.

**Returns**

*XynovaErrorCode*

inline *XynovaErrorCode* **setJointAngle** (const *AngleControl* &angle)

Set joint angle command.

**Parameters**

**angle** -- Joint angle command.

**Returns**

*XynovaErrorCode*

inline *XynovaErrorCode* **setJointAngleAsync** (const *AngleControl* &angle)

Asynchronously set joint angle command.

The command is queued and returns immediately.

**Parameters**

**angle** -- Joint angle command.

**Returns**

*XynovaErrorCode*

inline *XynovaErrorCode* **getJointAngle** (*AngleStates* &states, uint64\_t &timestamp\_us)

Get current joint angle states.

**Parameters**

- **states** -- Output joint angle states.
- **timestamp\_us** -- Output timestamp in microseconds.

**Returns**

*XynovaErrorCode*

inline *XynovaErrorCode* **getTactileWrench** (*TactileWrenchStates* &states, uint64\_t &timestamp\_us)

Get tactile wrench states.

**Parameters**

- **states** -- Output tactile wrench states.
- **timestamp\_us** -- Output timestamp in microseconds.

**Returns**

*XynovaErrorCode*

```
inline XynovaErrorCode getTactileTaxel (TactileTaxelStates &states, uint64_t
                                     &timestamp_us)
```

Get tactile taxel states.

**Parameters**

- **states** -- Output tactile taxel states.
- **timestamp\_us** -- Output timestamp in microseconds.

**Returns**

*XynovaErrorCode*

```
inline XynovaErrorCode getRange (RangeStates &states, uint64_t &timestamp_us)
```

Get range measurement states.

Retrieves two-dimensional range measurement data from the dexterous hand.

The caller provides the range point buffer. The device fills:

- width
- height
- range point data

Points are stored in row-major order:

$$\text{index} = y * \text{width} + x$$

**Parameters**

- **states** -- Output range measurement states.
- **timestamp\_us** -- Output timestamp in microseconds.

**Returns**

*XynovaErrorCode*

**Private Functions**

```
inline XynovaDeviceHandle *c_handle () const noexcept
```

Get underlying C API handle.

**Private Members**

*DeviceHandle* i\_dev\_handle

Managed device handle

C++ type definitions for the Xynova Dexterous Hand.

This file provides strongly typed C++ data types corresponding to Xynova Dexterous Hand C API structures, including joint identifiers, control modes, joint commands, joint states, and tactile sensing data.

 参见

xynova/c/dexterous\_hand\_types.h

**Version**

1.0.0

**Date**

2026-07-17

**Author**

Xynova

namespace **xynova**namespace **hand**namespace **dexterous\_hand****Typedefs**using **JointID** = *XynovaDexterousHandJointID*

Joint identifier type.

using **TactileID** = *XynovaDexterousHandTactileID*

Tactile region identifier type.

using **ErrorCode** = *XynovaDexterousHandErrorCode*

Joint error code type.

using **WarningFlags** = *XynovaDexterousHandWarningFlags*

Global warning flags type.

using **FaultFlags** = *XynovaDexterousHandFaultFlags*

Global fault flags type.

template<typename **T**>using **JointValueArray** = std::array<*infra::tools::IdValue<T>*, *JOINT\_NUM*>

Fixed-size joint value array.

Each element contains:

- Joint identifier.
- Strongly typed joint value.

**Template Parameters****T** -- Joint value type.

```

using HandType = XynovaDexterousHandType
    Hand type.

using ControlMode = XynovaDexterousHandControlMode
    Joint control mode type.

using RangePointStatus = XynovaDexterousHandRangePointStatus
    Range point measurement status type.

using WrenchDimension = XynovaDexterousHandWrenchDimension
    Tactile wrench dimension type.

using ControlModeCommand = JointValueArray<ControlMode>
    Joint control mode command.

    Each element stores:
    • Joint identifier.
    • Joint control mode.

using AngleControl = JointValueArray<float>
    Joint angle command.

    Each element stores:
    • Joint identifier.
    • Target joint angle.
    Unit: rad.

using AngleStates = JointValueArray<float>
    Joint angle feedback state.

    Each element stores:
    • Joint identifier.
    • Measured joint angle.
    Unit: rad.

using TactileWrenchStates = std::array<TactileWrench, TACTILE_NUM>
    Tactile resultant wrench states.

    Each element represents one tactile region.

using TactileTaxelStates = std::vector<TactileTaxel>
    Dynamic tactile taxel state array.

    The container owns tactile taxel data.

using RangePoint = XynovaDexterousHandRangePoint
    Range point measurement.

    Represents distance measurement from a single range sensing point.

```

## Enums

enum class **JointError** : uint16\_t

Joint error flags.

*Values:*

enumerator **None**

enumerator **StallProtection**

enumerator **OverTemperature**

enumerator **OverCurrent**

enumerator **MotorAbnormal**

enumerator **Communication**

enumerator **EncoderAbnormal**

## Variables

static constexpr size\_t **JOINT\_NUM** = XYNOVA\_DEXTEROUS\_HAND\_JOINT\_NUM

Number of joints supported by the dexterous hand.

static constexpr size\_t **TACTILE\_NUM** = XYNOVA\_DEXTEROUS\_HAND\_TACTILE\_NUM

Number of tactile regions supported by the dexterous hand.

static constexpr size\_t **TACTILE\_TOTAL\_TAXEL\_NUM** =  
XYNOVA\_DEXTEROUS\_HAND\_TACTILE\_TOTAL\_TAXEL\_NUM

Number of tactile taxels supported by the dexterous hand.

static constexpr size\_t **RANGE\_POINT\_NUM** =  
XYNOVA\_DEXTEROUS\_HAND\_RANGE\_POINT\_NUM

Number of range points supported by the default range sensor.

class **JointErrorCode**

*#include <types.h>* Joint error code wrapper.

Provides type-safe access to joint error flags.

## Public Functions

inline constexpr **JointErrorCode** (*ErrorCode* code = 0) noexcept

inline void **fromC** (const *XynovaTypedValue* &value)

```

inline XynovaTypedValue toC () const

inline constexpr bool has (JointError error) const noexcept

inline constexpr bool ok () const noexcept

inline constexpr ErrorCode value () const noexcept

```

### Private Members

```
ErrorCode i_code = {0}
```

struct **JointErrorState**

*#include <types.h>* Joint error state.

Contains joint identifier and corresponding error code.

### Public Members

```
JointID id = {0}
```

Joint identifier

```
JointErrorCode error_code = {}
```

Joint error code

struct **ErrorState**

*#include <types.h>* Dexterous hand error state.

Contains global warning, global fault and joint error information.

### Public Functions

```
inline bool hasFault () const noexcept
```

Check whether any global fault exists.

```
inline bool hasWarning () const noexcept
```

Check whether any global warning exists.

```
inline JointErrorState &joint (JointID id) noexcept
```

Get joint error by joint identifier.

```
inline const JointErrorState &joint (JointID id) const noexcept
```

Get joint error by joint identifier.

### Public Members

```
uint64_t timestamp_us = {0}
```

Error timestamp (us): system monotonic clock

*WarningFlags* **warning\_flags** = {0}

Global warning flags

*FaultFlags* **fault\_flags** = {0}

Global fault flags

`std::array<JointErrorState, JOINT_NUM>` **joint\_error\_codes** = {}

Joint error states

struct **TactileWrench**

*#include <types.h>* Tactile resultant wrench data.

Represents force/torque measurement of a tactile region.

### Public Members

`uint64_t` **timestamp\_us** = {0}

Sampling timestamp (us): system monotonic clock

*TactileID* **region** = {0}

Tactile region identifier

*WrenchDimension* **dimension** = {0}

Wrench dimension

*infra::tools::Value<float, 6>* **wrench**

Force/torque values: force unit: N, torque unit: Nm

struct **TactileTaxel**

*#include <types.h>* Tactile taxel measurement.

Represents force/torque data from a single tactile sensing element.

### Public Members

`uint64_t` **timestamp\_us** = {0}

Sampling timestamp (us): system monotonic clock

*TactileID* **region** = {0}

Tactile region identifier

`uint16_t` **taxel\_id** = {0}

Taxel identifier

*WrenchDimension* **dimension** = {0}

Wrench dimension

```
uint8_t reserved = {0}
    Reserved

    infra::tools::Value<float, 6> wrench
    Force/torque values: force unit: N, torque unit: Nm

struct RangeStates
    #include <types.h> Dynamic range point states.
    Represents a two-dimensional range map stored in row-major order.
    The point corresponding to coordinate (x, y) is stored at: index = y * width + x

    Public Functions

    inline explicit RangeStates (size_t capacity = 0)

    Public Members

    uint16_t width = {0}
    Range map width

    uint16_t height = {0}
    Range map height

    std::vector<RangePoint> points
    Range points
```

另请参阅

- 通信 (*C++ API*)
- 基础设施 (*C++ API*)
- 示例

#### 4.2.2.2.6 基础设施 (C++ API)

##### 概述

**基础设施模块**提供所有 Xynova SDK 组件共用的运行时工具。

它包含可复用的、与具体设备无关的设施，用于提升：

- 可靠性
- 可观测性
- 性能
- 开发便捷性

该层独立于通信或手部设备，可在通用 C++ 项目中独立使用。

## 公共头文件

本模块对外暴露的 C++ 头文件包括:

- `xynova/cpp/infra/tools/value_types.hpp` 一强类型值封装, 见[值类型](#) 页面

日志入口见[日志](#) 页面:

- `xynova/cpp/infra/log/LogManager.hpp` 一单例日志管理器
- `xynova/cpp/infra/log/types.h` 一日志配置类型

## 另请参阅

- [通信 \(C++ API\)](#)
- [手部控制 \(C++ API\)](#)
- [示例](#)

### 4.2.2.2.7 日志 (C++ API)

#### 概述

**日志模块**为 Xynova SDK 提供现代、高性能的异步日志框架。

它面向以下场景设计:

- 实时机器人系统
- 多线程环境
- 高吞吐消息记录
- 低延迟、最小阻塞

该模块封装底层 C 日志后端, 提供简洁、RAII 友好、面向对象的 C++ 接口。

#### 主要特性

- 基于线程池的 sink
- 文件与控制台 writer
- 自动日志轮转
- 可配置格式化
- 运行时级别控制
- 无锁后端队列
- 零拷贝内存池设计

## 公共头文件

- `xynova/cpp/infra/log/LogManager.hpp`
- `xynova/cpp/infra/log/types.h`

## API 参考

```
namespace xynova
```

```
    namespace infra
```

```
        namespace log
```

```
            class LogManager
```

*#include <LogManager.hpp>* Singleton manager for Xynova logging subsystem.

Provides C++ wrappers around the C logging API while keeping the C ABI boundary stable.

Supported operations:

- Initialize logging objects.
- Create sinks.
- Create writers.
- Create formatters.
- Create logger backends.

### Public Functions

```
inline XynovaErrorCode initDefaultObjects ()
```

Initialize default logging objects.

#### Returns

Operation result code.

```
inline bool isInitialized () const
```

Check whether object logger is initialized.

#### Returns

true if initialized.

```
inline XynovaErrorCode createSinkTP (const std::string &type, const std::string &name,  
                                     const SinkTPConfiguration &config)
```

Create thread-pool sink.

#### Parameters

- **type** -- Registered sink type.
- **name** -- Sink instance name.
- **config** -- Sink configuration.

#### Returns

Operation result code.

```
inline XynovaErrorCode createFileWriter (const std::string &type, const std::string  
                                         &name, const FileWriterConfiguration  
                                         &config)
```

Create file writer.

#### Parameters

- **type** -- Registered writer type.
- **name** -- Writer instance name.
- **config** -- Writer configuration.

#### Returns

Operation result code.

```
inline XynovaErrorCode createConsoleWriter (const std::string &type, const
                                             std::string &name, const
                                             ConsoleWriterConfiguration &config)
```

Create console writer.

**Parameters**

- **type** -- Registered writer type.
- **name** -- Writer instance name.
- **config** -- Writer configuration.

**Returns**

Operation result code.

```
inline XynovaErrorCode createFormatter (const std::string &type, const std::string
                                             &name, const FormatterConfiguration
                                             &config)
```

Create formatter.

**Parameters**

- **type** -- Registered formatter type.
- **name** -- Formatter instance name.
- **config** -- Formatter configuration.

**Returns**

Operation result code.

```
inline XynovaErrorCode createLogger (const std::string &name, const
                                         LoggerConfiguration &conf)
```

Create logger backend.

**Parameters**

- **name** -- Logger instance name.
- **conf** -- Logger configuration.

**Returns**

Operation result code.

```
inline bool hasSink (const std::string &name) const
```

Check sink existence.

**Parameters**

**name** -- Sink instance name.

**Returns**

true if exists.

```
inline bool hasWriter (const std::string &name) const
```

Check writer existence.

**Parameters**

**name** -- Writer instance name.

**Returns**

true if exists.

```
inline bool hasFormatter (const std::string &name) const
```

Check formatter existence.

**Parameters**

**name** -- Formatter instance name.

**Returns**

true if exists.

```
inline bool hasLogger (const std::string &name) const
```

Check logger existence.

**Parameters**

**name** -- Logger instance name.

**Returns**

true if exists.

```
inline XynovaErrorCode setLoggerLevel (const std::string &name, XynovaLogLevel
                                     level)
```

Set logger level.

**Parameters**

- **name** -- Logger instance name.
- **level** -- New log level.

**Returns**

Operation result code.

**Public Static Functions**

```
static inline LogManager &instance ()
```

Get singleton instance.

**Returns**

Singleton *LogManager* reference.

**Private Functions**

```
LogManager () = default
```

```
~LogManager () = default
```

```
LogManager (const LogManager&) = delete
```

```
LogManager (LogManager&&) = delete
```

```
LogManager &operator= (const LogManager&) = delete
```

```
LogManager &operator= (LogManager&&) = delete
```

```
namespace xynova
```

```
    namespace infra
```

```
        namespace log
```

```
            struct SinkConfiguration
```

```
                #include <types.h> Logging sink base configuration.
```

Defines timeout parameters and worker loop intervals used by sink implementations.

**Public Members**

```
uint64_t block_obtain_timeout_us = 0
```

Timeout for obtaining log blocks.

```
uint64_t writing_task_push_timeout_us = 500
```

Timeout for pushing writing tasks.

```
uint64_t writing_task_wait_timeout_us = 500
```

Timeout for waiting writing task completion.

```
uint64_t min_looping_task_idle_interval_us = 1 * 1000
```

Minimum worker loop idle interval in microseconds.

```
uint64_t max_looping_task_idle_interval_us = 5 * 1000
```

Maximum worker loop idle interval in microseconds.

```
uint32_t max_continuous_push_retry_time_ms = 16 * 1000
```

Maximum continuous retry time when pushing logs fails.

```
struct SinkTPConfiguration
```

*#include <types.h>* Thread-pool based sink configuration.

Extends *SinkConfiguration* with worker thread and queue parameters.

### Public Members

```
SinkConfiguration base_conf
```

Common sink configuration.

```
uint32_t num_threads = std::thread::hardware_concurrency()
```

Number of worker threads.

```
uint32_t max_queue_size = 0
```

Maximum queue size.

A value of zero means unlimited.

```
struct WriterConfiguration
```

*#include <types.h>* Base writer configuration.

Shared configuration fields for all log writers.

### Public Members

```
std::string formatter
```

Formatter instance name.

```
bool auto_flush = true
```

Enable automatic flushing.

```
uint32_t flush_interval_ms = 1000
```

Flush interval in milliseconds.

```
struct FileWriterConfiguration
```

*#include <types.h>* File writer configuration.

Configures output to a file with optional log rotation.

### Public Members

*WriterConfiguration* **base\_conf**

Base writer configuration.

std::string **file\_path**

Output file path.

```
uint64_t max_file_size = 10 * 1024 * 1024
```

Maximum file size before rotation.

```
uint32_t max_roll_files = 5
```

Maximum number of rotated files.

```
struct ConsoleWriterConfiguration
```

*#include <types.h>* Console writer configuration.

Configures output to standard output or standard error.

### Public Members

*WriterConfiguration* **base\_conf**

Base writer configuration.

```
bool use_stderr = false
```

Output to stderr when true, otherwise stdout.

```
struct FormaterConfiguration
```

*#include <types.h>* Formatter configuration.

Reserved for future formatter extensions.

### Public Members

```
uint32_t reserved = 0
```

Reserved field.

```
struct LoggerBackendConfiguration
```

*#include <types.h>* Logger backend configuration.

Defines memory pool parameters, buffering behavior and performance tuning options for logger backend instances.

### Public Members

```
uint32_t pool_size
```

Number of blocks in the memory pool.

```
uint32_t block_size
```

Number of records per block.

```
uint32_t message_size
```

Maximum message size.

```
uint32_t max_continuous_repeated_log = std::numeric_limits<uint32_t>::max()
```

Maximum continuous repeated log count.

```
uint32_t flush_interval_us = 500 * 1000
```

Logger flush interval in microseconds.

```
float queue_size_multiple = 1.5
```

Queue size multiplier.

```
struct LoggerConfiguration
```

*#include <types.h>* Logger instance creation configuration.

Contains sink, writer and backend configuration information required to create a logger instance.

### Public Members

```
std::string sink_name
```

Sink instance name.

```
std::vector<std::string> writer_name_list
```

Writer instance names.

```
std::string lb_type
```

Logger backend implementation type.

```
XynovaLogLevel level
```

Minimum log level.

*LoggerBackendConfiguration* **lb\_config**  
Logger backend configuration.

#### 4.2.2.2.8 值类型 (C++ API)

##### 概述

**值类型模块**为 Xynova SDK 的值容器提供强类型 C++ 封装。它属于基础设施层，供灵巧手与通信模块以类型安全的方式交换类型化数据（如关节角度、力分量）。

主要特性:

- `infra::tools::Value<T, N>` 一固定大小的强类型值数组
- `infra::tools::IdValue<T>` 一带标识符的类型化值
- `DataTypeTraits` — C++ 类型与 `XynovaDataType` 的编译期映射

这些封装桥接了兼容 ABI 的 C 值联合体（`XynovaTypedValue` 、 `XynovaIdValue` ）与现代 C++ 接口。

##### 头文件

- `xynova/cpp/infra/tools/value_types.hpp`

##### API 参考

C++ strongly typed value wrappers for Xynova SDK.

This file provides C++ wrappers for C API value types defined in `xynova/c/value_types.h`.

Features:

- Automatic C++ type to `XynovaDataType` mapping.
- Strongly typed scalar and array value containers.
- C API conversion support.
- Identifier value wrappers.

Example:

```
using namespace xynova::infra::tools;

Value<float> angle(30.0f);
Value<float, 3> position({1.0f, 2.0f, 3.0f});

auto c_value = angle.toC();
```

##### Version

1.0.0

##### Date

2026-07-17

##### Defines

**XYNOVA\_DEFINE\_DATA\_TYPE\_TRAIT** (cpp\_type, c\_type)

Define a C++ type to XynovaDataType mapping.

#### Parameters

- **cpp\_type** -- C++ type.
- **c\_type** -- Xynova data type identifier.

namespace **xynova**

namespace **infra**

namespace **tools**

#### Functions

template<typename **T**>

inline void **assignValue** (*XynovaValue* &dst, const *T* &src)

Store C++ value into C API union.

#### Template Parameters

**T** -- *Value* type.

#### Parameters

- **dst** -- Destination C union.
- **src** -- Source C++ value.

template<typename **T**>

inline *T* **readValue** (const *XynovaValue* &src)

Read C API union into C++ value.

#### Template Parameters

**T** -- *Value* type.

#### Parameters

**src** -- Source C union.

#### Returns

Converted C++ value.

template<typename **T**>

struct **DataTypeTraits**

*#include <value\_types.hpp>* Mapping between C++ type and Xynova data type.

Specializations are generated for supported SDK value types.

#### Template Parameters

**T** -- C++ type.

#### Public Static Attributes

static constexpr uint8\_t **value** = *XYNOVA\_DATA\_TYPE\_NONE*

template<typename **T**, size\_t **N** = 1>

class **Value**

*#include <value\_types.hpp>* Strongly typed value wrapper.

This class provides a C++ abstraction over:

- *XynovaTypedValue*
- *XynovaTypedValueArray2*
- *XynovaTypedValueArray3*
- *XynovaTypedValueArray4*

#### Template Parameters

- **T** -- Element type.
- **N** -- Number of elements. Range: 1~4.

#### Public Types

using **Storage** = std::conditional\_t<*N* == 1, *T*, std::array<*T*, *N*>

#### Public Functions

**Value** () = default

Default constructor.

inline explicit **Value** (const *Storage* &value)

Construct from value.

##### Parameters

**value** -- Initial value.

inline void **setValue** (const *Storage* &value)

Set value.

##### Parameters

**value** -- New value.

inline const *Storage* &**value** () const noexcept

Get constant value.

##### Returns

Constant reference.

inline *Storage* &**value** () noexcept

Get mutable value.

##### Returns

Mutable reference.

template<typename **CValue**>

inline bool **fromC** (const *CValue* &value)

Convert from C API value.

The input type must match the template element count.

##### Template Parameters

**CValue** -- C API value type.

##### Parameters

**value** -- Source C API value.

##### Returns

true if conversion succeeds, false otherwise.

inline auto **toC** () const

Convert to C API value.

##### Returns

C API value object.

## Private Members

*Storage* **i\_value** = { }

template<typename **T**, size\_t **N** = 1>

class **IdValue**

*#include <value\_types.hpp>* Strongly typed identifier value wrapper.

Wraps an identifier together with a typed value.

### Template Parameters

- **T** -- Element type.
- **N** -- Number of elements. Range: 1~6.

## Public Types

using **ValueType** = *Value*<**T**, **N**>

using **Storage** = typename *ValueType*::Storage

## Public Functions

**IdValue**() = default

Default constructor.

inline **IdValue**(uint16\_t id, const *Storage* &value)

Construct identifier value.

### Parameters

- **id** -- Identifier.
- **value** -- Initial value.

inline void **setId**(uint16\_t id) noexcept

Set identifier.

### Parameters

**id** -- Identifier value.

inline void **setValue**(const *Storage* &value)

Set contained value.

### Parameters

**value** -- New value.

inline uint16\_t **id**() const noexcept

Get identifier.

### Returns

Identifier value.

inline const *ValueType* &**value**() const noexcept

Get constant value wrapper.

### Returns

Constant reference.

inline *ValueType* &**value**() noexcept

Get mutable value wrapper.

**Returns**

Mutable reference.

```
template<typename CValue>
inline bool fromC (const CValue &value)
```

Convert from C API identifier value.

**Template Parameters**

**CValue** -- C API identifier value type.

**Parameters**

**value** -- Source C API value.

**Returns**

true if conversion succeeds, false otherwise.

```
inline auto toC () const
```

Convert to C API identifier value.

**Returns**

C API identifier value object.

**Private Members**

```
uint16_t i_id = {0}
```

```
ValueType i_value
```

**另请参阅**

- 基础设施 (C++ API)
- 日志 (C++ API)
- 基础设施 (C API)
- 示例

**4.2.3 说明**

- C++ API 类大多是相应 C API 结构与函数的薄封装。
- C++ API 的内存管理在适用场景下由 RAII 处理，减少常见误用。
- 两层 API 共享相同的模块命名与设备约定，切换使用非常直接。

**4.2.4 使用**

- 根据所选层，包含 *xynova/c/...* 或 *xynova/cpp/...* 树中对应的头文件。
- 在使用设备或日志接口前，先初始化通信管理器或日志后端。
- 完整的用法模式与集成技巧参见示例章节。

**4.3 术语****taxel**

触觉传感器阵列中的单个敏感元素。每个 taxel 可测量一个力分量 (1D taxel) 或三个正交力分量 (3D taxel)。参见触觉敏感元感知。

**tactile region**

覆盖传感器阵列某一模块的一组 **tactel**，例如指尖、指骨或手掌。每个区域具有固定的 **tactel** 数量，并由区域 ID 标识。参见[触觉传感](#)。

**wrench**

六分量力/扭矩向量  $[F_x, F_y, F_z, M_x, M_y, M_z]$ 。触觉区域的 **wrench** 报告该区域的合成力和力矩，力的单位为牛顿 (N)，力矩单位为牛顿·米 (Nm)。

**joint**

灵巧手的一个自由度。手部暴露一组固定的关节 (**JOINT\_NUM**)，每个关节由覆盖手指、拇指和腕部的关节 ID 标识。

**control mode**

关节的工作模式，通过 `setJointControlMode()` 配置，并应用于所有关节。当前 SDK 实现 **DISABLE** (禁用控制) 与 **ANGLE** (角度控制) 两种模式；位置、速度、力等其他模式为预留，尚未实现。

**master**

驱动控制循环并与总线连接的从站 (**slave**) 交换数据的通信节点。主站 (**master**) 配置有通信类型和 CAN 接口。

**slave**

总线上的物理设备，响应主站 (**master**)，例如灵巧手。从站 (**slave**) 由从站 ID 标识，并向主站 (**master**) 暴露设备功能。

**device**

一个逻辑上的高级封装器，将一个主站 (**master**) 和一个或多个从站 (**slave**) 绑定为单个可控单元，例如灵巧手设备。

**CAN / CAN-FD**

控制器局域网网络。**CAN-FD** 在传统 **CAN** 的基础上支持更高的数据比特率和更大的载荷。SDK 通过可配置比特率和采样点的 `CommType::Can` 传输支持 **CAN** 和 **CAN-FD**。

**sink**

一个日志组件，用于归并一个或多个 **writer**，并管理日志记录的发送。

**writer**

日志输出目标，例如文件或控制台，具有各自的格式化、刷新和轮转设置。

**logger**

一个命名的日志实例，用于应用程序或 SDK 模块输出日志记录。**logger** 绑定到 **sink** 和一个或多个 **writer**。

**SDK version**

SDK 构建版本信息，包括版本号、git 分支、git 提交和构建时间，可通过 `xynova/c/version.h` 中的函数查询。

**range point**

接近传感器 (DToF) 的一次距离测量。每个 **range point** 报告一个测量值 **status** (有效、无目标、信号弱或无效) 和一个距离 **distance** (单位为米)。参见[接近觉测量](#)。

**range map**

由 `getRange()` 返回的二维距离点集合，按 **row-major** 顺序存储 ( $index = y * width + x$ )。当前 SDK 报告 4×4 的 16 个点的范围图。

**error code**

公共 API 函数返回的标准化 `XynovaErrorCode` 值，例如 `XYNOVA_ERROR_CODE_OK` 或 `XYNOVA_ERROR_CODE_DATA_OUT_OF_DATE`。完整参考请见[错误码](#)。

- **生成的参考文档：** Doxygen XML 与 Sphinx HTML 输出会写入构建树中，便于本地查阅。
- **最佳实践：** 使用前先校验设备有效性，较高频率循环中优先使用异步控制，并在运行前初始化日志。

## 符号

[anonymous]::XYNOVA\_COMMUNICATION\_CAN\_FRAME\_TYPE\_EXTENDED (C++ enumerator), 53  
[anonymous]::XYNOVA\_COMMUNICATION\_CAN\_FRAME\_TYPE\_STANDARD (C++ enumerator), 53  
[anonymous]::XYNOVA\_COMMUNICATION\_CAN\_MASK\_EXTENDED (C++ enumerator), 53  
[anonymous]::XYNOVA\_COMMUNICATION\_CAN\_MASK\_STANDARD (C++ enumerator), 53  
[anonymous]::XYNOVA\_COMMUNICATION\_COMM\_TYPE\_CAN (C++ enumerator), 47  
[anonymous]::XYNOVA\_DATA\_TYPE\_BOOL (C++ enumerator), 83  
[anonymous]::XYNOVA\_DATA\_TYPE\_DOUBLE (C++ enumerator), 83  
[anonymous]::XYNOVA\_DATA\_TYPE\_FLOAT (C++ enumerator), 83  
[anonymous]::XYNOVA\_DATA\_TYPE\_NONE (C++ enumerator), 82  
[anonymous]::XYNOVA\_DATA\_TYPE\_S16 (C++ enumerator), 83  
[anonymous]::XYNOVA\_DATA\_TYPE\_S32 (C++ enumerator), 83  
[anonymous]::XYNOVA\_DATA\_TYPE\_S64 (C++ enumerator), 83  
[anonymous]::XYNOVA\_DATA\_TYPE\_S8 (C++ enumerator), 83  
[anonymous]::XYNOVA\_DATA\_TYPE\_U16 (C++ enumerator), 83  
[anonymous]::XYNOVA\_DATA\_TYPE\_U32 (C++ enumerator), 83  
[anonymous]::XYNOVA\_DATA\_TYPE\_U64 (C++ enumerator), 83  
[anonymous]::XYNOVA\_DATA\_TYPE\_U8 (C++ enumerator), 83  
[anonymous]::XYNOVA\_DEXTEROUS\_HAND\_CONTROL\_MODE\_ANGLE (C++ enumerator), 72  
[anonymous]::XYNOVA\_DEXTEROUS\_HAND\_CONTROL\_MODE\_DISABLE (C++ enumerator), 72  
[anonymous]::XYNOVA\_DEXTEROUS\_HAND\_ERROR\_COMMUNICATION (C++ enumerator), 66  
[anonymous]::XYNOVA\_DEXTEROUS\_HAND\_ERROR\_ENCODER\_ABNORMAL (C++ enumerator), 66  
[anonymous]::XYNOVA\_DEXTEROUS\_HAND\_ERROR\_MOTOR\_ABNORMAL (C++ enumerator), 66  
[anonymous]::XYNOVA\_DEXTEROUS\_HAND\_ERROR\_NONE (C++ enumerator), 66  
[anonymous]::XYNOVA\_DEXTEROUS\_HAND\_ERROR\_OVER\_CURRENT (C++ enumerator), 66  
[anonymous]::XYNOVA\_DEXTEROUS\_HAND\_ERROR\_OVER\_TEMPERATURE (C++ enumerator), 66  
[anonymous]::XYNOVA\_DEXTEROUS\_HAND\_ERROR\_STALL\_PROTECTION (C++ enumerator), 66  
[anonymous]::XYNOVA\_DEXTEROUS\_HAND\_FAULT\_ACTUATOR\_COMMUNICATION (C++ enumerator), 67  
[anonymous]::XYNOVA\_DEXTEROUS\_HAND\_FAULT\_ACTUATOR\_OVER\_CURRENT (C++ enumerator), 69  
[anonymous]::XYNOVA\_DEXTEROUS\_HAND\_FAULT\_ACTUATOR\_OVER\_TEMPERATURE (C++ enumerator), 69  
[anonymous]::XYNOVA\_DEXTEROUS\_HAND\_FAULT\_ACTUATOR\_OVER\_VOLTAGE (C++ enumerator), 69  
[anonymous]::XYNOVA\_DEXTEROUS\_HAND\_FAULT\_ACTUATOR\_UNDER\_VOLTAGE (C++ enumerator), 69  
[anonymous]::XYNOVA\_DEXTEROUS\_HAND\_FAULT\_ANGLE\_SENSOR\_COMMUNICATION (C++ enumerator), 67  
[anonymous]::XYNOVA\_DEXTEROUS\_HAND\_FAULT\_CANFD1\_STATUS (C++ enumerator), 68  
[anonymous]::XYNOVA\_DEXTEROUS\_HAND\_FAULT\_CANFD2\_STATUS (C++ enumerator), 68  
[anonymous]::XYNOVA\_DEXTEROUS\_HAND\_FAULT\_CONTROL\_LOOP\_TIMEOUT (C++ enumerator), 69  
[anonymous]::XYNOVA\_DEXTEROUS\_HAND\_FAULT\_MAINBOARD\_24V\_OVER\_VOLTAGE (C++ enumerator), 68  
[anonymous]::XYNOVA\_DEXTEROUS\_HAND\_FAULT\_MAINBOARD\_24V\_UNDER\_VOLTAGE (C++ enumerator), 68  
[anonymous]::XYNOVA\_DEXTEROUS\_HAND\_FAULT\_MAINBOARD\_48V\_OVER\_VOLTAGE (C++ enumerator), 68  
[anonymous]::XYNOVA\_DEXTEROUS\_HAND\_FAULT\_MAINBOARD\_48V\_UNDER\_VOLTAGE (C++ enumerator), 68  
[anonymous]::XYNOVA\_DEXTEROUS\_HAND\_FAULT\_MAINBOARD\_OVER\_CURRENT1 (C++ enumerator), 68  
[anonymous]::XYNOVA\_DEXTEROUS\_HAND\_FAULT\_MAINBOARD\_OVER\_CURRENT2 (C++ enumerator), 68  
[anonymous]::XYNOVA\_DEXTEROUS\_HAND\_FAULT\_MAINBOARD\_OVER\_CURRENT3 (C++ enumerator), 68  
[anonymous]::XYNOVA\_DEXTEROUS\_HAND\_FAULT\_MAINBOARD\_OVER\_TEMPERATURE (C++ enumerator), 68

```

[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_MAINBOARD_UNDER_TEMPERATURE (C++ enumerator), 68
[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_NONE (C++ enumerator), 67
[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_PARAMETER_STORAGE (C++ enumerator), 68
[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_PDM_FRAME_LOSS (C++ enumerator), 69
[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_RANGE_SENSOR_COMMUNICATION (C++ enumerator), 68
[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_SUBBOARD_COMMUNICATION (C++ enumerator), 67
[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_SUBBOARD_OVER_TEMPERATURE (C++ enumerator), 68
[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_SUBBOARD_UNDER_TEMPERATURE (C++ enumerator), 68
[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_TACTILE_SENSOR_COMMUNICATION (C++ enumerator), 67
[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_FF1 (C++ enumerator), 69
[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_FF2 (C++ enumerator), 69
[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_FF3 (C++ enumerator), 69
[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_FF4 (C++ enumerator), 70
[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_LF1 (C++ enumerator), 70
[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_LF2 (C++ enumerator), 70
[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_LF3 (C++ enumerator), 70
[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_LF4 (C++ enumerator), 70
[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_LF5 (C++ enumerator), 70
[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_MF1 (C++ enumerator), 70
[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_MF2 (C++ enumerator), 70
[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_MF3 (C++ enumerator), 70
[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_MF4 (C++ enumerator), 70
[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_RF1 (C++ enumerator), 70
[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_RF2 (C++ enumerator), 70
[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_RF3 (C++ enumerator), 70
[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_RF4 (C++ enumerator), 70
[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_TH1 (C++ enumerator), 70
[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_TH2 (C++ enumerator), 71
[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_TH3 (C++ enumerator), 71
[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_TH4 (C++ enumerator), 71
[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_WR1 (C++ enumerator), 71
[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_WR2 (C++ enumerator), 71
[anonymous]::XYNOVA_DEXTEROUS_HAND_RANGE_POINT_STATUS_INVALID (C++ enumerator), 73
[anonymous]::XYNOVA_DEXTEROUS_HAND_RANGE_POINT_STATUS_NO_TARGET (C++ enumerator), 73
[anonymous]::XYNOVA_DEXTEROUS_HAND_RANGE_POINT_STATUS_VALID (C++ enumerator), 73
[anonymous]::XYNOVA_DEXTEROUS_HAND_RANGE_POINT_STATUS_WEAK_SIGNAL (C++ enumerator), 73
[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_FT1 (C++ enumerator), 71
[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_FT2 (C++ enumerator), 71
[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_FT3 (C++ enumerator), 71
[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_LT1 (C++ enumerator), 72
[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_LT2 (C++ enumerator), 72
[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_LT3 (C++ enumerator), 72
[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_MT1 (C++ enumerator), 71
[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_MT2 (C++ enumerator), 71
[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_MT3 (C++ enumerator), 71
[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_PT1 (C++ enumerator), 72
[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_RT1 (C++ enumerator), 72
[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_RT2 (C++ enumerator), 72
[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_RT3 (C++ enumerator), 72
[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_TT1 (C++ enumerator), 71
[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_TT2 (C++ enumerator), 71
[anonymous]::XYNOVA_DEXTEROUS_HAND_TYPE_LEFT (C++ enumerator), 69
[anonymous]::XYNOVA_DEXTEROUS_HAND_TYPE_RIGHT (C++ enumerator), 69
[anonymous]::XYNOVA_DEXTEROUS_HAND_TYPE_UNKNOWN (C++ enumerator), 69
[anonymous]::XYNOVA_DEXTEROUS_HAND_WARNING_ANGLE_SENSOR_COMMUNICATION (C++ enumerator), 66
[anonymous]::XYNOVA_DEXTEROUS_HAND_WARNING_CANFD1_STATUS (C++ enumerator), 67
[anonymous]::XYNOVA_DEXTEROUS_HAND_WARNING_CANFD2_STATUS (C++ enumerator), 67
[anonymous]::XYNOVA_DEXTEROUS_HAND_WARNING_CONTROL_LOOP_TIMEOUT (C++ enumerator), 67
[anonymous]::XYNOVA_DEXTEROUS_HAND_WARNING_FIRMWARE_UPGRADE (C++ enumerator), 67
[anonymous]::XYNOVA_DEXTEROUS_HAND_WARNING_MAINBOARD_OVER_TEMPERATURE (C++ enumerator), 66
[anonymous]::XYNOVA_DEXTEROUS_HAND_WARNING_MAINBOARD_UNDER_TEMPERATURE (C++ enumerator), 67
[anonymous]::XYNOVA_DEXTEROUS_HAND_WARNING_NONE (C++ enumerator), 66
[anonymous]::XYNOVA_DEXTEROUS_HAND_WARNING_PDM_FRAME_LOSS (C++ enumerator), 67
[anonymous]::XYNOVA_DEXTEROUS_HAND_WARNING_RANGE_SENSOR_COMMUNICATION (C++ enumerator), 66
[anonymous]::XYNOVA_DEXTEROUS_HAND_WARNING_SUBBOARD_OVER_TEMPERATURE (C++ enumerator), 67
[anonymous]::XYNOVA_DEXTEROUS_HAND_WARNING_SUBBOARD_UNDER_TEMPERATURE (C++ enumerator), 67
[anonymous]::XYNOVA_DEXTEROUS_HAND_WARNING_TACTILE_SENSOR_COMMUNICATION (C++ enumerator), 66
[anonymous]::XYNOVA_DEXTEROUS_HAND_WRENCH_DIMENSION_1D (C++ enumerator), 72

```

```
[anonymous]::XYNOVA_DEXTEROUS_HAND_WRENCH_DIMENSION_3D (C++ enumerator), 72
[anonymous]::XYNOVA_DEXTEROUS_HAND_WRENCH_DIMENSION_6D (C++ enumerator), 72
[anonymous]::XYNOVA_ERROR_CODE_LIST (C++ enumerator), 80
[anonymous]::XYNOVA_FALSE (C++ enumerator), 78
[anonymous]::XYNOVA_LOG_LEVEL_DEBUG (C++ enumerator), 93
[anonymous]::XYNOVA_LOG_LEVEL_ERROR (C++ enumerator), 93
[anonymous]::XYNOVA_LOG_LEVEL_FATAL (C++ enumerator), 93
[anonymous]::XYNOVA_LOG_LEVEL_INFO (C++ enumerator), 93
[anonymous]::XYNOVA_LOG_LEVEL_TRACE (C++ enumerator), 93
[anonymous]::XYNOVA_LOG_LEVEL_WARN (C++ enumerator), 93
[anonymous]::XYNOVA_TRUE (C++ enumerator), 78
[anonymous] (C++ enum), 47, 53, 66, 67, 69, 71–73, 78, 80, 82, 93
```

## C

CAN / CAN-FD, 133  
control mode, 133

## D

device, 133

## E

error code, 133

## J

joint, 133

## L

logger, 133

## M

master, 133

## R

range map, 133  
range point, 133

## S

SDK version, 133  
sink, 133  
slave, 133

## T

tactile region, 133  
taxel, 132

## W

wrench, 133  
writer, 133

## X

```
xynova::communication::can::CanConfig::bitrate (C++ member), 108
xynova::communication::can::CanConfig::dbitrte (C++ member), 108
xynova::communication::can::CanConfig::dsample_point (C++ member), 109
xynova::communication::can::CanConfig::iface (C++ member), 108
xynova::communication::can::CanConfig::manager_can (C++ member), 108
xynova::communication::can::CanConfig::sample_point (C++ member), 108
xynova::communication::can::CanConfig (C++ struct), 108
xynova::communication::can::CanMask::Extended (C++ enumerator), 108
xynova::communication::can::CanMask::Standard (C++ enumerator), 108
xynova::communication::can::CanMask (C++ enum), 108
xynova::communication::can::FrameType::Extended (C++ enumerator), 108
```

```

xynova::communication::can::FrameType::Standard (C++ enumerator), 108
xynova::communication::can::FrameType (C++ enum), 108
xynova::communication::can::RecvFilter::id (C++ member), 109
xynova::communication::can::RecvFilter::mask (C++ member), 109
xynova::communication::can::RecvFilter::type (C++ member), 109
xynova::communication::can::RecvFilter (C++ struct), 109
xynova::communication::can (C++ type), 107
xynova::communication::CommConfig::can (C++ member), 104
xynova::communication::CommConfig::logger_name (C++ member), 105
xynova::communication::CommConfig::sub_type (C++ member), 104
xynova::communication::CommConfig::type (C++ member), 104
xynova::communication::CommConfig (C++ struct), 104
xynova::communication::CommType::Can (C++ enumerator), 104
xynova::communication::CommType (C++ enum), 104
xynova::communication::DeviceConfig::commands (C++ member), 107
xynova::communication::DeviceConfig::master_name (C++ member), 107
xynova::communication::DeviceConfig::name (C++ member), 106
xynova::communication::DeviceConfig::slaves (C++ member), 107
xynova::communication::DeviceConfig::type (C++ member), 106
xynova::communication::DeviceConfig (C++ struct), 106
xynova::communication::DeviceHandle::~DeviceHandle (C++ function), 102
xynova::communication::DeviceHandle::DeviceHandle (C++ function), 102
xynova::communication::DeviceHandle::get (C++ function), 102
xynova::communication::DeviceHandle::i_handle (C++ member), 102
xynova::communication::DeviceHandle::operator bool (C++ function), 102
xynova::communication::DeviceHandle::operator= (C++ function), 102
xynova::communication::DeviceHandle (C++ class), 102
xynova::communication::Manager::~Manager (C++ function), 104
xynova::communication::Manager::getDevice (C++ function), 103
xynova::communication::Manager::init (C++ function), 103
xynova::communication::Manager::instance (C++ function), 104
xynova::communication::Manager::Manager (C++ function), 103, 104
xynova::communication::Manager::operator= (C++ function), 103
xynova::communication::Manager::run (C++ function), 103
xynova::communication::Manager::stop (C++ function), 103
xynova::communication::Manager::upgradeFirmware (C++ function), 103
xynova::communication::ManagerConfig::devices (C++ member), 107
xynova::communication::ManagerConfig::masters (C++ member), 107
xynova::communication::ManagerConfig (C++ struct), 107
xynova::communication::Manager (C++ class), 102
xynova::communication::MasterConfig::comm (C++ member), 105
xynova::communication::MasterConfig::cpu_index (C++ member), 105
xynova::communication::MasterConfig::name (C++ member), 105
xynova::communication::MasterConfig::period_us (C++ member), 105
xynova::communication::MasterConfig (C++ struct), 105
xynova::communication::ResponseTimeSetting::execute_max_retry (C++ member), 105
xynova::communication::ResponseTimeSetting::execute_poll_interval_us (C++ member), 105
xynova::communication::ResponseTimeSetting::execute_timeout_us (C++ member), 105
xynova::communication::ResponseTimeSetting::user_max_timeout_us (C++ member), 105
xynova::communication::ResponseTimeSetting::user_poll_interval_us (C++ member), 105
xynova::communication::ResponseTimeSetting (C++ struct), 105
xynova::communication::SchedulingCommand::command (C++ member), 106
xynova::communication::SchedulingCommand::response_time_setting (C++ member), 106
xynova::communication::SchedulingCommand (C++ struct), 106
xynova::communication::SlaveConfig::fast_time_setting (C++ member), 106
xynova::communication::SlaveConfig::general_time_setting (C++ member), 106
xynova::communication::SlaveConfig::id (C++ member), 106
xynova::communication::SlaveConfig::logger_name (C++ member), 106
xynova::communication::SlaveConfig::type (C++ member), 106
xynova::communication::SlaveConfig (C++ struct), 106
xynova::communication (C++ type), 102, 104, 107
xynova::hand::dexterous_hand::AngleControl (C++ type), 116
xynova::hand::dexterous_hand::AngleStates (C++ type), 116
xynova::hand::dexterous_hand::ControlModeCommand (C++ type), 116
xynova::hand::dexterous_hand::ControlMode (C++ type), 116
xynova::hand::dexterous_hand::Device::c_handle (C++ function), 114
xynova::hand::dexterous_hand::Device::DeviceHandle (C++ type), 111
xynova::hand::dexterous_hand::Device::Device (C++ function), 111

```

```

xynova::hand::dexterous_hand::Device::enableRange (C++ function), 113
xynova::hand::dexterous_hand::Device::enableTactileTaxel (C++ function), 113
xynova::hand::dexterous_hand::Device::enableTactileWrench (C++ function), 112
xynova::hand::dexterous_hand::Device::getErrorState (C++ function), 112
xynova::hand::dexterous_hand::Device::getHandType (C++ function), 112
xynova::hand::dexterous_hand::Device::getHardwareVersion (C++ function), 112
xynova::hand::dexterous_hand::Device::getJointAngle (C++ function), 113
xynova::hand::dexterous_hand::Device::getProductModel (C++ function), 112
xynova::hand::dexterous_hand::Device::getProductSN (C++ function), 112
xynova::hand::dexterous_hand::Device::getRange (C++ function), 114
xynova::hand::dexterous_hand::Device::getSoftwareVersion (C++ function), 112
xynova::hand::dexterous_hand::Device::getTactileTaxel (C++ function), 114
xynova::hand::dexterous_hand::Device::getTactileWrench (C++ function), 113
xynova::hand::dexterous_hand::Device::i_dev_handle (C++ member), 114
xynova::hand::dexterous_hand::Device::operator= (C++ function), 111
xynova::hand::dexterous_hand::Device::run (C++ function), 111
xynova::hand::dexterous_hand::Device::setJointAngleAsync (C++ function), 113
xynova::hand::dexterous_hand::Device::setJointAngle (C++ function), 113
xynova::hand::dexterous_hand::Device::setJointControlMode (C++ function), 112
xynova::hand::dexterous_hand::Device::stop (C++ function), 111
xynova::hand::dexterous_hand::Device::valid (C++ function), 111
xynova::hand::dexterous_hand::Device (C++ class), 111
xynova::hand::dexterous_hand::ErrorCode (C++ type), 115
xynova::hand::dexterous_hand::ErrorState::fault_flags (C++ member), 119
xynova::hand::dexterous_hand::ErrorState::hasFault (C++ function), 118
xynova::hand::dexterous_hand::ErrorState::hasWarning (C++ function), 118
xynova::hand::dexterous_hand::ErrorState::joint_error_codes (C++ member), 119
xynova::hand::dexterous_hand::ErrorState::joint (C++ function), 118
xynova::hand::dexterous_hand::ErrorState::timestamp_us (C++ member), 118
xynova::hand::dexterous_hand::ErrorState::warning_flags (C++ member), 118
xynova::hand::dexterous_hand::ErrorState (C++ struct), 118
xynova::hand::dexterous_hand::FaultFlags (C++ type), 115
xynova::hand::dexterous_hand::HandType (C++ type), 115
xynova::hand::dexterous_hand::JOINT_NUM (C++ member), 117
xynova::hand::dexterous_hand::JointError::Communication (C++ enumerator), 117
xynova::hand::dexterous_hand::JointError::EncoderAbnormal (C++ enumerator), 117
xynova::hand::dexterous_hand::JointError::MotorAbnormal (C++ enumerator), 117
xynova::hand::dexterous_hand::JointError::None (C++ enumerator), 117
xynova::hand::dexterous_hand::JointError::OverCurrent (C++ enumerator), 117
xynova::hand::dexterous_hand::JointError::OverTemperature (C++ enumerator), 117
xynova::hand::dexterous_hand::JointError::StallProtection (C++ enumerator), 117
xynova::hand::dexterous_hand::JointErrorCode::fromC (C++ function), 117
xynova::hand::dexterous_hand::JointErrorCode::has (C++ function), 118
xynova::hand::dexterous_hand::JointErrorCode::i_code (C++ member), 118
xynova::hand::dexterous_hand::JointErrorCode::JointErrorCode (C++ function), 117
xynova::hand::dexterous_hand::JointErrorCode::ok (C++ function), 118
xynova::hand::dexterous_hand::JointErrorCode::toC (C++ function), 117
xynova::hand::dexterous_hand::JointErrorCode::value (C++ function), 118
xynova::hand::dexterous_hand::JointErrorCode (C++ class), 117
xynova::hand::dexterous_hand::JointErrorState::error_code (C++ member), 118
xynova::hand::dexterous_hand::JointErrorState::id (C++ member), 118
xynova::hand::dexterous_hand::JointErrorState (C++ struct), 118
xynova::hand::dexterous_hand::JointError (C++ enum), 117
xynova::hand::dexterous_hand::JointID (C++ type), 115
xynova::hand::dexterous_hand::JointValueArray (C++ type), 115
xynova::hand::dexterous_hand::RANGE_POINT_NUM (C++ member), 117
xynova::hand::dexterous_hand::RangePointStatus (C++ type), 116
xynova::hand::dexterous_hand::RangePoint (C++ type), 116
xynova::hand::dexterous_hand::RangeStates::height (C++ member), 120
xynova::hand::dexterous_hand::RangeStates::points (C++ member), 120
xynova::hand::dexterous_hand::RangeStates::RangeStates (C++ function), 120
xynova::hand::dexterous_hand::RangeStates::width (C++ member), 120
xynova::hand::dexterous_hand::RangeStates (C++ struct), 120
xynova::hand::dexterous_hand::TACTILE_NUM (C++ member), 117
xynova::hand::dexterous_hand::TACTILE_TOTAL_TAXEL_NUM (C++ member), 117
xynova::hand::dexterous_hand::TactileID (C++ type), 115
xynova::hand::dexterous_hand::TactileTaxel::dimension (C++ member), 119
xynova::hand::dexterous_hand::TactileTaxel::region (C++ member), 119

```

```

xynova::hand::dexterous_hand::TactileTaxel::reserved (C++ member), 119
xynova::hand::dexterous_hand::TactileTaxel::taxel_id (C++ member), 119
xynova::hand::dexterous_hand::TactileTaxel::timestamp_us (C++ member), 119
xynova::hand::dexterous_hand::TactileTaxel::wrench (C++ member), 120
xynova::hand::dexterous_hand::TactileTaxelStates (C++ type), 116
xynova::hand::dexterous_hand::TactileTaxel (C++ struct), 119
xynova::hand::dexterous_hand::TactileWrench::dimension (C++ member), 119
xynova::hand::dexterous_hand::TactileWrench::region (C++ member), 119
xynova::hand::dexterous_hand::TactileWrench::timestamp_us (C++ member), 119
xynova::hand::dexterous_hand::TactileWrench::wrench (C++ member), 119
xynova::hand::dexterous_hand::TactileWrenchStates (C++ type), 116
xynova::hand::dexterous_hand::TactileWrench (C++ struct), 119
xynova::hand::dexterous_hand::WarningFlags (C++ type), 115
xynova::hand::dexterous_hand::WrenchDimension (C++ type), 116
xynova::hand::dexterous_hand (C++ type), 111, 115
xynova::hand (C++ type), 111, 115
xynova::infra::log::ConsoleWriterConfiguration::base_conf (C++ member), 126
xynova::infra::log::ConsoleWriterConfiguration::use_stderr (C++ member), 126
xynova::infra::log::ConsoleWriterConfiguration (C++ struct), 126
xynova::infra::log::FileWriterConfiguration::base_conf (C++ member), 126
xynova::infra::log::FileWriterConfiguration::file_path (C++ member), 126
xynova::infra::log::FileWriterConfiguration::max_file_size (C++ member), 126
xynova::infra::log::FileWriterConfiguration::max_roll_files (C++ member), 126
xynova::infra::log::FileWriterConfiguration (C++ struct), 126
xynova::infra::log::FormaterConfiguration::reserved (C++ member), 126
xynova::infra::log::FormaterConfiguration (C++ struct), 126
xynova::infra::log::LoggerBackendConfiguration::block_size (C++ member), 127
xynova::infra::log::LoggerBackendConfiguration::flush_interval_us (C++ member), 127
xynova::infra::log::LoggerBackendConfiguration::max_continuous_repeated_log (C++ member), 127
xynova::infra::log::LoggerBackendConfiguration::message_size (C++ member), 127
xynova::infra::log::LoggerBackendConfiguration::pool_size (C++ member), 127
xynova::infra::log::LoggerBackendConfiguration::queue_size_multiple (C++ member), 127
xynova::infra::log::LoggerBackendConfiguration (C++ struct), 126
xynova::infra::log::LoggerConfiguration::lb_config (C++ member), 127
xynova::infra::log::LoggerConfiguration::lb_type (C++ member), 127
xynova::infra::log::LoggerConfiguration::level (C++ member), 127
xynova::infra::log::LoggerConfiguration::sink_name (C++ member), 127
xynova::infra::log::LoggerConfiguration::writer_name_list (C++ member), 127
xynova::infra::log::LoggerConfiguration (C++ struct), 127
xynova::infra::log::LogManager::~LogManager (C++ function), 124
xynova::infra::log::LogManager::createConsoleWriter (C++ function), 122
xynova::infra::log::LogManager::createFileWriter (C++ function), 122
xynova::infra::log::LogManager::createFormater (C++ function), 123
xynova::infra::log::LogManager::createLogger (C++ function), 123
xynova::infra::log::LogManager::createSinkTP (C++ function), 122
xynova::infra::log::LogManager::hasFormater (C++ function), 123
xynova::infra::log::LogManager::hasLogger (C++ function), 123
xynova::infra::log::LogManager::hasSink (C++ function), 123
xynova::infra::log::LogManager::hasWriter (C++ function), 123
xynova::infra::log::LogManager::initDefaultObjects (C++ function), 122
xynova::infra::log::LogManager::instance (C++ function), 124
xynova::infra::log::LogManager::isInitialized (C++ function), 122
xynova::infra::log::LogManager::LogManager (C++ function), 124
xynova::infra::log::LogManager::operator= (C++ function), 124
xynova::infra::log::LogManager::setLoggerLevel (C++ function), 124
xynova::infra::log::LogManager (C++ class), 122
xynova::infra::log::SinkConfiguration::block_obtain_timeout_us (C++ member), 124
xynova::infra::log::SinkConfiguration::max_continuous_push_retry_time_ms (C++ member), 125
xynova::infra::log::SinkConfiguration::max_looping_task_idle_interval_us (C++ member), 125
xynova::infra::log::SinkConfiguration::min_looping_task_idle_interval_us (C++ member), 125
xynova::infra::log::SinkConfiguration::writing_task_push_timeout_us (C++ member), 124
xynova::infra::log::SinkConfiguration::writing_task_wait_timeout_us (C++ member), 125
xynova::infra::log::SinkConfiguration (C++ struct), 124
xynova::infra::log::SinkTPConfiguration::base_conf (C++ member), 125
xynova::infra::log::SinkTPConfiguration::max_queue_size (C++ member), 125
xynova::infra::log::SinkTPConfiguration::num_threads (C++ member), 125
xynova::infra::log::SinkTPConfiguration (C++ struct), 125
xynova::infra::log::WriterConfiguration::auto_flush (C++ member), 125

```

```

xynova::infra::log::WriterConfiguration::flush_interval_ms (C++ member), 125
xynova::infra::log::WriterConfiguration::formater (C++ member), 125
xynova::infra::log::WriterConfiguration (C++ struct), 125
xynova::infra::log (C++ type), 122, 124
xynova::infra::tools::assignValue (C++ function), 129
xynova::infra::tools::DataTypeTraits::value (C++ member), 129
xynova::infra::tools::DataTypeTraits (C++ struct), 129
xynova::infra::tools::IdValue::fromC (C++ function), 132
xynova::infra::tools::IdValue::i_id (C++ member), 132
xynova::infra::tools::IdValue::i_value (C++ member), 132
xynova::infra::tools::IdValue::IdValue (C++ function), 131
xynova::infra::tools::IdValue::id (C++ function), 131
xynova::infra::tools::IdValue::setId (C++ function), 131
xynova::infra::tools::IdValue::setValue (C++ function), 131
xynova::infra::tools::IdValue::Storage (C++ type), 131
xynova::infra::tools::IdValue::toC (C++ function), 132
xynova::infra::tools::IdValue::ValueType (C++ type), 131
xynova::infra::tools::IdValue::value (C++ function), 131
xynova::infra::tools::IdValue (C++ class), 131
xynova::infra::tools::readValue (C++ function), 129
xynova::infra::tools::Value::fromC (C++ function), 130
xynova::infra::tools::Value::i_value (C++ member), 131
xynova::infra::tools::Value::setValue (C++ function), 130
xynova::infra::tools::Value::Storage (C++ type), 130
xynova::infra::tools::Value::toC (C++ function), 130
xynova::infra::tools::Value::Value (C++ function), 130
xynova::infra::tools::Value::value (C++ function), 130
xynova::infra::tools::Value (C++ class), 129
xynova::infra::tools (C++ type), 129
xynova::infra (C++ type), 122, 124, 129
XYNOVA_API (C macro), 78
XYNOVA_CONSTEXPR (C macro), 78
XYNOVA_DEFINE_DATA_TYPE_TRAIT (C macro), 128
XYNOVA_DEXTEROUS_HAND_JOINT_NUM (C macro), 63
XYNOVA_DEXTEROUS_HAND_RANGE_POINT_NUM (C macro), 63
XYNOVA_DEXTEROUS_HAND_TACTILE_NUM (C macro), 63
XYNOVA_DEXTEROUS_HAND_TACTILE_TOTAL_TAXEL_NUM (C macro), 63
XYNOVA_ERROR_CODE_LIST (C macro), 79
XYNOVA_EXPORT_TEMPLATE (C macro), 79
XYNOVA_IMPL_API (C macro), 78
XYNOVA_INLINE_API (C macro), 78
XYNOVA_LOCAL (C macro), 78
XYNOVA_NOEXCEPT (C macro), 78
XYNOVA_SDK_BUILD_TIME (C macro), 88
XYNOVA_SDK_GIT_BRANCH (C macro), 87
XYNOVA_SDK_GIT_COMMIT (C macro), 88
XYNOVA_SDK_LOCAL_VERSION (C macro), 87
XYNOVA_SDK_VERSION (C macro), 87
XynovaBool (C++ type), 77
XynovaCommunicationCan_registerDriver (C++ function), 51
XynovaCommunicationCanConfig::bitrate (C++ member), 53
XynovaCommunicationCanConfig::dbitrate (C++ member), 54
XynovaCommunicationCanConfig::dsample_point (C++ member), 54
XynovaCommunicationCanConfig::iface (C++ member), 53
XynovaCommunicationCanConfig::manager_can (C++ member), 53
XynovaCommunicationCanConfig::sample_point (C++ member), 54
XynovaCommunicationCanConfig (C++ struct), 53
XynovaCommunicationCanConfig (C++ type), 52
XynovaCommunicationCanFrameType (C++ type), 52
XynovaCommunicationCanMask (C++ type), 52
XynovaCommunicationCanRecvFilter::id (C++ member), 54
XynovaCommunicationCanRecvFilter::mask (C++ member), 54
XynovaCommunicationCanRecvFilter::type (C++ member), 54
XynovaCommunicationCanRecvFilter (C++ struct), 54
XynovaCommunicationCanRecvFilter (C++ type), 52
XynovaCommunicationCanVTable::close (C++ member), 55
XynovaCommunicationCanVTable::create (C++ member), 55
XynovaCommunicationCanVTable::destroy (C++ member), 55

```

XynovaCommunicationCanVTable::isOpen (C++ member), 55  
 XynovaCommunicationCanVTable::open (C++ member), 55  
 XynovaCommunicationCanVTable::recv (C++ member), 56  
 XynovaCommunicationCanVTable::send (C++ member), 56  
 XynovaCommunicationCanVTable::setRecvFilters (C++ member), 55  
 XynovaCommunicationCanVTable (C++ struct), 54  
 XynovaCommunicationCanVTable (C++ type), 52  
 XynovaCommunicationCommConfig::can (C++ member), 48  
 XynovaCommunicationCommConfig::config (C++ member), 48  
 XynovaCommunicationCommConfig::logger\_name (C++ member), 48  
 XynovaCommunicationCommConfig::sub\_type (C++ member), 48  
 XynovaCommunicationCommConfig::type (C++ member), 48  
 XynovaCommunicationCommConfig (C++ struct), 47  
 XynovaCommunicationCommConfig (C++ type), 47  
 XynovaCommunicationCommType (C++ type), 47  
 XynovaCommunicationDeviceConfig::command\_count (C++ member), 50  
 XynovaCommunicationDeviceConfig::commands (C++ member), 50  
 XynovaCommunicationDeviceConfig::master\_name (C++ member), 50  
 XynovaCommunicationDeviceConfig::name (C++ member), 50  
 XynovaCommunicationDeviceConfig::slave\_count (C++ member), 50  
 XynovaCommunicationDeviceConfig::slaves (C++ member), 50  
 XynovaCommunicationDeviceConfig::type (C++ member), 50  
 XynovaCommunicationDeviceConfig (C++ struct), 49  
 XynovaCommunicationDeviceConfig (C++ type), 47  
 XynovaCommunicationManager\_freeDevice (C++ function), 46  
 XynovaCommunicationManager\_getDevice (C++ function), 45  
 XynovaCommunicationManager\_init (C++ function), 45  
 XynovaCommunicationManager\_run (C++ function), 45  
 XynovaCommunicationManager\_stop (C++ function), 45  
 XynovaCommunicationManager\_upgradeFirmware (C++ function), 46  
 XynovaCommunicationManagerConfig::device\_count (C++ member), 50  
 XynovaCommunicationManagerConfig::devices (C++ member), 50  
 XynovaCommunicationManagerConfig::master\_count (C++ member), 50  
 XynovaCommunicationManagerConfig::masters (C++ member), 50  
 XynovaCommunicationManagerConfig (C++ struct), 50  
 XynovaCommunicationManagerConfig (C++ type), 47  
 XynovaCommunicationMasterConfig::comm (C++ member), 48  
 XynovaCommunicationMasterConfig::cpu\_index (C++ member), 48  
 XynovaCommunicationMasterConfig::name (C++ member), 48  
 XynovaCommunicationMasterConfig::period\_us (C++ member), 48  
 XynovaCommunicationMasterConfig (C++ struct), 48  
 XynovaCommunicationMasterConfig (C++ type), 47  
 XynovaCommunicationResponseTimeSetting::execute\_max\_retry (C++ member), 49  
 XynovaCommunicationResponseTimeSetting::execute\_poll\_interval\_us (C++ member), 49  
 XynovaCommunicationResponseTimeSetting::execute\_timeout\_us (C++ member), 48  
 XynovaCommunicationResponseTimeSetting::user\_max\_timeout\_us (C++ member), 48  
 XynovaCommunicationResponseTimeSetting::user\_poll\_interval\_us (C++ member), 48  
 XynovaCommunicationResponseTimeSetting (C++ struct), 48  
 XynovaCommunicationResponseTimeSetting (C++ type), 47  
 XynovaCommunicationSchedulingCommand::command (C++ member), 49  
 XynovaCommunicationSchedulingCommand::response\_time\_setting (C++ member), 49  
 XynovaCommunicationSchedulingCommand (C++ struct), 49  
 XynovaCommunicationSchedulingCommand (C++ type), 47  
 XynovaCommunicationSlaveConfig::fast\_time\_setting (C++ member), 49  
 XynovaCommunicationSlaveConfig::general\_time\_setting (C++ member), 49  
 XynovaCommunicationSlaveConfig::id (C++ member), 49  
 XynovaCommunicationSlaveConfig::logger\_name (C++ member), 49  
 XynovaCommunicationSlaveConfig::type (C++ member), 49  
 XynovaCommunicationSlaveConfig (C++ struct), 49  
 XynovaCommunicationSlaveConfig (C++ type), 47  
 XynovaDataType (C++ type), 81  
 XynovaDeviceHandle (C++ type), 45  
 XynovaDexterousHand\_enableRange (C++ function), 60  
 XynovaDexterousHand\_enableTactileTaxel (C++ function), 60  
 XynovaDexterousHand\_enableTactileWrench (C++ function), 60  
 XynovaDexterousHand\_getErrorState (C++ function), 59  
 XynovaDexterousHand\_getHandType (C++ function), 59  
 XynovaDexterousHand\_getHardwareVersion (C++ function), 58

XynovaDexterousHand\_getJointAngle (C++ function), 61  
 XynovaDexterousHand\_getProductModel (C++ function), 58  
 XynovaDexterousHand\_getProductSN (C++ function), 58  
 XynovaDexterousHand\_getRange (C++ function), 62  
 XynovaDexterousHand\_getSoftwareVersion (C++ function), 59  
 XynovaDexterousHand\_getTactileTaxel (C++ function), 62  
 XynovaDexterousHand\_getTactileWrench (C++ function), 61  
 XynovaDexterousHand\_run (C++ function), 57  
 XynovaDexterousHand\_setJointAngle\_async (C++ function), 61  
 XynovaDexterousHand\_setJointAngle (C++ function), 61  
 XynovaDexterousHand\_setJointControlAllMode (C++ function), 59  
 XynovaDexterousHand\_stop (C++ function), 58  
 XynovaDexterousHandAngleControl::values (C++ member), 74  
 XynovaDexterousHandAngleControl (C++ struct), 74  
 XynovaDexterousHandAngleControl (C++ type), 65  
 XynovaDexterousHandAngleStates::values (C++ member), 74  
 XynovaDexterousHandAngleStates (C++ struct), 74  
 XynovaDexterousHandAngleStates (C++ type), 65  
 XynovaDexterousHandControlModeCommand::values (C++ member), 74  
 XynovaDexterousHandControlModeCommand (C++ struct), 73  
 XynovaDexterousHandControlModeCommand (C++ type), 64  
 XynovaDexterousHandControlMode (C++ type), 64  
 XynovaDexterousHandErrorCode (C++ type), 63  
 XynovaDexterousHandErrorState::fault\_flags (C++ member), 73  
 XynovaDexterousHandErrorState::joint\_error\_code (C++ member), 73  
 XynovaDexterousHandErrorState::timestamp\_us (C++ member), 73  
 XynovaDexterousHandErrorState::warning\_flags (C++ member), 73  
 XynovaDexterousHandErrorState (C++ struct), 73  
 XynovaDexterousHandErrorState (C++ type), 64  
 XynovaDexterousHandFaultFlags (C++ type), 64  
 XynovaDexterousHandJointID (C++ type), 64  
 XynovaDexterousHandRangePoint::distance (C++ member), 76  
 XynovaDexterousHandRangePoint::id (C++ member), 76  
 XynovaDexterousHandRangePoint::reserved (C++ member), 76  
 XynovaDexterousHandRangePoint::status (C++ member), 76  
 XynovaDexterousHandRangePoint::timestamp\_us (C++ member), 76  
 XynovaDexterousHandRangePointStatus (C++ type), 64  
 XynovaDexterousHandRangePoint (C++ struct), 75  
 XynovaDexterousHandRangePoint (C++ type), 65  
 XynovaDexterousHandRangeStates::count (C++ member), 76  
 XynovaDexterousHandRangeStates::height (C++ member), 76  
 XynovaDexterousHandRangeStates::points (C++ member), 76  
 XynovaDexterousHandRangeStates::width (C++ member), 76  
 XynovaDexterousHandRangeStates (C++ struct), 76  
 XynovaDexterousHandRangeStates (C++ type), 65  
 XynovaDexterousHandTactileID (C++ type), 64  
 XynovaDexterousHandTactileTaxel::dimension (C++ member), 75  
 XynovaDexterousHandTactileTaxel::region (C++ member), 75  
 XynovaDexterousHandTactileTaxel::reserved (C++ member), 75  
 XynovaDexterousHandTactileTaxel::taxel\_id (C++ member), 75  
 XynovaDexterousHandTactileTaxel::timestamp\_us (C++ member), 75  
 XynovaDexterousHandTactileTaxel::wrench (C++ member), 75  
 XynovaDexterousHandTactileTaxelStates::count (C++ member), 75  
 XynovaDexterousHandTactileTaxelStates::taxels (C++ member), 75  
 XynovaDexterousHandTactileTaxelStates (C++ struct), 75  
 XynovaDexterousHandTactileTaxelStates (C++ type), 65  
 XynovaDexterousHandTactileTaxel (C++ struct), 75  
 XynovaDexterousHandTactileTaxel (C++ type), 65  
 XynovaDexterousHandTactileWrench::dimension (C++ member), 74  
 XynovaDexterousHandTactileWrench::region (C++ member), 74  
 XynovaDexterousHandTactileWrench::timestamp\_us (C++ member), 74  
 XynovaDexterousHandTactileWrench::wrench (C++ member), 74  
 XynovaDexterousHandTactileWrenchStates::values (C++ member), 75  
 XynovaDexterousHandTactileWrenchStates (C++ struct), 75  
 XynovaDexterousHandTactileWrenchStates (C++ type), 65  
 XynovaDexterousHandTactileWrench (C++ struct), 74  
 XynovaDexterousHandTactileWrench (C++ type), 65  
 XynovaDexterousHandType (C++ type), 64

XynovaDexterousHandWarningFlags (C++ type), 63  
 XynovaDexterousHandWrenchDimension (C++ type), 64  
 xynovaErrorCodeToString (C++ function), 80  
 XynovaErrorCode (C++ type), 79  
 xynovaGetSdkBuildTime (C++ function), 88  
 xynovaGetSdkGitBranch (C++ function), 88  
 xynovaGetSdkGitCommit (C++ function), 88  
 xynovaGetSdkLocalVersion (C++ function), 88  
 xynovaGetSdkVersion (C++ function), 88  
 XynovaIdValue::data (C++ member), 86  
 XynovaIdValue::id (C++ member), 86  
 XynovaIdValueArray2::data (C++ member), 86  
 XynovaIdValueArray2::id (C++ member), 86  
 XynovaIdValueArray2 (C++ struct), 86  
 XynovaIdValueArray2 (C++ type), 82  
 XynovaIdValueArray3::data (C++ member), 86  
 XynovaIdValueArray3::id (C++ member), 86  
 XynovaIdValueArray3 (C++ struct), 86  
 XynovaIdValueArray3 (C++ type), 82  
 XynovaIdValueArray4::data (C++ member), 87  
 XynovaIdValueArray4::id (C++ member), 87  
 XynovaIdValueArray4 (C++ struct), 86  
 XynovaIdValueArray4 (C++ type), 82  
 XynovaIdValueArray5::data (C++ member), 87  
 XynovaIdValueArray5::id (C++ member), 87  
 XynovaIdValueArray5 (C++ struct), 87  
 XynovaIdValueArray5 (C++ type), 82  
 XynovaIdValueArray6::data (C++ member), 87  
 XynovaIdValueArray6::id (C++ member), 87  
 XynovaIdValueArray6 (C++ struct), 87  
 XynovaIdValueArray6 (C++ type), 82  
 XynovaIdValue (C++ struct), 86  
 XynovaIdValue (C++ type), 82  
 XynovaLogConsoleWriterConfiguration::base\_conf (C++ member), 96  
 XynovaLogConsoleWriterConfiguration::use\_stderr (C++ member), 96  
 XynovaLogConsoleWriterConfiguration (C++ struct), 96  
 XynovaLogConsoleWriterConfiguration (C++ type), 92  
 XynovaLogDateTimeRaw::microseconds (C++ member), 93  
 XynovaLogDateTimeRaw::seconds (C++ member), 93  
 XynovaLogDateTimeRaw (C++ struct), 93  
 XynovaLogDateTimeRaw (C++ type), 92  
 XynovaLogFileWriterConfiguration::base\_conf (C++ member), 96  
 XynovaLogFileWriterConfiguration::file\_path (C++ member), 96  
 XynovaLogFileWriterConfiguration::max\_file\_size (C++ member), 96  
 XynovaLogFileWriterConfiguration::max\_roll\_files (C++ member), 96  
 XynovaLogFileWriterConfiguration (C++ struct), 95  
 XynovaLogFileWriterConfiguration (C++ type), 92  
 XynovaLogFormatterConfiguration::reserved (C++ member), 96  
 XynovaLogFormatterConfiguration (C++ struct), 96  
 XynovaLogFormatterConfiguration (C++ type), 92  
 XynovaLogGlobalLogger\_getLogLevel (C++ function), 89  
 XynovaLogGlobalLogger\_init (C++ function), 89  
 XynovaLogGlobalLogger\_setLogLevel (C++ function), 89  
 XynovaLogLevel (C++ type), 92  
 XynovaLogLoggerBackendConfiguration::block\_size (C++ member), 96  
 XynovaLogLoggerBackendConfiguration::flush\_interval\_us (C++ member), 97  
 XynovaLogLoggerBackendConfiguration::max\_continuous\_repeated\_log (C++ member), 97  
 XynovaLogLoggerBackendConfiguration::message\_size (C++ member), 97  
 XynovaLogLoggerBackendConfiguration::pool\_size (C++ member), 96  
 XynovaLogLoggerBackendConfiguration::queue\_size\_multiple (C++ member), 97  
 XynovaLogLoggerBackendConfiguration (C++ struct), 96  
 XynovaLogLoggerBackendConfiguration (C++ type), 92  
 XynovaLogLoggerConfiguration::lb\_config (C++ member), 97  
 XynovaLogLoggerConfiguration::lb\_type (C++ member), 97  
 XynovaLogLoggerConfiguration::level (C++ member), 97  
 XynovaLogLoggerConfiguration::sink\_name (C++ member), 97  
 XynovaLogLoggerConfiguration::writer\_name\_list (C++ member), 97  
 XynovaLogLoggerConfiguration (C++ struct), 97

XynovaLogLoggerConfiguration (C++ type), 93  
 XynovaLogLogHandler (C++ type), 92  
 XynovaLogLogRecord::message (C++ member), 94  
 XynovaLogLogRecord::meta (C++ member), 94  
 XynovaLogLogRecordMeta::file (C++ member), 94  
 XynovaLogLogRecordMeta::func (C++ member), 94  
 XynovaLogLogRecordMeta::level (C++ member), 94  
 XynovaLogLogRecordMeta::line (C++ member), 94  
 XynovaLogLogRecordMeta::thread\_id (C++ member), 94  
 XynovaLogLogRecordMeta::timestamp (C++ member), 94  
 XynovaLogLogRecordMeta (C++ struct), 93  
 XynovaLogLogRecordMeta (C++ type), 92  
 XynovaLogLogRecord (C++ struct), 94  
 XynovaLogLogRecord (C++ type), 92  
 XynovaLogObjectLogger\_createConsoleWriter (C++ function), 90  
 XynovaLogObjectLogger\_createFileWriter (C++ function), 90  
 XynovaLogObjectLogger\_createFormatter (C++ function), 90  
 XynovaLogObjectLogger\_createLogger (C++ function), 90  
 XynovaLogObjectLogger\_createSinkTP (C++ function), 89  
 XynovaLogObjectLogger\_hasFormatter (C++ function), 91  
 XynovaLogObjectLogger\_hasLogger (C++ function), 91  
 XynovaLogObjectLogger\_hasSink (C++ function), 91  
 XynovaLogObjectLogger\_hasWriter (C++ function), 91  
 XynovaLogObjectLogger\_initDefaultObjects (C++ function), 89  
 XynovaLogObjectLogger\_isInitialized (C++ function), 89  
 XynovaLogObjectLogger\_setLoggerLevel (C++ function), 91  
 XynovaLogSinkConfiguration::block\_obtain\_timeout\_us (C++ member), 94  
 XynovaLogSinkConfiguration::max\_continuous\_push\_retry\_time\_ms (C++ member), 95  
 XynovaLogSinkConfiguration::max\_looping\_task\_idle\_interval\_us (C++ member), 95  
 XynovaLogSinkConfiguration::min\_looping\_task\_idle\_interval\_us (C++ member), 95  
 XynovaLogSinkConfiguration::writing\_task\_push\_timeout\_us (C++ member), 94  
 XynovaLogSinkConfiguration::writing\_task\_wait\_timeout\_us (C++ member), 95  
 XynovaLogSinkConfiguration (C++ struct), 94  
 XynovaLogSinkConfiguration (C++ type), 92  
 XynovaLogSinkTPConfiguration::base\_conf (C++ member), 95  
 XynovaLogSinkTPConfiguration::max\_queue\_size (C++ member), 95  
 XynovaLogSinkTPConfiguration::num\_threads (C++ member), 95  
 XynovaLogSinkTPConfiguration (C++ struct), 95  
 XynovaLogSinkTPConfiguration (C++ type), 92  
 XynovaLogWriterConfiguration::auto\_flush (C++ member), 95  
 XynovaLogWriterConfiguration::flush\_interval\_ms (C++ member), 95  
 XynovaLogWriterConfiguration::formatter (C++ member), 95  
 XynovaLogWriterConfiguration (C++ struct), 95  
 XynovaLogWriterConfiguration (C++ type), 92  
 XynovaResult::code (C++ member), 81  
 XynovaResult::message (C++ member), 81  
 XynovaResult (C++ struct), 80  
 XynovaResult (C++ type), 80  
 XynovaTypedValue::type (C++ member), 84  
 XynovaTypedValue::value (C++ member), 84  
 XynovaTypedValueArray2::type (C++ member), 85  
 XynovaTypedValueArray2::values (C++ member), 85  
 XynovaTypedValueArray2 (C++ struct), 84  
 XynovaTypedValueArray2 (C++ type), 81  
 XynovaTypedValueArray3::type (C++ member), 85  
 XynovaTypedValueArray3::values (C++ member), 85  
 XynovaTypedValueArray3 (C++ struct), 85  
 XynovaTypedValueArray3 (C++ type), 82  
 XynovaTypedValueArray4::type (C++ member), 85  
 XynovaTypedValueArray4::values (C++ member), 85  
 XynovaTypedValueArray4 (C++ struct), 85  
 XynovaTypedValueArray4 (C++ type), 82  
 XynovaTypedValueArray5::type (C++ member), 85  
 XynovaTypedValueArray5::values (C++ member), 85  
 XynovaTypedValueArray5 (C++ struct), 85  
 XynovaTypedValueArray5 (C++ type), 82  
 XynovaTypedValueArray6::type (C++ member), 86  
 XynovaTypedValueArray6::values (C++ member), 86

XynovaTypedValueArray6 (C++ struct), 85  
XynovaTypedValueArray6 (C++ type), 82  
XynovaTypedValue (C++ struct), 84  
XynovaTypedValue (C++ type), 81  
XynovaValue::f32 (C++ member), 84  
XynovaValue::f64 (C++ member), 84  
XynovaValue::s16 (C++ member), 84  
XynovaValue::s32 (C++ member), 84  
XynovaValue::s64 (C++ member), 84  
XynovaValue::s8 (C++ member), 83  
XynovaValue::u16 (C++ member), 84  
XynovaValue::u32 (C++ member), 84  
XynovaValue::u64 (C++ member), 84  
XynovaValue::u8 (C++ member), 83  
XynovaValue (C++ type), 81  
XynovaValue (C++ union), 83  
xynova (C++ type), 102, 104, 107, 111, 115, 122, 124, 129  
X (C macro), 79